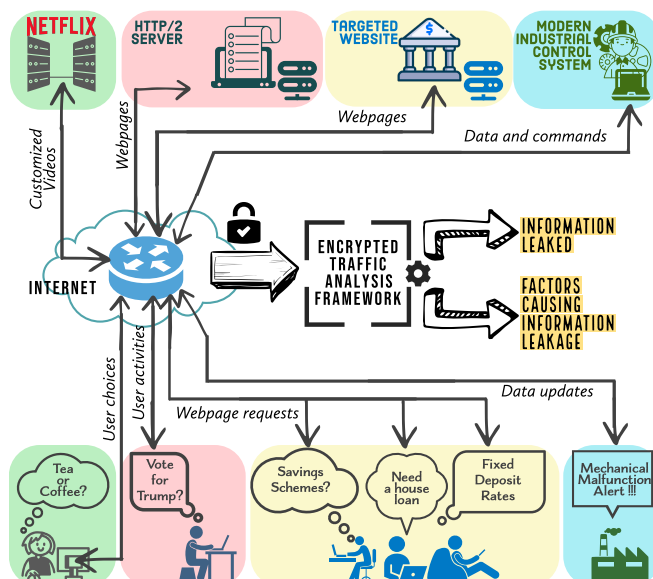


DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Encrypted Traffic Analysis based Cyber Surveillance and Possible Mitigation Techniques



A Thesis

Submitted by

GARGI MITRA

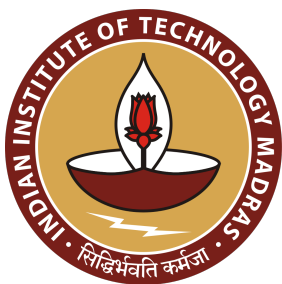
CS15D303

For the award of the degree

Of

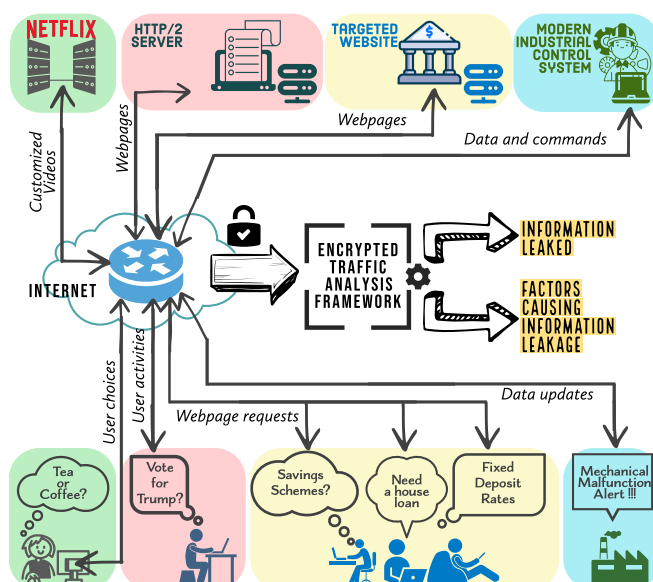
DOCTOR OF PHILOSOPHY

November 2022



DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Encrypted Traffic Analysis based Cyber Surveillance and Possible Mitigation Techniques



A Thesis

Submitted by

GARGI MITRA

CS15D303

For the award of the degree

Of

DOCTOR OF PHILOSOPHY

November 2022

This is just metadata

– Dianne Feinstein

(US Senator, California)

We kill people based on metadata

– General Michael Hayden

(Former Director, CIA and NSA)

*To my parents and teachers, who inspired me, encouraged me,
supported me, and evolved with me in this journey*

THESIS CERTIFICATE

This is to undertake that the Thesis titled **ENCRYPTED TRAFFIC ANALYSIS BASED CYBER SURVEILLANCE AND POSSIBLE MITIGATION TECHNIQUES**, submitted by me to the Indian Institute of Technology Madras, for the award of **Doctor of Philosophy**, is a bona fide record of the research work done by me under the supervision of **Dr. V. Kamakoti and Dr. Nitin Chandrachoodan**. The contents of this Thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Place: Chennai 600036

Gargi Mitra

Date: November 2022

CS15D303

Dr. V. Kamakoti

Research advisor

Professor

Department of Computer Science

& Engineering

IIT Madras

Dr. Nitin Chandrachoodan

Research co-advisor

Associate Professor

Department of

Electrical Engineering

IIT Madras

LIST OF PUBLICATIONS

I. LIST OF PAPERS BASED ON THESIS

Depending on HTTP/2 for Privacy? Good Luck!

Gargi Mitra, Prasanna Karthik Vairam, Patanjali SLPSK, Nitin Chandrachoodan, V. Kamakoti, in *Proceedings of IEEE/IFIP Conference on Dependable Systems and Networks (DSN) 2020*, pp. 278 - 285, DOI: 10.1109/DSN48063.2020.00044.

White Mirror: Leaking Sensitive Information from Interactive Netflix Movies using Encrypted Traffic Analysis

Gargi Mitra, Prasanna Karthik Vairam, Patanjali SLPSK, Nitin Chandrachoodan, V. Kamakoti, in *Proceedings of the ACM SIGCOMM 2019 Conference Posters and Demos*, pp. 122–124, DOI: 10.1145/3342280.3342330.

Snoopy: A Webpage Fingerprinting Framework with Finite Query Model for Mass-Surveillance

Gargi Mitra, Prasanna Karthik Vairam, Sandip Saha, Nitin Chandrachoodan, V. Kamakoti, *Published at the IEEE Transactions on Dependable and Secure Computing*, DOI: 10.1109/TDSC.2022.3222462.

Encrypted Traffic Analysis based Attack on a Modern ICS – A Case Study of a Leaky Water Treatment Plant

Gargi Mitra, Pritam Dash, Yingao Yao, Nitin Chandrachoodan, V. Kamakoti, Karthik Pattabiraman, *Under revision and to be submitted at ASIACCS 2022*.

II. LIST OF POSTERS RELATED TO THESIS

White Mirror: Leaking Sensitive Information from Interactive Netflix Movies using Encrypted Traffic Analysis

Gargi Mitra, Prasanna Karthik Vairam, Patanjali SLPSK, Nitin Chandrachoodan, V. Kamakoti, at *ACM SIGCOMM Conference, 2019*

Hastakshara: A passive side channel based webpage fingerprinting attack for uncovering client intent.

Gargi Mitra, Prasanna Karthik Vairam, Nitin Chandrachoodan, V. Kamakoti, at *ACM Internet Measurement Conference (IMC), 2017*

III. AWARDS AND RECOGNITIONS

Mitacs Globalink Research Award (2021), sponsored by Mitacs and Shastri Indo-Canadian Institute, for conducting collaborative research with Canada.

Applied Research Competition (2020) (First prize) at Cyber Security Awareness Week (CSAW), organized by New York University and IIT Kanpur.

Selected as one of the top 5 contestants of the technical blog writing competition at Maitreyee 2020, IBM Research-India's annual women outreach event.

AWSAR top 100 Award (2019), Awarded for being among the top 100 contestants in a national scientific story-writing competition organized by the Department of Science & Technology, Govt. of India, as part of the AWSAR (Augmenting Writing Skills for Articulating Research) initiative.

Gandhi Hazare Award (2018) (First prize), awarded by the Centre for Social Innovation and Entrepreneurship, IIT Madras, for proposing technical solutions to fight corruption in India.

TCS Poster Competition Winner (2018) (Second prize), awarded by TCS Research India, for presenting 'Ankapala: The Account Book for Fighting Corruption in Public Distribution System'.

Embedded Security Challenge Winner (2017) (First prize) at Cyber Security Awareness Week, organized by NYU and IIT Kanpur, for presenting 'Eradicator: An Integrated Approach for Defense against Cyber Attacks in PLC based Industrial Control Systems'.

IV. OTHER PUBLICATIONS

Towards Measuring Quality of Service in Untrusted Multi-Vendor Service Function Chains: Balancing Security and Resource Consumption

Prasanna Karthik Vairam, Gargi Mitra, Vignesh Manoharan, Chester Rebeiro, Byrav Ramamurthy, Kamakoti Veezhinathan, in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 163-171.

ApproxBC: Blockchain Design Alternatives for Approximation-Tolerant Resource-Constrained Applications

Prasanna Karthik Vairam, Gargi Mitra, Chester Rebeiro, Byrav Ramamurthy, Kamakoti Veezhinathan, in *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 45-51, September 2018.

ACKNOWLEDGEMENTS

It was the summer of 2015 when I joined IIT Madras, as a starry-eyed Ph.D. student. The following years have been nothing short of an incredible journey, full of challenges, lessons, joy, and excitement. This thesis would be incomplete without me acknowledging the contributions of all who have made this journey worth remembering.

I would start by expressing my deepest regards to my thesis advisors, Dr. V. Kamakoti and Dr. Nitin Chandrachoodan, who have played an indispensable role in molding my research skills during my Ph.D. The unwavering belief that Dr. Kamakoti showed in me from the beginning gave me a much-needed sense of self-esteem. He has always encouraged me to pursue my research interests independently and fearlessly while providing all the support and advice necessary for doing so. Dr. Nitin Chandrachoodan has been the most supportive co-advisor, whose suggestions and continuous feedback have always helped me understand the concerns and perspectives of a broader research community. Their whole-hearted appreciation and constant motivation, both during my moments of success and failure, kept me going through the highs and lows of my Ph.D. journey. I also thank my Doctoral Committee members - Dr. P. Sreenivasa Kumar (Chairperson), Dr. Chester Rebeiro, Dr. R Manivasakan, and Dr. Karthik Raman, for providing crucial feedback about my work from time to time, and for helping me resolve any issue that came in the way of my progress. During my Ph.D. program, I got the opportunity of being mentored by Dr. Karthik Pattabiraman (UBC-V), who not only provided valuable guidance for one of my research problems but also played the role of a wonderful host during my research internship at UBC. I also thank Dr. Vyas Sekar (CMU) for his critical comments that often helped me improve the practical aspects of my research work. Thanks to all the other faculty members and staff of IIT Madras, who have enriched my Ph.D. journey and overall experience at IITM in various ways.

I must also acknowledge the role of my friends and colleagues, who have shared my

personal and professional joys and sorrows during the last few years. Many of the research ideas that have been presented in this thesis were born out of intense brainstorming sessions with Prasanna Karthik and Patanjali SLPSK. I am thankful to Prasanna especially for being an ideal collaborator, and for instilling in me the values of research ethics and professional integrity. I would also like to thank Sandip Saha (IITM), Pritam Dash (UBC-V) and Yingao Yao (UBC-V) for their productive collaboration on various research problems. The IITM campus has now become my second home, thanks to my wonderful friends and lab mates, who have helped me in innumerable ways, both academically and beyond. At the risk of missing out few names, I would like to mention Sritama, Ahana, Prasanna, Patanjali, Arnab, Prithwish, Keerthi, Gnanambikai, Indrani, Himanshi, Akriti, Madhura, Preksha, Sugandha, Sareena, and Nikhilesh. I am also immensely thankful to my friends back home who stayed connected with me daily and made me a part of their lives, even when the academic pressure would keep me from fulfilling my personal obligations. Thanks to Rajasee, Suvanwita, Parami, Deepsikha, Sanjana, Piyali, and Priyanka for providing me with immense mental support and motivation.

I would not have had the courage to embark on the path of higher studies, had it not been for Dr. Dipak Kole (my undergraduate mentor) and Mr. Goutam Banerjea (Director, St. Thomas' College of Engg. & Tech). Dr. Dipak Kole often went far beyond the line of duties to teach me advanced research topics during my undergrad years, helped me identify my passion, and provided all the support necessary for pursuing it. I am thankful to Mr. Banerjea for his constant encouragement and professional counseling during my undergrad days. I am also thankful to all my teachers from my school and college for empowering me with the education that was necessary for following my passion.

Last but not the least, I cannot be grateful enough to my parents, Mrs. Mita Mitra and Mr. Arup Kumar Mitra, for having my back at all times, giving me a strong educational foundation, providing me incredible mental support, and lastly, for understanding and respecting my wishes even if that required going against societal norms.

ABSTRACT

KEYWORDS Encrypted traffic analysis, network surveillance, interactive Netflix videos, HTTP/2 privacy, mass-scale surveillance, webpage fingerprinting, modern industrial control systems

Encrypted traffic analysis (ETA) is a novel form of network surveillance, where the surveillant relies on encrypted traffic metadata for inferring sensitive information about their targets. Existing research works in this area have focused on designing ETA-based surveillance techniques for users of conventional web-based applications. However, most modern web-based applications are yet to be explored in light of ETA-based surveillance. Compared to conventional applications, modern web-based applications have a much more diverse user base; transmit huge volumes and various types of sensitive information over the internet (e.g., the results of an online political survey); and use contemporary communication techniques and protocols. Therefore, it is both crucial and challenging to investigate if modern web applications are vulnerable to ETA-based surveillance, from the perspective of different types of internet users.

In this thesis, we explore four scenarios in which we investigate the vulnerability of modern web applications to ETA-based surveillance. First, we consider a scenario where a network surveillant aims to infer the choices made by a targeted viewer of Netflix interactive videos, which are often sensitive to the viewer. We show that the surveillant can achieve this by exploiting the metadata of encrypted HTTP/1.1 traffic of Netflix. For instance, the surveillant can infer the choice made by a targeted viewer in response to the question ‘How about you decide what you want for your breakfast?’ in the movie ‘Black Mirror: Bandersnatch’. In the second work, we disprove the popular hypothesis that the HTTP/2 protocol can be used as an effective countermeasure against existing ETA-based surveillance techniques. For this, we consider a scenario where a network surveillant aims to infer the browsing activities of a targeted user of an HTTP/2-based website. We show

that a network surveillant can achieve this by preventing an HTTP/2 server to operate in multi-threaded mode, i.e., the operational mode that was speculated to provide resilience against ETA-based surveillance. In the third work, we investigate if a surveillant can use ETA-based techniques for conducting mass-scale surveillance on a large number of users of a targeted website, as compared to the previous ETA techniques, which target specific users. Mass-scale surveillance is challenging due to the sheer number of combinations of user behavior and preferences that the surveillant needs to account for. We present Snoopy, a traffic analysis framework that can predict fine-grained browsing activities of a large number of users of a targeted website. Such mass-scale surveillance can provide fine-grained insights to a surveillant about the business performance of the organization hosting the targeted website. In our final work, we consider a scenario where a network attacker aims to cause damage to an internet-connected industrial control system (ICS) that uses encrypted control communications. An attacker can achieve this by disrupting sensitive control communications of the targeted ICS. However, contrary to publicly deployed websites, the behavior of an ICS is invisible to surveillants outside its perimeter, which makes ETA-based ICS attacks challenging. To this end, we design the first ETA-based attack on encrypted ICS traffic, which can distinguish the sensitive ICS communications from the other non-sensitive ICS communications.

We demonstrated the proposed traffic analysis techniques on real websites and a realistic ICS testbed. We also suggest countermeasures against the proposed techniques based on our observations.

CONTENTS

	Page
ACKNOWLEDGEMENTS	i
ABSTRACT	iii
LIST OF TABLES	ix
LIST OF FIGURES	xi
ABBREVIATIONS	xiii
CHAPTER 1 INTRODUCTION	1
1.1 Perils of network surveillance	3
1.1.1 Government Surveillance	3
1.1.2 Business Surveillance	4
1.1.3 Adversarial Surveillance	4
1.2 Traffic Encryption as a Countermeasure Against Network Surveillance .	4
1.3 Encrypted Traffic Analysis	5
1.4 Objective of this Thesis	7
1.5 Contributions of the Thesis	9
1.6 Outline of this Thesis	13
CHAPTER 2 BACKGROUND	17
2.1 Network Surveillant and its capabilities	17
2.1.1 Position of the network surveillant / adversary	17
2.1.2 Capabilities of the adversary	19
2.1.3 Assumptions and scope of this thesis	20
2.2 Secure Communication Protocols Used by Modern Internet-connected Systems	20
2.2.1 Interactive Netflix video streaming	21
2.2.2 Web browsing traffic of internet users	22
2.2.3 Industrial Control System Traffic	23
2.3 Encrypted Traffic Metadata: An Adversarial Perspective	24
2.4 Summary	26
CHAPTER 3 WHITE MIRROR: LEAKING SENSITIVE INFORMATION FROM INTERACTIVE NETFLIX MOVIES USING ENCRYPTED TRAFFIC ANALYSIS	29
3.1 Related Work	31
3.2 Streaming Process for Interactive Netflix Videos	32

3.3	Encrypted Traffic Analysis (Technique-1)	33
3.3.1	Dataset and Results	34
3.3.2	Impact of our work and response from Netflix	35
3.4	Interactive Video Streaming Process of Netflix (2019 - Present)	36
3.5	Traffic Analysis on Present Streaming Process (Technique-2)	38
3.5.1	Results	39
3.5.1.1	Failure Analysis	40
3.6	Summary and Future Work	41

CHAPTER 4 DEPENDING ON HTTP/2 FOR PRIVACY? GOOD LUCK! 45

4.1	Background	46
4.2	Preliminary Concepts	49
4.2.1	HTTP/2 Multiplexing	50
4.2.2	Privacy of HTTPS Communications and Related Works	51
4.2.3	Network Parameters	52
4.2.4	Measuring Privacy	53
4.3	High Level Overview	53
4.3.1	Capability of Adversary	55
4.3.2	Assumptions and Scope	56
4.4	Impact of Network Parameters on HTTP/2 Multiplexing	57
4.4.1	Delay	58
4.4.2	Jitter	58
4.4.3	Bandwidth Limitation	59
4.4.4	Targeted Packet Drops – forcing HTTP/2 Stream Reset	62
4.5	The Privacy Attack	65
4.5.1	Target Website	65
4.5.2	Client setup	66
4.5.3	Adversary Setup	66
4.5.4	Results	67
4.5.5	Ethical Considerations	67
4.6	Countermeasures and Future Work	68
4.6.1	Improving the Adversary	68
4.6.2	Exploring other types of web traffic	68
4.6.3	Improving HTTP/2 privacy	69
4.7	Summary	69

CHAPTER 5 SNOOPY: A NOVEL WEBPAGE FINGERPRINTING FRAMEWORK FOR MASS-SURVEILLANCE 71

5.1	Background	72
5.2	Related Work	78
5.3	The Snoopy Framework	82
5.3.1	Adversary Capabilities	83
5.3.2	Assumptions and Scope	83
5.3.3	High Level Overview	83
5.4	Snoopy Design: Predictability of Fingerprints	86

5.5	Snoopy Design: A Deep Dive	90
5.5.1	Webpage Profiling Module	90
5.5.1.1	Extracting Structure of the Website	91
5.5.1.2	Focused Traffic Sample Collection and Formulating Signatures of Web Resources	92
5.5.1.3	Constructing Feature and Resource Databases	93
5.5.2	Webpage Prediction Module	97
5.5.2.1	Predicting Web Resource Sequence	97
5.5.2.2	Predicting Webpages Accessed by the User	101
5.6	Implementation and Evaluation	102
5.6.1	Implementation	102
5.6.2	Metrics	103
5.6.3	Experimental setup	105
5.6.4	Results	106
5.6.4.1	Average prediction accuracy of Snoopy across diverse browsing scenarios	107
5.6.4.2	Compliance with a finite query model: Snoopy vs. ML-based solutions	108
5.6.4.3	Generalization across User Interests	109
5.6.4.4	Generalization across Browser Configurations	111
5.6.4.5	Support for prediction on multi-tab browsing traffic	112
5.6.4.6	Generalization across various Operating Systems	114
5.6.4.7	Generalization across various Browsers	115
5.6.4.8	Generalization across network conditions	116
5.6.4.9	Resource-level prediction accuracy of Snoopy	116
5.6.4.10	Relation between resource prediction accuracy and webpage prediction accuracy	117
5.6.5	Snoopy-ML Ensemble	119
5.6.5.1	Experiment 1	119
5.6.5.2	Experiment 2	120
5.7	Countermeasures	121
5.8	Summary	122

CHAPTER 6 NOT JUST ANOTHER PRIVACY LEAK: AN ACTIVE ETA-BASED ATTACK ON A *LEAKY* WATER TREATMENT PLANT **125**

6.1	Background	126
6.2	Preliminary Concepts	132
6.2.1	Modern Industrial Control Systems (ICS)	132
6.2.1.1	ICS and its components	132
6.2.1.2	Communications in a Modern ICS	133
6.2.2	Related Works	135
6.2.2.1	Network Perturbation-based ICS Attacks	135
6.2.2.2	Challenges in performing ETA attack on ICS	136
6.2.2.3	ETA-based Active Attacks	136
6.3	Proposed Adversarial Scheme	137
6.3.1	Attacker Capabilities and Assumptions	137
6.3.2	Attack overview	138

6.3.3	Encrypted Traffic Analysis Techniques	140
6.3.3.1	Scenario 1 - ICS design and implementation details are known to adversary:	140
6.3.3.2	Scenario 2 - ICS design and implementation details are NOT known to adversary:	144
6.4	A Case Study	148
6.4.1	Experimental Setup	148
6.4.1.1	Test-bed.	148
6.4.1.2	Targeted process.	150
6.4.1.3	Adversary setup	151
6.4.2	Evaluation Metrics	152
6.4.3	Results	152
6.4.3.1	Intermittent random network perturbations (No traffic analysis) . .	153
6.4.3.2	Scenario 1 - ICS design and implementation details known to adversary:	156
6.4.3.3	Scenario 2 - ICS design and implementation details NOT known to adversary:	160
6.5	Possible Countermeasures	162
6.6	Discussions	164
6.7	Summary	166
CHAPTER 7 CONCLUSION		167
7.1	Implications and Potential of this Thesis	168
7.2	Limitations and Future Work	169
REFERENCES		173
CURRICULUM VITAE		189
DOCTORAL COMMITTEE		191

LIST OF TABLES

Table	Caption	Page
3.1	Attributes of the dataset	34
4.1	Effect of jitter on HTTP/2 multiplexing	61
4.2	Prediction Accuracy: O_{prev} and O_{next} denote the objects requested by the client immediately before and after O_{curr} respectively. $T(\text{Req } X)$ denotes the time of request for object X by the client.	67
5.1	Compliance of webpage fingerprinting techniques with the requirements for mass-surveillance	79
5.2	Information fetched from the Snoopy Database for resource prediction	97
5.3	Characteristics of Profiled Websites RS - Retail Store website, SBC - Service Based Company website, IC - Insurance Company website, B - Bank website, PS - Political Survey website	104
5.4	Webpage prediction accuracy of Snoopy	108
5.5	Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different browser configuration (for 200 webpages).	111
5.6	Snoopy vs. Existing works: Number of website accesses required for fingerprinting in multi-tab browsing scenario	113
5.7	Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different operating system (for 95 webpages).	114
5.8	Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different browser (for 95 webpages).	115
5.9	Prediction accuracy of resources in bot traces [#] The number indicates the resources that are of interest to the adversary ^{##} Accuracy computed with respect to the number of resources of interest	117
6.1	Details of random network perturbations and their impact on network traffic of the ICS set-up	153
6.2	Summary of attacks executed by proposed adversary on our test-bed and their impact	154

LIST OF FIGURES

Figure	Caption	Page
1.1	Contributions of the thesis in the area of encrypted traffic analysis of modern communication systems	11
1.2	Application of the proposed ETA framework in developing secure web applications / communication protocols	12
2.1	Positions of the network surveillant / adversary	18
2.2	Examples of information inferred by an ETA-based adversary from network traffic metadata	24
3.1	The streaming process of Bandersnatch	32
3.2	SSL record lengths of different types of JSON files for two different browsing contexts – (Desktop, Firefox, Ethernet, Ubuntu) and (Desktop, Firefox, Ethernet, Windows)	35
4.1	Estimating object sizes from encrypted traffic in non-multiplexed (i.e., O_2 after O_1) vs. multiplexed object transmissions (i.e., $O_1Seg_1 \rightarrow O_2Seg_1 \rightarrow O_1Seg_2 \rightarrow O_2Seg_2$)	49
4.2	High level overview of proposed adversarial scheme	54
4.3	Interleaved object transmission by an HTTP/2 server	57
4.4	Effect of network jitter on multiplexing of objects in HTTP/2	60
4.5	Effect of bandwidth limitation on multiplexing of objects in HTTP/2	61
4.6	Targeted packet drop by adversary to force non-multiplexed object transmission in HTTP/2	64
5.1	Snoopy, the proposed framework for webpage identification attack on a mass-scale	73
5.2	The components of Snoopy	84
5.3	Representation of the structure of a sample website	92
5.4	Constructing featureDB and reverseFeatureDB from TSamples and FS	95
5.5	Steps involved in selecting resources that might correspond to a given feature value	98
5.6	Importance of training set size N_q on fingerprinting accuracy FA (in %) of existing ML models vs. Snoopy (for 200 web pages)	109
5.7	Fingerprinting accuracy FA (in %) of Snoopy vs. existing techniques for different number of webpages of interest	110
5.8	Fingerprinting accuracy (FA) of Snoopy in multi-tab browsing scenario (for website B_4)	112
5.9	Webpage prediction by Snoopy-ML ensemble	120
6.1	Modern ICS communications and emergence of new threat surface in the form of Internet communications.	134
6.2	Overview of proposed adversarial scheme	139

6.3	Process analysis and message fingerprinting	140
6.4	High-level overview of the emulated SWaT test-bed	149
6.5	Schematic representation of Sub-process 1 (Raw Water Storage) and operational modes of PLC1	150
6.6	Effect of intermittent network perturbations caused by a network adversary (X: T1 cross-section)	155
6.7	Effect of network perturbations by the adversary in Scenario 1, during transition of PLC1 from Mode-1 to Mode-2 (X: T1 cross-section) . . .	157
6.8	Effect of adversarial network perturbations during transition of PLC1 from Mode-3 to Mode-2 (Scenario 1)	158
6.9	Legend for Figure 6.7 and Figure 6.8	158
6.10	Effect of packets dropped by the adversary in Scenario 2 (X: T1 cross- section)	161

ABBREVIATIONS

ACK	Acknowledgement.
BCE	Before Common Era.
BDP	Bandwidth Delay Product.
CCTV	Closed-circuit Television.
CIP	Common Industrial Protocol.
CPU	Central Processing Unit.
DDoS	Distributed Denial of Service.
DL	Deep Learning.
DNS	Domain Name Server.
DoS	Denial of Service.
DPI	Deep Packet Inspection.
ENIP	Ethernet Industrial Protocol.
ETA	Encrypted Traffic Analysis.
HMI	Human-Machine Interface.
HTML	Hypertext Markup Language.
HTTP	Hypertext Transfer Protocol.
HTTPS	Hypertext Transfer Protocol Secure.
ICS	Industrial Control System.
IDS	Intrusion Detection System.
IoT	Internet of Things.
IP	Internet Protocol.
IPSec	IP Security.

ISP	Internet Service Provider.
IT	Information Technology.
JSON	JavaScript Object Notation.
Mbps	Megabits per second.
MDNS	Multicast DNS.
MITM	Man-in-the-middle.
ML	Machine Learning.
MTU	Maximum Transmission Unit.
MW	MegaWatts.
NAT	Network Address Translation.
OS	Operating System.
OT	Operations Technology.
PLC	Programmable Logic Controller.
RTT	Round trip Time.
SCADA	Supervisory Control And Data Acquisition.
SSL	Secure Sockets Layer.
SUTD	Singapore University of Technology and Design.
SWaT	Secure Water Treatment.
TCP	Transmission Control Protocol.
TLS	Transport Layer Security.
Tor	The Onion Router.
URL	Uniform Resource Locator.
USA	United States of America.
VM	Virtual Machine.
VPN	Virtual Private Network.

CHAPTER 1

INTRODUCTION

Surveillance has been used by powerful rulers and governments for ages for the purpose of collecting personal information about targeted sections of the population. The earliest known instances of surveillance date back to as early as the 4th century BCE, when Indian and Chinese emperors relied on in-person espionage for their military intelligence (Yuen, 2014; Mazhar Abbas, 2021). The target of the surveillance could vary from a high-profile individual to a group of ordinary citizens. Examples of personal information collected through surveillance include communication records, financial information, social and economic status, political alignment and ties, and relationships among various sections of the population.

Surveillants can harvest and use the collected information in multiple ways. Especially, countries at war have found such personal information to be very useful for understanding the temperament of both their allies and enemies and watching out for possibilities of any secret conspiracies against them. For instance, surveillance was extensively used by the USA during the First World War for identifying domestic insurgents (BBC, 2019). Even today, surveillance is widely used by developed nations for governance and in warfare ¹. Over the years, surveillance techniques have become increasingly sophisticated and evolved from labor-intensive in-person espionage to wiretapping (1895) (Pollak, 2015), monitoring radio communications (circa 1917) (United States Army, 1991), followed by digital techniques (circa 1960) (Spillane, 2022), and cyber surveillance (circa 1986) (SentinelOne, 2019; Edwards, 2020).

Today, with the internet penetrating almost all aspects of human lives, *cyber surveillance* is being seen by surveillants as a powerful tool for gathering information about their

¹Source: https://en.wikipedia.org/wiki/List_of_government_mass_surveillance_projects

targets. Cyber surveillance systems are designed to capture the enormous volume of digital footprints left behind by the daily activities (e.g., banking, shopping, education, and entertainment) and communications (e.g., social media interactions, email communications, and video conferences) of internet users from internet-connected systems. In this age of information, these digital footprints provide valuable insight to the surveillants, which could be crucial for understanding the personal preferences and building behavioral profiles of targeted individuals, as well as for inferring the nature of relationships among people. Needless to say, surveillants also get access to a huge volume of communication records containing confidential information (e.g., login credentials of social media accounts) about their targets. Cyber surveillance systems can collect these information *remotely* by compromising and snooping on internet-connected systems such as end-user devices (for e.g., smartphones, IoT devices, laptops) using spyware (Kaspersky, 2018), web servers and data centers (Klein, 2019), and network devices (for e.g., routers, firewalls) (PrivacyPolicies.com, 2022). Unlike earlier techniques (Cybersecurity and Infrastructure Security Agency, 2019), cyber surveillance techniques do not require the surveillants to physically access their target's devices, which makes the techniques stealthier. For instance, today, a surveillant can remotely install a keylogger software on the system of their target to monitor their keystrokes from across the world (Malwarebytes, 2022). The Snowden leaks of 2013 (Macaskill and Dance, 2013) exposed the extent to which global-scale cyber surveillance is still conducted by powerful nations, often unknown to their citizens.

Network traffic surveillance (Diffie and Landau, 2008), a form of cyber surveillance technique, involves intercepting or eavesdropping on internet communications from a network device (for e.g., router, firewall) that is located on the *path of communication* of the targeted internet user. Unlike other forms of cyber surveillance techniques (for e.g., keylogging), network traffic surveillance does not always require intruding on the end-user devices or the web servers (targets of surveillance). This helps the surveillant remain stealthy from its targets for long periods of time. For instance, surveillants such

as the Government might collaborate with local Internet Service Providers (ISPs) for eavesdropping on the internet communications of the citizens. Today, Governments use network surveillance for enabling censorship ², detecting malware, snooping on confidential communications, extracting behavioral profiles of targeted people from their online communications, as well as in cyber warfare (Robinson *et al.*, 2015; Fortinet, n.d.).

1.1 PERILS OF NETWORK SURVEILLANCE

So far we have highlighted the widespread use of surveillance by Governments for the benefits of their citizens. However, surveillance can also be conducted by people with malicious or unethical intentions, which can have devastating consequences on the targeted section of the population.

1.1.1 Government Surveillance

Surveillance programs conducted by Governments have always remained a controversial topic (Liaropoulos, 2016; Königs, 2022). This is largely due to the lack of unanimous trust in the Government by their citizens, lack of transparency regarding the information collected, and scepticism regarding the use of the collected information. For instance, the citizens of Singapore were highly concerned when the Singapore Government revealed that surveillance data collected for tracking Covid cases have been shared with the police for aiding criminal investigations (Illmer, 2021). Furthermore, citizens get even more concerned when the surveillant is stealthy in nature, as in the case of network surveillance. For instance, for around a year, the citizens of Kazakhstan were not aware that their Government had been colluding with a major ISP and intercepting all their web communications (Raman *et al.*, 2020).

²Source: https://en.wikipedia.org/wiki/Internet_censorship_and_surveillance_by_country

1.1.2 Business Surveillance

Network surveillance is not only beneficial to Governments, but also to private entities such as business organizations. Large business organizations are often interested in monitoring the web traffic directed to the websites of their competitors (Semrush, n.d.). This is because, an analysis of the traffic volume patterns provides valuable insight to these organizations about their performance of their competitor, and help them understand the needs and preferences of their targeted consumer groups.

1.1.3 Adversarial Surveillance

Network surveillance is also widely used by cyber attackers for causing financial losses to organizations, or for disrupting the lives of citizens for their personal benefits (Symantec, 2011). Cyber attackers often use network surveillance as a precursor to more impactful attacks. For instance, a cyber attacker might monitor the volume of traffic directed to a banking server over a period of time to understand the peak load time of the server, so that it can launch a denial-of-service attack when the server is the busiest. This would help the attacker maximize the impact of the attack.

Due to the aforementioned concerns, network surveillance is being seen as a serious security and privacy threat by internet users today. This has fuelled the need for protecting information communicated over the internet.

1.2 TRAFFIC ENCRYPTION AS A COUNTERMEASURE AGAINST NETWORK SURVEILLANCE

Encryption of internet traffic started in 1995 when Netscape Communications Corporation developed and popularized the use of Secure Sockets Layer (SSL) protocol (Hickman, 1995), primarily for protecting only highly confidential communications, such as credit card transactions. Encryption of network traffic ensured that the confidential information of internet users are hidden from network surveillants eavesdropping on on-path network devices. Traffic encryption was complimented by

strong endpoint security (Tan *et al.*, 2021), device authentication (Heo *et al.*, 2008) and service authorization techniques (Ziebermayr and Probst, 2004), which prevented network surveillants from decrypting the network traffic or accessing the information in plaintext directly from the end user devices.

Over time, as the number of internet-connected products and services kept growing, communication of personal information over the internet became increasingly common. This made the internet users feel a growing need for encrypting almost all types of web traffic (such as emails and web browsing). Consequently, there has been a steady increase in the percentage of encrypted web traffic over recent years. While only 53% of the internet traffic was encrypted as of 2016 (Meeker, 2019), it has soared to approximately 95% as of January 2022 (Google, 2022). Keeping up with technological advancements, security experts also continue to develop and deploy strong encryption algorithms that cannot be easily broken by unauthorized surveillants. Presently, Transport Layer Security v1.3 (TLS 1.3) (Rescorla, 2018) is the latest version of the most widely used encryption standard for internet traffic.

While traffic encryption obfuscates the application data from network surveillants, it might not be a complete solution against network surveillance. This is because, traffic encryption does not obfuscate the traffic metadata. This includes the unencrypted packet headers and traffic flow characteristics. In this thesis, we investigate some of the concerns that might arise due to the exposure of traffic metadata to network surveillants.

1.3 ENCRYPTED TRAFFIC ANALYSIS

For a long time, encryption of network traffic was considered to be a foolproof solution to network surveillance. However, in recent times, a novel form of network surveillance technique, known as *Encrypted Traffic Analysis* (ETA) (Wang *et al.*, 2014), has been developed. The first reported case of large-scale ETA-based surveillance is the PRISM surveillance program launched in 2007 by the National Security Agency of the

USA (Sottek and Kopfstein, 2013). This form of network surveillance does not require compromising the decryption key or breaking the encryption. Instead, the surveillant only relies on the *metadata* of encrypted traffic communicated by the targeted Internet user. Examples of such metadata include source and destination IP addresses, packet sizes, TCP port numbers, and inter-packet timings. Examples of network devices that a surveillant can compromise include switches and routers that are misconfigured, have open ports, or hve default login credentials (Matherly, 2022). Once the surveillant records the traffic metadata of its target, it subsequently analyzes these metadata using statistical and machine learning techniques to reveal *web access patterns* that are indicative of activities or behavior of targeted internet users and internet-connected systems.

The Snowden leaks revealed the rampant use of encrypted traffic metadata by cyber surveillants, without the knowledge and consent of the targeted internet users (Sottek and Kopfstein, 2013). At that point, however, the exact techniques and tools used by these surveillants for analyzing traffic metadata were largely unknown. This prompted security researchers to dig deeper and start exploring the potential of traffic metadata in exposing sensitive information. Since then, encrypted traffic analysis has remained a highly researched field, especially because of the following two reasons - (i) today, people use the internet for a much wider range of purposes than they used to a decade ago. To support the needs of internet users, the number of websites, apps, internet-connected services and products, and web communication protocols has also been growing rapidly. Consequently, the volume and diversity of information transferred over the internet is much higher today as compared to a decade earlier, and; (ii) internet users are much more concerned about their online privacy, and aware of their right to privacy than they were in the early 2000s. Therefore, there is an ever-growing need for analyzing encrypted traffic metadata generated by different types of web applications to learn what information they might leak to network surveillants.

1.4 OBJECTIVE OF THIS THESIS

The primary objective of this thesis is to answer the question: *‘Is encryption enough for protecting sensitive user information in modern communication systems from surveillants on the internet? If not, what countermeasures can possibly be adopted to prevent leakage of information on the internet?’*

Specifically, we attempt to answer the aforementioned questions from the perspective of different groups of internet users and in the context of different types of modern internet-connected systems/applications that have *not been widely explored* by the research community yet. Following are the specific objectives of this thesis.

1. The first objective of this thesis is to investigate if choices made by viewers while watching interactive videos are susceptible to ETA-based surveillance. Specifically, we examine the encrypted traffic of interactive Netflix movies (Netflix, 2022a), which are streamed using modern state-of-the-art *interactive video streaming* techniques. Interactive videos on Netflix enables viewers to create a story-line of their choice for a given movie title (for e.g., Black Mirror: Bandersnatch). For this, Netflix uses a novel video streaming technique, where it transmits video chunks depending on choices made by the viewer at certain points of time while watching the movie. Leakage of the choices is concerning for Netflix users, especially when the video contents are of sensitive nature. This is because studies (Golbeck and Norris, 2013; Nave *et al.*, 2020) have shown that the viewing habits of Netflix users are indicative of their fine-grained personal preferences and behavioral traits, which might be highly confidential to the users.
2. The second objective of this thesis is to investigate if the *Hypertext Transfer Protocol v2.0* (HTTP/2), when combined with state-of-the-art encryption standards, is a dependable countermeasure against ETA-based surveillance. HTTP/2 (Belshe *et al.*, 2015) is an advanced version of the HTTP/1.1 web communication protocol, that has been designed for enhancing the performance of websites. To facilitate

this, unlike its predecessor, HTTP/2 supports multi-threaded object transmission by web servers. Recent research works have shown that the multi-threaded object transmission feature also unexpectedly improved its privacy against ETA-based surveillance. Our goal is to analyze the encrypted browsing traffic of users of HTTP/2 websites to understand if they leak browsing information of the website users to ETA-based surveillants.

3. The third objective of this thesis is to investigate if a network surveillant can conduct *mass-scale surveillance* ³ using ETA-based techniques. So far, ETA research works have been successful in predicting the activities of only a small number of targeted website visitors. However, these works have not been successful in demonstrating mass-scale surveillance capabilities. Surveillance on the browsing activities of a large number of visitors of a targeted website is extremely important for a surveillant, since it can reveal intricate details about the success of different products/services/policies of the organization hosting the targeted website. This information, if leaked to its competitors, might have detrimental effects on the organization. Our goal is to specifically investigate if the Secure Hypertext Transfer Protocol (HTTPS) is sufficient for preventing the leakage of mass-scale web browsing information of a website.
4. The fourth objective of this thesis is to analyze the security provided by encryption to the *control traffic of Industrial Control Systems (ICS)*. An ICS is an automated cyber-physical system that controls the operations of an industrial process. For this, different cyber-physical components (for e.g., computational devices, temperature sensors, chemical dispensers) of the ICS need to communicate with each other. Till a few years back, these ICS control communications took place over private networks isolated from the internet. However, in recent years, industries have been interfacing these control communications with the internet to facilitate

³Refer to https://en.wikipedia.org/wiki/Mass_surveillance

setting up ICS networks that can be geographically distributed and remotely monitored (Andreeva *et al.*, 2016). For protecting these communications from network surveillants, industries have been using the same encryption standard, i.e., Transport Layer Security (TLS) protocol, that is used for protecting web browsing traffic (Rescorla, 2018; Rinaldi, 2019; Schneider Electric USA, Inc., 2018). However, ICS communications have a specific set of requirements as compared to web browsing communications - for instance, ICS communications are much more time-sensitive and less tolerant to packet loss. A cyber attacker can exploit these properties of ICS communications to cause damages to a targeted ICS by disrupting its sensitive communications. Our goal is to investigate if state-of-the-art encryption standards are sufficient for protecting ICS communications from such attackers.

1.5 CONTRIBUTIONS OF THE THESIS

The main contributions of this thesis are as follows:

1. We present a first-of-its-kind encrypted traffic analysis (ETA) technique which shows that choices made by viewers of interactive Netflix videos can leak from the encrypted streaming traffic. This discovery is important, since Netflix continues to use the same technology for developing more interactive videos, and plans to use the technology for designing other products in the future. We also present the first encrypted traffic dataset for analysis of interactive Netflix video traffic. This dataset was used to validate our findings, and could also be used for future research works in the area of encrypted interactive video traffic. We also suggest easy-to-deploy countermeasures against our ETA technique, that can possibly be adopted by Netflix for preventing the leakage of user choice information. Similar countermeasures are being used by Netflix today.
2. We present a first-of-its-kind encrypted traffic analysis (ETA) technique which

shows that HTTP/2 Multiplexing is not a dependable solution for ETA-based network surveillance. Our work shows that a network adversary with the same privileges as a traditional on-path passive eavesdropper can completely break the privacy guaranteed by HTTP/2 Multiplexing, and infer the browsing activities of a targeted website user. We demonstrate a real-life adversarial attack using our proposed technique on a popular political survey website. The encrypted traffic dataset built for this purpose would be made available to interested researchers. We also suggest countermeasures against our ETA technique that can possibly be used by the website administrators for protecting the privacy of their users.

3. We present the first encrypted traffic analysis (ETA) framework for practical mass-scale surveillance of website users. Our traffic analysis framework, Snoopy, shows that network surveillants can infer fine-grained browsing information (i.e., webpages visited) of the visitors of a targeted website, despite the diversity in their browsing contexts (i.e., browser, operating system, number of parallel tabs used, number of webpages visited, cache and cookie settings, network conditions, etc.). Since leakage of such information is more concerning to the organizations hosting the website than the website visitors, we suggest countermeasures that can be implemented and deployed by the website developers without the involvement of the website visitors. Specifically, we suggest countermeasures that leverage Snoopy as a tool that can be used by website developers for designing websites that are resilient to mass surveillance, before they are deployed in the public domain.
4. We present the first-of-its-kind traffic analysis (ETA) technique that reveals that despite encryption network surveillants can identify highly sensitive communications of targeted ICS systems from remote on-path network devices. Thereafter, a cyber attacker can attack targeted industrial control systems by disrupting these highly sensitive communications. These ETA-based attacks are stealthier and more difficult to locate than conventional ICS attacks that infiltrate

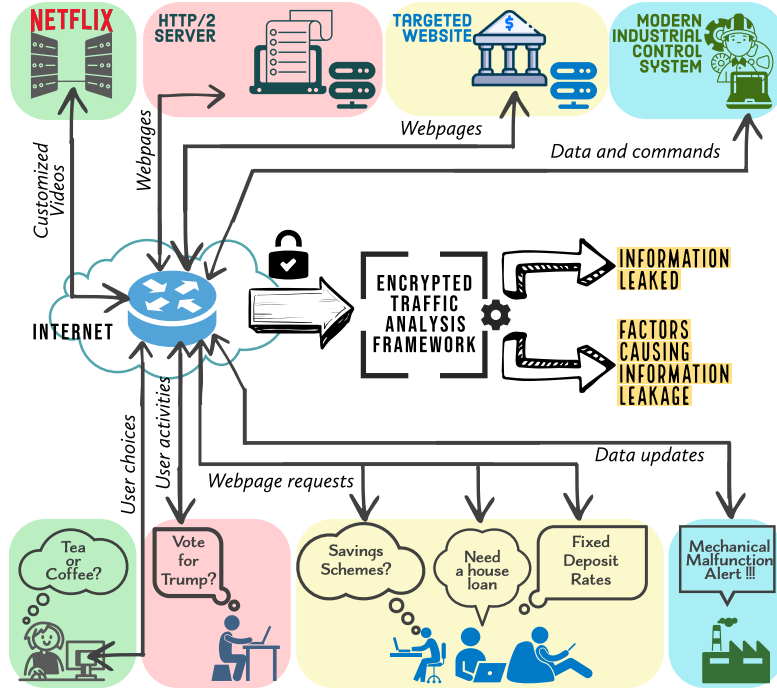


Figure 1.1: Contributions of the thesis in the area of encrypted traffic analysis of modern communication systems

ICS devices. We demonstrate the effectiveness of the proposed traffic analysis technique with the help of a case study, where we emulated a realistic water treatment plant that uses state-of-the-art communication and security protocols. Furthermore, we study the impact of different factors (communication protocol, system design, etc.) on leakage of information from encrypted ICS traffic. This is useful for the security community, especially since standardization of encryption protocols for ICS communication is still in the initial stages. We also suggest some basic countermeasures against our proposed technique and the associated trade-offs.

At a high level, this thesis can be considered as a framework for analyzing encrypted traffic generated by a diverse range of communication systems and protocols, for uncovering potential information leaks. Figure 1.1 shows the different communication systems that have been considered in this thesis. These systems use different types of techniques and protocols for effective communication between web applications and internet users. For instance, interactive Netflix movies use the HTTP/1.1 protocol (Fielding *et al.*, 1999)

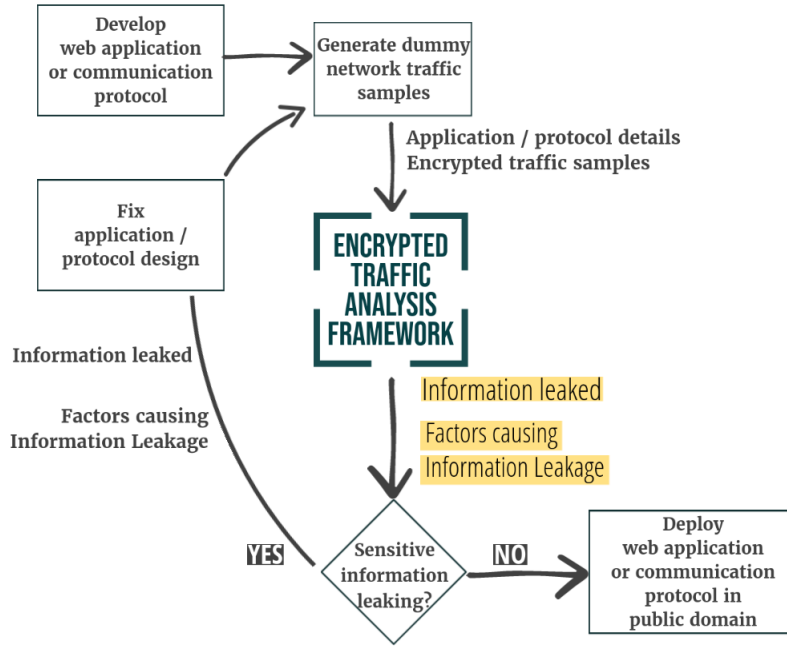


Figure 1.2: Application of the proposed ETA framework in developing secure web applications / communication protocols

with TLS encryption for transmitting its user choices, whereas ICS devices use secure ENIP (Rinaldi, 2019) or secure Modbus (Schneider Electric USA, Inc., 2018) protocols for communication. Both of these communication systems work in a single-threaded fashion, where the server can process only one request from a given client at a given point of time. On the other hand, HTTP/2 servers work in a multi-threaded fashion, where the server processes multiple requests from a given client in parallel. The difference in server operations is manifested on the network traffic metadata. Similarly, in the mass-surveillance scenario, the visitors of a given website may use different types of operating systems (OS), which in turn use different TLS fragment sizes while constructing network packets. Therefore, even when two website visitors access the website using the same HTTP protocol, the choice of their OS might result in differences in their traffic characteristics. To address these concerns, we need to use metadata and analysis techniques that are tailor-made for each of these communication systems.

The framework that we propose in this thesis exploits different combinations of encrypted traffic metadata and analysis techniques for each of the aforementioned communication

systems. The choice of the metadata and analysis technique are governed by the communication techniques and protocols used by the application, and the sensitive information that needs to be protected from a surveillant. For instance, we exploit encrypted file size sequences in the case of interactive Netflix videos, whereas we exploit a combination of encrypted packet sizes and inter-packet arrival timings in the case of modern ICS traffic. Similarly, in the case of interactive Netflix videos, we use statistical methods to correlate encrypted file sizes with user choice options, whereas in the case of modern ICS systems, we use cyclic pattern identification techniques to detect repetitions in the encrypted traffic metadata.

Our framework, although primarily designed from the perspective of a network surveillant, can also be used by business and infrastructure owners for evaluating the resistance of their communication systems against ETA-based surveillance. For instance, an application developer can use the framework as shown in Figure 1.2. The proposed framework not only tells the developer whether the web application is resilient, but also indicates the potential reasons for information leakage if the application is not resilient. The developer can use this feedback to strengthen the application, and then re-evaluate it using the proposed framework. The developer might continue this cycle until all the sensitive information are found to be protected.

1.6 OUTLINE OF THIS THESIS

The rest of this thesis is organized as follows: Chapter 2 provides the common background knowledge required for a clear understanding of the main contributions of this thesis. For that, we first describe the network surveillant model that we consider in this thesis, including their position and capabilities. Thereafter, we provide a high level overview of encrypted web traffic metadata, as seen by the network surveillant from an on-path network device. While this chapter does not intend to provide a comprehensive discussion on network traffic metadata, it provides an insight into how different types of traffic

metadata might help a network surveillant infer sensitive information about their target.

Chapters 3, 4, 5 and 6 present the main contributions of this thesis. In each of these chapters, we analyze the encrypted traffic generated by either a modern web application or a modern communication protocol, with the aim of uncovering potential information leaks to a network surveillant. Additionally, each of these chapters also approaches the problem of information leakage from the perspective of different stakeholders involved in different communication systems. Furthermore, our analyses reveal the factors in web application or protocol design that contribute to the information leakage. Such an analysis is necessary for understanding the impact of leakage of different types of information, which in turn is useful for designing effective countermeasures. We conclude each chapter with potential countermeasures that would help developers develop applications or protocols that would be practical and resilient by design to information leakage.

Chapter 3 presents an analysis of encrypted network traffic generated during Netflix interactive video streaming. We propose an ETA technique that shows that network surveillants can infer the user (viewer) choice information from traffic metadata. We also suggest potential countermeasures to prevent this information leakage.

Chapter 4 presents an analysis of encrypted traffic generated by websites using the HTTP/2 protocol. Contrary to the recent findings which speculate that multiplexing in HTTP/2 makes it resilient to ETA-based surveillance, we show that it is still possible for a network surveillant to infer the browsing information of a targeted website user from the encrypted traffic metadata. We further suggest potential countermeasures that can strengthen the HTTP/2 protocol against ETA-based surveillance.

Chapter 5 presents an analysis of mass-scale encrypted browsing traffic exchanged between a targeted website and its visitors. We propose an ETA technique which shows that network surveillants can infer fine-grained browsing activities of a large number of internet users visiting a targeted website, despite the diversity in their browsing contexts.

We further suggest potential countermeasures that can be used by website developers for designing websites that are resilient to ETA-based surveillance.

Chapter 6 presents an analysis of modern industrial control system (ICS) traffic, encrypted using the TLS protocol. We propose an ETA technique that can reveal sensitive ICS communications from traffic metadata. A cyber attacker can use this information to launch attacks on a targeted ICS. We further suggest potential countermeasures that can be used by ICS designers and administrators to prevent such information leakage.

Finally, Chapter 7 summarizes the thesis and identifies future research directions.

CHAPTER 2

BACKGROUND

In this chapter we discuss the background information required for understanding the subsequent chapters of this thesis. We begin with a description of the network surveillant, which includes its position and capabilities. Since we are concerned about network surveillance conducted with an adversarial intent, in the rest of this thesis we have used the terms ‘surveillant’ and ‘adversary’ interchangeably. The description of the surveillant is followed by a discussion on the popular secure communication techniques used by some of the modern internet-connected systems and their users, for securing their information from network adversaries. Finally, we give an overview of the traffic metadata that is still visible to the network adversaries, and how they might exploit the metadata for inferring sensitive information about the internet-connected systems.

2.1 NETWORK SURVEILLANT AND ITS CAPABILITIES

In practical scenarios, a network adversary can be a rogue ISP, a Government or business organization who is capable of colluding with an ISP, or a cyber-attacker who is capable of compromising one or more network devices deployed on the internet. We now discuss the positions on the network path that are favorable for network adversaries, and the capabilities that a network adversary should possess for conducting ETA-based surveillance.

2.1.1 Position of the network surveillant / adversary

A network adversary should position itself at a vantage point from where it can easily observe the encrypted traffic of its targeted system / internet users. For instance, in the scenario depicted in Figure 2.1, a network adversary can compromise any or all of the three routers if it aims to conduct surveillance on the communications between the web

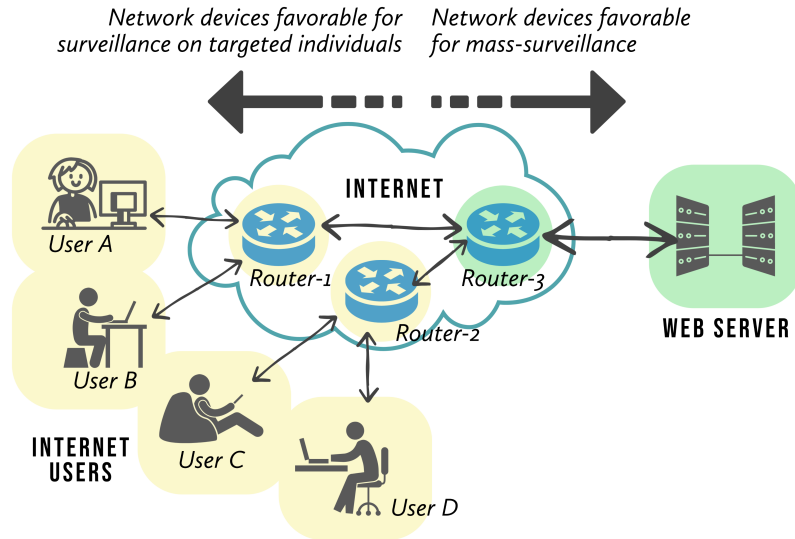


Figure 2.1: Positions of the network surveillant / adversary

server and its end users (website visitors).

Depending on its target, the adversary can position itself either on a network device that is close to the end devices of targeted internet users, or on a network device that is closer to a targeted web server (Refer to Figure 2.1). For instance, in the scenario depicted in Chapter 3, the adversary is interested in knowing the choices made by a targeted Netflix viewer. Therefore, the adversary would compromise a network device that is close to the viewer (Router-1 or Router-2 in Figure 2.1). On the other hand, for the mass-surveillance scenario depicted in Chapter 5, the adversary is interested in the cumulative browsing behavior of the visitors of a targeted website, and therefore does not target any one internet user in particular. Hence, it would eavesdrop on a network device from where it can access the browsing traffic of a large number of website visitors, ideally closer to the targeted web server (Router-3 in Figure 2.1). For mass-surveillance on visitors of web services that are hosted on the cloud, the adversary can compromise any network device on the path of the cloud server(s) and the website visitors.

2.1.2 Capabilities of the adversary

A network adversary (surveillant) should have the following capabilities to perform ETA-based surveillance.

1. The adversary should be capable of compromising a network device (for e.g., router, load balancer, firewall) on the internet path of its target. Examples of such adversaries include entities capable of colluding with an ISP (Raman *et al.*, 2020) or a rogue ISP (Singel, 2008; Federal Trade Commission, USA, 2009). The adversary can also be a cyber attacker capable of gaining unauthorized access to a network device on the internet. The adversary can do so by exploiting vulnerabilities such as open ports, device misconfigurations, or devices with default login credentials. The Shodan search engine (Shodan, 2022) shows that the number of such vulnerable devices deployed across the world is significantly high.
2. The adversary should be able to observe all the network packets of its target from the compromised device. This includes the encrypted application data and the traffic metadata. A network adversary can use traffic monitoring tools such as tshark (Wireshark, n.d.) or deep packet inspection (DPI) tools such as nDPI (Deri, 2022) for this purpose. In case the IP address of the targeted user is behind a network address translator (NAT), an adversary can employ existing encrypted traffic analysis techniques (Verde *et al.*, 2014) for identifying the traffic flow of its target.
3. The adversary should be capable of introducing network perturbations at the compromised network device. This includes dropping or delaying selected network packets. At the outset this might seem to be an additional capability as compared to a conventional network surveillant. However, in practice, the privileges required by an adversary for introducing network perturbations is the same as that required for monitoring network traffic (i.e., superuser access). The adversary can introduce

network perturbations either by using techniques such as port flooding (Cloudflare, n.d.a) or by modifying the traffic routing policies.

2.1.3 Assumptions and scope of this thesis

1. We assume that the adversary does not infiltrate *any* end device of its targets, including web servers. The adversary also cannot decrypt the network traffic by hijacking the decryption key or breaking the encryption. These assumptions preclude man-in-the-middle (MITM) attacks (Callegati *et al.*, 2009; Raman *et al.*, 2020) that involve installing compromised certificates on the end user systems. These assumptions are reasonable in scenarios where strong intrusion detection systems are deployed at the end user devices, or in scenarios where the adversary wants to remain stealthy from its targets over long periods of time.
2. In this thesis, we only focus on the leakage of information from encrypted traffic and its causes. Though we discuss the potential damages that can be caused due to the information leakage, we do not focus on designing techniques that an adversary might use for harvesting the leaked information.

2.2 SECURE COMMUNICATION PROTOCOLS USED BY MODERN INTERNET-CONNECTED SYSTEMS

Today, internet-connected systems and their users use different types of secure communication protocols for keeping their communication details private from network adversaries. The most popular secure communication protocols include TLS encryption (i.e., Hypertext Transfer Protocol Secure (HTTPS) (Cloudflare, n.d.b) for web applications, and secure Common Industrial Protocol over Ethernet Industrial Protocol (Rinaldi, 2019) (CIP-over-ENIP) and secure Modbus/TCP (Schneider Electric USA, Inc., 2018) for ICS communications), Virtual Private Networks (VPN) (Kaspersky, n.d.), and the Onion Routing (Tor) (Tor, n.d.). Each of these protocols provide different

levels of security and privacy to internet users by obfuscating different attributes of a communication session. While TLS encryption provides content privacy by securing the data exchanged between the communicating web applications (Rescorla, 2018), VPN adds an extra layer of privacy on top of TLS encryption, i.e., relationship privacy, by obfuscating the source and destination IP addresses from network adversaries (Kaspersky, n.d.). Tor provides the highest level of privacy, i.e., path anonymity, in addition to content and relationship privacy (Tor, n.d.). Tor achieves this by routing the traffic over multiple hops across the world, and by obfuscating the location of these hops from network adversaries.

However, all these secure communication protocols are not equally suitable for all types of internet-connected systems/applications due to factors such as operational overheads and conflict with web application policies. We now analyze the functional requirements and policies of each of the modern internet-connected systems and identify the secure communication protocol that is best suited for each of them.

2.2.1 Interactive Netflix video streaming

Privacy-aware consumers of Netflix are concerned about their viewing habits getting leaked to network adversaries. In case of interactive movies, viewing habit also includes the choices made by the viewer while watching the movie. However, Netflix viewers do not always have the liberty to use solutions such as Tor or VPN for protecting their choice information from network adversaries. This is because, *firstly*, Netflix prevents IP addresses belonging to VPN service providers from accessing most of its contents, since there are geographical restrictions on the distribution of contents hosted on Netflix (Netflix, 2022b). Presently, only a few VPN service providers are known to be able to bypass the traffic filtering mechanism used by Netflix (Marshall, 2022). However, Netflix consumers would not be able to rely on this handful of VPN services for a long time, as Netflix keeps strengthening its VPN detection mechanisms. *Secondly*, Tor is not suitable for bandwidth-intensive and delay-sensitive applications such as video

streaming. This is because, Tor incurs a high bandwidth overhead due to additional header information per packet, which is necessary for routing the traffic over anonymous paths. Moreover, Tor communications (Syverson *et al.*, 2004) incur additional delays due to multiple layers of header encryption at the source and repeated decryption of routing information at each network hop, thereby making it unsuitable for seamless video streaming. Therefore, Netflix users predominantly rely on the HTTPS protocol for protecting their viewing habits from network adversaries. Not only Netflix supports HTTPS encryption (Netflix Technology Blog, 2016), but HTTPS is also highly suitable for video streaming applications due to its negligible latency.

2.2.2 Web browsing traffic of internet users

Internet users can access websites (both HTTP/1 and HTTP/2) using any of the three aforementioned secure communication protocols to protect their information from network adversaries. However, the high bandwidth consumption and delay incurred by Tor significantly degrade the browsing experience of internet users. This deters most of the internet users from using Tor on a daily basis, unless they reside in countries with heightened levels of censorship (for e.g., Russia), or access sensitive web contents from regions with strict authoritarian regimes (for e.g., the USA) (Tor, 2022). Besides, countries with stricter levels of censorship can even prevent its citizens from using Tor (for e.g., China)⁴. The delay incurred by VPN is much lesser than that of Tor. However, VPN services are provided by third parties, who might have malicious intents. VPN service providers can easily observe the HTTPS traffic metadata of their clients. This allows a rogue VPN service provider to either exploit the traffic metadata of its clients, or sell it to other malicious entities for their own financial benefits (Rafter, 2021). In the past, several VPN service providers have been found tracking their clients' activities and/or selling their personal data without the knowledge of their clients (Newman, 2018). Compared to VPN and Tor, HTTPS is still the most preferred communication protocol among internet users worldwide. Presently, more than 90% of the web traffic is transmitted using the

⁴Refer to https://en.wikipedia.org/wiki/Great_Firewall

HTTPS protocol (Google, 2022). The immense popularity of the HTTPS protocol can be attributed to its low latency and ease of use.

2.2.3 Industrial Control System Traffic

Modern industrial control systems need to protect their sensitive control data and commands from network adversaries on the internet. However, unlike web browsing communications, ICS communications cannot use Tor. This is because, the additional delay incurred by Tor makes it unsuitable for ICS applications, which are highly sensitive to network delays. Therefore, the control traffic of modern internet-connected ICS systems is predominantly routed over VPNs and TLS-encrypted communication channels such as secure CIP-over-ENIP (Rinaldi, 2019) and secure Modbus/TCP (Schneider Electric USA, Inc., 2018). The metadata of the TLS-encrypted communications can be easily observed by the network adversaries. Similar metadata is also exposed to the VPN service providers when the ICS traffic is routed over VPNs. Therefore, even when using VPNs, ICS networks might face security threats if the VPN service provider is dishonest.

To summarize, Tor is not suitable for several modern internet-connected systems, owing to its high bandwidth consumption and delay. While VPNs are popular among internet users today, they do not guarantee complete security and privacy of user information, since a rogue VPN service provider might exploit the traffic metadata in a way similar to a network adversary eavesdropping on HTTPS traffic or TLS-encrypted ICS traffic. Therefore, it is important to ensure content privacy of TLS-encrypted communications between the application endpoints before relying on third parties for additional security features such as relationship privacy or path anonymity. This necessitates the analysis of TLS-encrypted traffic of internet-connected systems for identifying potential vulnerabilities that might cause information leakage.

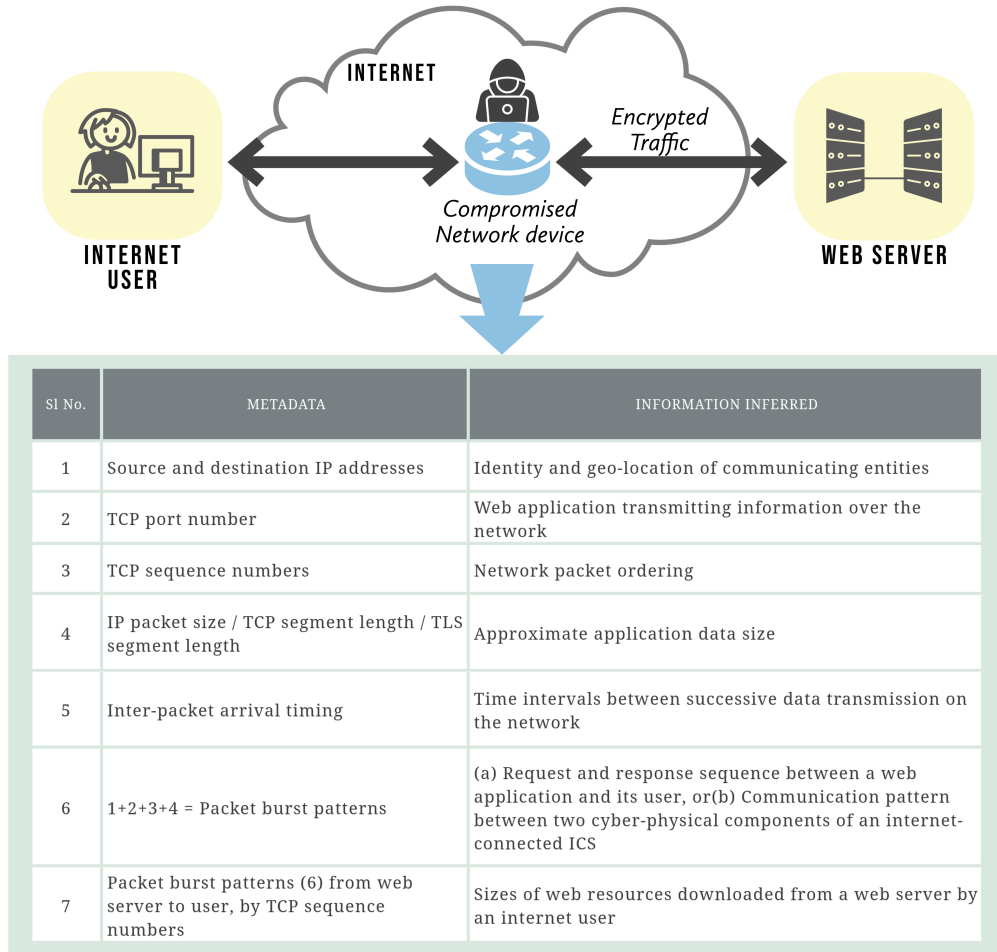


Figure 2.2: Examples of information inferred by an ETA-based adversary from network traffic metadata

2.3 ENCRYPTED TRAFFIC METADATA: AN ADVERSARIAL PERSPECTIVE

The TLS encryption standard encrypts the application data communicated by a pair of sender and receiver (for e.g., web server and website visitor). Web application data includes information such as the web URL accessed, and the HTML and multimedia files downloaded by a website visitor. In an industrial control system network, application data includes data and commands exchanged between different cyber-physical components.

Even when the application data is encrypted, a network adversary can still observe the following traffic metadata from compromised network devices - (i) *Unencrypted packet headers* - this includes IP layer (Information Sciences Institute, 1981), Transport layer (i.e., TCP layer (Information Sciences Institute of USC, 1981) for the systems considered

in this thesis), and TLS layer (Rescorla, 2018) headers, and; (ii) *Traffic flow characteristics* - this includes packet sizes, inter-packet timing intervals, and other volumetric and timing information that are derivatives or combinations of packet sizes and inter-packet intervals (for e.g., packet burst patterns).

Unencrypted packet headers. The network adversary can use the source and destination IP addresses to determine the identities and geo-location of the parties involved in a communication session. The TCP port number indicates the application generating the network traffic. For instance, port 443 indicates HTTPS traffic, whereas port 44818 indicates traffic generated from ICS devices using the secure CIP-over-ENIP protocol. TCP sequence numbers indicate the order in which the observed network packets are transmitted by the communicating sender and receiver. TLS and TCP segment sizes are indicative of the size of the application data encrypted in the network packet.

Traffic flow characteristics. Traffic flow characteristics refer to the volumetric and temporal information about the observed network traffic. Primarily, it includes packet size information and inter-packet arrival timings at the compromised network device. An adversary might use different statistical properties and combinations of these primary information to understand more complex traffic characteristics. For instance, an adversary can combine packet size and inter-packet timings to understand the traffic burst patterns.

Combining packet headers and traffic flow characteristics. ETA-based network adversaries attempt to derive complex combinations of packet header information and traffic characteristics that are indicative of the targeted user or system behavior. For instance, prior research has shown that an adversary might use a combination of IP addresses, TCP port number and packet burst patterns to infer the web resource download sequence when a targeted internet user accesses a website (Sun *et al.*, 2002). Figure 2.2 shows a few examples of how an adversary can use different combinations of traffic metadata to infer sensitive information about the targeted system / internet user.

The metadata of the network traffic generated by internet-connected systems depend on the design of the system and the nature of interactions between the system and its users. For instance, when an internet user browses a website, network traffic is generated from the user's device when the user requests for a web resource (GET request) or submits a web form (POST request). The volume of traffic generated depends on the amount of information transmitted over the network. For instance, the traffic volume generated by a GET request would be much smaller in comparison to the traffic volume generated by a POST request that carries the user's response to a web form. Similarly, traffic is generated from the web server when it sends web resources (HTML and multimedia files) to the user. In this case, the volume of traffic generated depends on the size of the web resource transmitted. A prior research work (Sun *et al.*, 2002) has shown that a network adversary who is aware of the volume of traffic generated by different types of web requests and resource downloads can infer the web resources requested and downloaded by targeted website visitors from traffic burst patterns. The adversary can use this information to infer the web browsing habits of the targeted website visitors.

Leakage of information from encrypted traffic metadata can be prevented by making changes to the web application design or communication protocols. For instance, in the aforementioned scenario, the adversary would not be able to accurately infer the web resources from traffic volume, if the resources had similar sizes and URL lengths. However, generally web application designers are unaware of the impact of the application design on the network traffic metadata. This thesis attempts to address this problem by analyzing the impact of different design aspects of modern internet-connected systems on ETA-based information leakage.

2.4 SUMMARY

In this chapter, we first discussed the position and capabilities of an ETA-based adversarial network surveillant. We also discussed the different types of security protocols used

by different modern internet-connected systems explored in this thesis. Thereafter, we discussed the different types of traffic metadata that a network adversary can observe from the encrypted traffic of these systems, and how the adversary can exploit the observed metadata to infer sensitive information about its targets.

Encrypting application data would be futile if the adversary can infer the same from the traffic metadata. However, often the system design and/or protocol design are responsible for leaking application data through network traffic metadata. Therefore, we need to explore the effect of different types of system design and communication protocols on the leakage of information through traffic metadata. An understanding of these factors will help the web developers design protocols that are resilient by design to ETA-based information leakage. The subsequent chapters analyze the encrypted traffic generated by four different modern communication systems that have not been widely studied by the research community yet. Based on our observations, in each chapter we suggest practical countermeasures for each of these communication systems against ETA-based network adversaries.

CHAPTER 3

WHITE MIRROR: LEAKING SENSITIVE INFORMATION FROM INTERACTIVE NETFLIX MOVIES USING ENCRYPTED TRAFFIC ANALYSIS

Netflix, the leading internet entertainment giant of the USA, released ‘Black Mirror: Bandersnatch’, a psychological thriller film on December 28, 2018. The film instantly became popular due to its unique *interactive* screenplay and a highly non-linear script. The film, the first of its kind in the mainstream media, allows a viewer to create his/her own story-line and climax by choosing one of the on-screen options (e.g., an option to throw tea over the computer or to shout at Dad) presented to them at different points while watching the movie. Depending on the viewer’s choice, the corresponding segment of the movie gets played. Interestingly, the choices made and the path followed can potentially reveal viewer preference information that ranges from benign (e.g., their food and music preferences) to sensitive (e.g., their affinity to violence and political inclination). Although this information is available to Netflix, they are bound by legal clauses that prevent them from misusing it. Today, to prevent this information from leaking to unauthorized parties, Netflix uses end-to-end encryption (Netflix Technology Blog, 2016). Recent works Reed and Kranch (2017); Schuster *et al.* (2017) in the domain of encrypted network traffic analysis were able to infer information such as the title of videos watched by a viewer. In this work, we raise the following research question: ‘*Do interactive movies leak fine-grained information about the viewers to passive eavesdroppers (compromised network devices) even when encrypted?*’. To answer this question, we need to answer the following : ‘*Is it possible for an adversary to distinguish between two branches of the same interactive video from the traffic metadata?*’

Privacy leaks from Netflix videos/movies are well researched. Current state-of-the-art works (Reed and Kranch, 2017; Schuster *et al.*, 2017; Li *et al.*, 2018) have been able to obtain coarse-grained information such as the genre and the title of videos from encrypted network traffic metadata. While conventional Netflix video traffic predominantly consists encrypted chunks of video files, recently released *interactive* Netflix videos also transmit some fine-grained viewer preference information over the internet. However, leakage of these fine-grained information from encrypted video traffic has not been studied so far. Such information can be used to build behavioral profiles of viewers. In this chapter we investigate if this user preference information is susceptible to leakage from the encrypted Netflix traffic metadata. This analysis is important, since interactive movies are becoming increasingly popular over the years and Netflix plans to use the same technology for developing more movies and games in the near future (Netflix, 2022a).

In this chapter, we use the movie ‘Black Mirror: Bandersnatch’ as a case-study to develop techniques for revealing information from encrypted *interactive* video traffic. Our preliminary studies showed for the first time that information such as the choices made by viewers can be revealed based on the characteristics of encrypted control traffic exchanged with Netflix. To evaluate our proposed technique, we built the first interactive video traffic dataset of 100 viewers. Our technique was able to reveal the choices 96% of the time in the case of ‘Black Mirror: Bandersnatch’. We communicated our findings to Netflix in early 2019. Further, around July 2019, we noticed that Netflix had fixed the vulnerability that our traffic analysis had exposed. The fix involved changing the mechanism by which the viewer choice information was communicated to the server. While this new mechanism defeated our original traffic analysis technique, we were able to discover a new combination of traffic metadata that could potentially leak the choice information to network adversaries. Specifically, using a combination of traffic burst patterns and encrypted file size, we were able to infer not only the choices made by the viewer, but also if the viewer deliberately clicked on the option or if the option got selected automatically due to a time-out. This technique was able to accurately

predict the choices made by a viewer, as well as whether they deliberately clicked on a choice or not, with $\approx 100\%$ accuracy, when a viewer watched the movie ‘Black Mirror: Bandersnatch’ for the first time. The contributions of this work are as follows.

1. We present the first traffic analysis technique for interactive videos that can leak more information about a targeted user’s choice than non-interactive videos.
2. We present the first dataset and identify future directions for encrypted traffic analysis on interactive videos. To facilitate researchers to work on this problem, we have made the dataset publicly available ⁵.

3.1 RELATED WORK

The objectives of existing research works (Reed and Kranch, 2017; Schuster *et al.*, 2017; Li *et al.*, 2018) have so far been limited to discerning video content, identifying the titles of videos, and the genre preferred by a viewer from encrypted traffic of *conventional* videos.

Existing techniques from the literature are not suitable for encrypted interactive video traffic analysis due to the following fundamental reason: inter-video features cannot be used to differentiate between segments from the same video. For instance, prior works (Reed and Kranch, 2017) have used bitrate as a feature to differentiate between two video streams. However, in the context of an interactive video, the bitrate of chunks pertaining to each choice will be the same and hence cannot be used to distinguish between two video segments. In this work, we identify an intra-video side-channel (i.e., network metadata that leaks information) and show that it holds across various operating conditions.

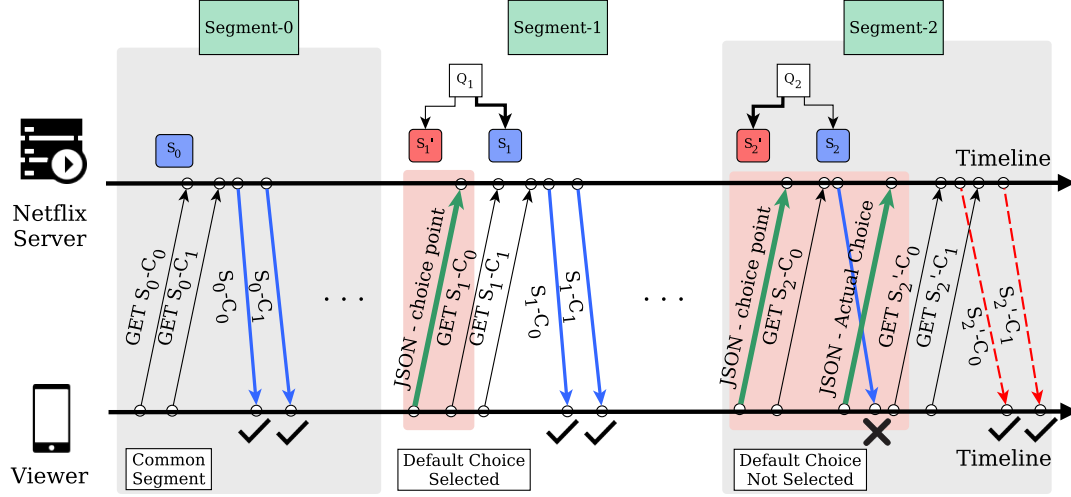


Figure 3.1: The streaming process of Bandersnatch

3.2 STREAMING PROCESS FOR INTERACTIVE NETFLIX VIDEOS

We study the streaming process of interactive Netflix videos for the first time in 2018, shortly after the movie ‘Black Mirror: Bandersnatch’ was released. In this section, we describe the process used by Netflix for streaming interactive videos at that time. Netflix uses MPEG-DASH (Dynamic Streaming over HTTP) over TLS/SSL and HTTPS (HTTP/1.1 + TLS) for streaming the video files and user choices respectively.

Due to the non-linearity of the script, the streaming process is check-pointed at each choice-question in the movie unlike conventional videos that stream continuously. The content of interactive videos is divided into several segments, each corresponding to one path segment in the script, with each segment consisting of multiple video chunks. Figure 3.1 shows an example of the control and data traffic exchanged when a viewer makes choices. The first segment of the movie (i.e., Segment-0) is common for all viewers at the end of which the choice-question Q_1 is presented. At this point, the viewer’s browser sends a JSON file (type-1 JSON file) to the server indicating that the viewer has encountered Q_1 . The viewers are then given ten seconds to choose one of the options. Our experiments revealed that Netflix considers one choice to be the default and prefetches

⁵<https://github.com/Gargi-Mitra/SIGCOMM2019-NetflixInteractive>

video chunks corresponding to this choice. We denote the default choice for question Q_i ($i \in \{1, 2\}$) as S_i and the non-default choice as S'_i . If the viewer chooses S_i , then the streaming of video chunks continues uninterrupted. In Figure 3.1, $S_i - C_j$ ($i \in \{1, 2\}$, $j \in \{0, 1\}$) denotes the j^{th} chunk of the i^{th} video segment. If the choice S'_i is chosen by the viewer, the prefetching for S_i stops and a request for S'_i is sent.

In the example shown in Figure 3.1, the viewer selects the default choice S_1 for Q_1 . The streaming continues uninterrupted until Q_2 appears. Like before, a type-1 JSON file is sent from the viewer to Netflix. We assume that the viewer selects S'_2 for Q_2 . In this case, a type-2 JSON file will be sent subsequently. Although some chunks corresponding to S_2 are pre-fetched, they will be discarded and the chunks corresponding to S'_2 will be streamed.

3.3 ENCRYPTED TRAFFIC ANALYSIS (TECHNIQUE-1)

First, we assume that the adversary can successfully infer the title of the movie being watched by the targeted viewer, using existing techniques (Reed and Kranch, 2017; Schuster *et al.*, 2017; Li *et al.*, 2018). Thereafter, we proceed to infer more fine-grained information, i.e., the choices made by the targeted viewer while watching an interactive movie on Netflix. Based on our knowledge of the streaming process used by Netflix, as discussed in § 3.2, we could infer that the number and type of JSON files sent indicate the choice made by the viewer. However, identifying the two types of JSON files from encrypted network traffic is challenging.

Our analysis revealed that the packets carrying the encrypted type-1 and type-2 JSON files can be distinguished from other packets by their SSL record lengths. This observation was found to be consistent across various operating systems, browsers, devices, connection media, network conditions, and for all interactive movies released so far. Hence, we use ‘*SSL record lengths of client packets*’ as the side-channel (i.e., the traffic metadata leaking information). The JSON file types can be used to infer if the choice is the default

Conditions	Attribute	Value
Operational	Operating System	Windows, Linux, Mac
	Platform	Desktop, Laptop
	Traffic Conditions	Morning, Noon and Night
	Connection Type	Wired, Wireless
	Browser	Google-chrome, Firefox
Behavioral	Age-group	< 20, 20-25,25-30, > 30
	Gender	Male,Female,Undisclosed
	Political Alignment	Liberal, Centrist, Communist, Undisclosed
	State of Mind	Happy, Stressed, Sad, Undisclosed

Table 3.1: Attributes of the dataset

or the non-default one. The actual on-screen choices can be inferred by re-creating the sequence.

3.3.1 Dataset and Results

For evaluating the proposed traffic analysis technique, we collected the encrypted network traces corresponding to viewing sessions of 100 viewers. The network traces were collected from the nearest gateway. The ground truth for evaluation was generated by asking the viewers to note down the choices they made. The viewers were free to choose the device, operating system, browser, and the geographic location from which they could watch the movie. We also collected some behavioral parameters of the viewers to highlight the implications of this privacy leak. Table 3.1 summarizes the different behavioral and operational conditions under which traffic samples were collected. We collected traffic samples across various combinations of OS, browser and traffic conditions, to understand if variations in these factors affect the accuracy of our technique. From our experimental results for the movie ‘Black Mirror: Bandersnatch’, we observed the following - (1) Browser had no impact on the prediction accuracy; (2) Platform had no impact on the prediction accuracy; (3) OS had an almost negligible impact on the accuracy of prediction. As shown in Figure 3.2, for the traffic samples

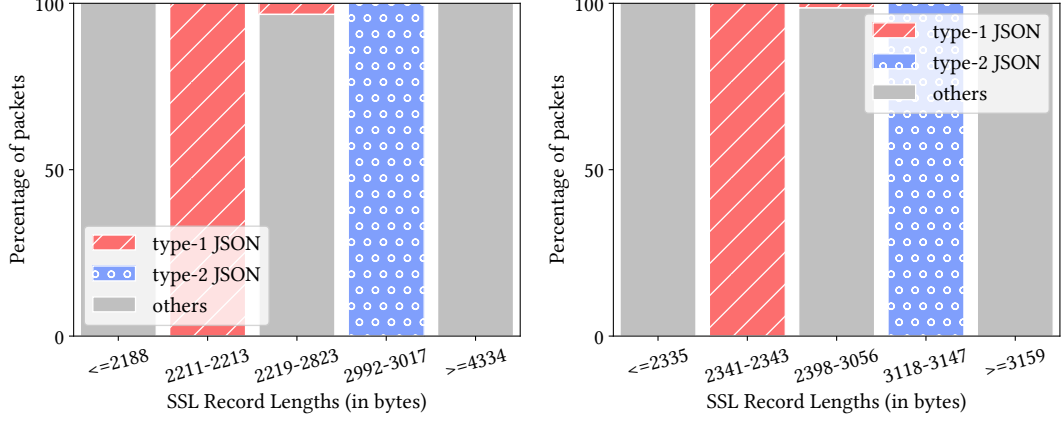


Figure 3.2: SSL record lengths of different types of JSON files for two different browsing contexts – (Desktop, Firefox, Ethernet, Ubuntu) and (Desktop, Firefox, Ethernet, Windows)

collected from Ubuntu systems, we saw mispredictions (‘Type-1’ JSON files mispredicted as ‘other’ type of JSON files) in 2.1% cases, whereas for traffic samples collected from Windows systems, we saw mispredictions in 1.9% cases. Note that the two JSON files have distinctive record lengths most of the time (96%), based on which the viewer choices can be inferred; and, (4) Network conditions also had no impact on the accuracy. We have released our dataset to interested researchers (Mitra *et al.*, 2019a).

3.3.2 Impact of our work and response from Netflix

In March 2019, we communicated our research outcomes to Netflix, with the hope that Netflix would be interested in fixing the vulnerability on the basis of our findings. Even though they appreciated our work, they considered it low-risk and declared it out of their scope ⁶. With the approval of Netflix, we published our work on arXiv, an open-access repository of electronic preprints (Mitra *et al.*, 2019b). Our work garnered public attention very quickly, and several media houses around the world expressed their interest in our work. In April 2019, an investigative report (Newman, 2019) on our work as published by the Wired magazine. The increased visibility of our work and

⁶Details of our communication with Netflix can be found here: <https://drive.google.com/drive/folders/1EMeeouumIyA7aCbbeG6hh-oQxmI9p8Im?usp=sharing>

its potential impact on Netflix users triggered extensive online discussions on popular platforms such as Reddit ⁷.

3.4 INTERACTIVE VIDEO STREAMING PROCESS OF NETFLIX (2019 - PRESENT)

The online discussions assured us that Netflix users were concerned about the leakage of their choice information to network surveillants. Around July 2019, we noticed that Netflix had fixed the vulnerability that was exposed by our work, despite their initial nonchalance. The fix involved modifying the way in which the user choices are communicated to the server. This modification was able to thwart our attack. Additionally, Netflix introduced a feature that allowed viewers (users) to undo any of their previously made choices at any point of time while watching Bandersnatch. This is unlike the initial days when a viewer had to restart the movie to change their previously made choices.

The newly made changes to the interactive Netflix video streaming technique made encrypted traffic analysis based surveillance more challenging. Therefore, we studied the modified video streaming process of Netflix, to understand if it is completely resilient against encrypted traffic analysis based surveillance. To be precise, we observed that Netflix had made the following modifications to their interactive video streaming process to facilitate the aforementioned changes.

Observation-1: The user choices are no more communicated explicitly to the Netflix server. Hence, no JSON file is sent to the server when the user encounters a choice-question. Instead, the video chunks that are to be fetched by the client when the viewer clicks on one of the two choices for a given question is determined by a Javascript. This Javascript file is downloaded by the client device of the viewer from the Netflix server, and executed during the viewing session. The Javascript also determines which video chunks are to be fetched when the viewer does not click on any choice within the ten

⁷Links to discussions on online forums can be found here: https://drive.google.com/drive/folders/1G_12329KJUiuW6T5oYHGb3BxEFaEaske?usp=sharing

second time-out period.

Observation-2: As soon as the viewer clicks on a choice at a choice-point, the corresponding video segments are fetched from the server. The number of video segments downloaded per second at this point is much higher, as compared to that in the linear sections of the videos. This results in a high traffic burst (spike) whenever a viewer clicks on a choice at a choice-point. Similar spike in network traffic can also be observed if the ten second time-out period gets over without the viewer making any choice.

Observation-3: Viewers are now allowed to undo any of their previous choices while watching the movie, without having to restart the movie. To enable this, Netflix adds a thumbnail on the screen as soon as the time-out period of a choice-point gets over. Each choice-point in the video is followed by a thumbnail. If a viewer wishes to undo the choice made at a previous choice-point, the viewer needs to click on the thumbnail corresponding to the choice-point they want to revisit. The thumbnail for a given choice-point contains an image snapshot of the scene when the choice-question had appeared on the screen. The thumbnail image for a given choice-point gets downloaded as soon as the corresponding ten second time-out period is over.

Observation-4: Netflix now uses a multi-CDN architecture for enhancing its streaming process. Therefore, different types of resources get downloaded from nodes belonging to different CDN service providers. For instance, when a viewer watches Bandersnatch from India, the video segments are often downloaded from the Netflix datacenters, whereas the thumbnails for choice-questions are downloaded from CDN nodes hosted by Reliance Jio Infocomm.

3.5 TRAFFIC ANALYSIS ON PRESENT STREAMING PROCESS

(TECHNIQUE-2)

Based on our knowledge of the modified streaming process used by Netflix, as discussed in § 3.4, we propose an encrypted traffic analysis based surveillance technique for inferring the choices made by viewers of interactive Netflix videos. The proposed technique has the following steps.

Step-1: *Identification of choice-points and choice-questions from encrypted traffic -*

From Observation-3 (refer to § 3.4), we infer that the downloading of a thumbnail image indicates that the viewer has encountered a choice-point in the video. These thumbnail images always get downloaded from IP addresses belonging to a specific CDN (refer to Observation-4), say, CDN_x . Therefore, the *time of occurrence of a choice-point* in the encrypted traffic can be determined by the presence of a traffic stream from CDN_x . Furthermore, our analysis revealed that all the thumbnail images have distinct sizes, even after encryption. Therefore, in addition to the time of occurrence of choice-points, we can infer the *question encountered* by the viewer at a given choice-point from the knowledge of the encrypted thumbnail image sizes. We estimate the size of an encrypted thumbnail image using the sum of the ‘*SSL record lengths of server side packets*’ in the traffic stream carrying the image.

Step-2: *Determining the choice made by the viewer -* From Observation-1 and Observation-2 (refer to § 3.4), we infer that a spike in per-second traffic volume between the client device and Netflix server indicates that either the viewer has clicked on a choice for a choice-question, or the ten-second time-out period of a choice-question is over. Our analysis revealed that the volume of the traffic spikes for the two choices at a given choice-point are different. Therefore, we use a combination of encrypted thumbnail image size and traffic volume per second to infer the choices made by the viewer while watching an interactive movie.

Step-3: *Determining if the viewer clicked on a choice, or the question was timed out -*

The time of occurrence of a choice-point and the corresponding choice-question can be identified from the encrypted traffic using encrypted thumbnail image sizes. The time at which the video segments for a given choice-point get downloaded can be inferred from the traffic volume per second. If the time interval between a traffic spike and the downloading of a thumbnail image is between 1 to 10 seconds, then it implies that the viewer clicked on a choice before the time-out period. On the other hand, if the time interval between a traffic spike and the downloading of a thumbnail image is less than 1 seconds, then it means either the viewer clicked on the choice at the last moment, or did not click on any choice at all.

A surveillant who used the proposed technique must watch the video and record the encrypted image sizes for each choice-point, and the traffic burst sizes corresponding to each choice for every choice-question. The surveillant might use this information later for predicting the choices of a targeted viewer from their encrypted traffic.

3.5.1 Results

Dataset. For evaluating the proposed traffic analysis technique, we collected the encrypted network traces corresponding to 20 viewing sessions of the movie ‘Black Mirror: Bandersnatch’, until the 5th choice-point (cumulatively 100 choice-points). The viewers were free to choose the device (laptop/desktop), start watching the movie from any point, at any time of the day, using any operating system and browser (similar to the summary shown in Table 3.1). The ground truth for evaluation was generated by asking the viewers to note down their choices, as well as whether they deliberately clicked on the choices or the default choice got selected on time-out.

Our experimental results showed that the proposed technique was able to predict the choices made by the viewers with 100% accuracy when viewers watched the movie for the first time. The technique was also able to predict whether the viewer clicked on a choice or it got selected automatically on time-out with 99% accuracy. However,

the accuracy of predicting the choice-question dropped to 60% in cases where viewers watched the movie more than once within a short time. In such cases, the accuracy of predicting the choice made by the viewer dropped to 40%.

3.5.1.1 Failure Analysis

Our detailed analysis reveals that the scenarios where the proposed technique does not perform well are the following.

1. The proposed technique could not identify the choice-points when a viewer watched the movie more than once within a short time and encountered the same choice-points as the previous time. This is because, in such cases the thumbnail images were already cached in the browser of the viewer and they did not get downloaded again. Consequently, the encrypted traffic did not contain any stream corresponding to thumbnail image download. For such repetitive viewing sessions, our technique was able to predict the choices only when the viewer encountered a choice-point that they had not encountered in the previous session.
2. Viewers who watched the movie more than once within a short time tended to fast-forward the video. In such cases, the proposed technique could not predict the choices made at a choice-point. This is because, fast-forwarding the video resulted in spikes in traffic volume, which obfuscated the traffic spikes close to the choice-points.
3. In cases when the viewer selected the default choice very close to the time-out period, the proposed technique could not distinguish it from the scenario where the default choice gets selected automatically due to a time-out.

However, we believe that when a viewer watches the same interactive movie for a second time within a short span of time, intention is to simply explore all the paths in the movie. Further, a viewer would repeatedly fast-forward the video only when they are not interested in the contents.

3.6 SUMMARY AND FUTURE WORK

In this work, we show the first encrypted traffic analysis attack on viewers of Netflix interactive videos. Our work shows that designing an interactive video streaming technique that is resilient to ETA-based surveillance is challenging. In fact, the traffic streaming technique described in § 3.4 is still being used by Netflix today. Therefore, we believe that our findings would pave the way for future research directions, some of which are listed as follows. We also release the dataset used for evaluating our work for other researchers.

Countermeasures. An easy fix for obfuscating the choice-question would be to design thumbnail images of the same size, and to ensure that the same volume of traffic gets downloaded at a choice-point irrespective of the choice made by the viewer. While redesigning thumbnails would not require much effort, ensuring equal traffic burst patterns for both choices might be challenging. This is because, at present, Netflix used the MPEG-DASH protocol for traffic streaming for ensuring the best end user experience. Therefore, altering the traffic streaming technique might degrade the end user experience, which is of utmost importance to Netflix. Furthermore, obfuscating the choice-points is even more challenging, since ensuring a uniform traffic burst pattern in the linear parts of the video and the choice-points might require Netflix to completely redesign the traffic streaming protocol.

Advancing privacy research. The video streaming engine used by Netflix is proprietary and hence we could not implement and evaluate our countermeasures. Practical implementation of even simple countermeasures may require significant changes to the software and may result in overheads and poor viewing experience. Further, these countermeasures may in-turn expose stronger unobvious side-channels, and therefore should be evaluated repeatedly. To implement and evaluate countermeasures, there is a need for an open source model of the interactive video control packet exchange.

Inferring more fine-grained information. In addition to the choices made by the

viewers, it would be interesting to see if an adversary can infer how promptly the user made the choice (time taken by the user to click on an option). This timing information might help an adversary infer more sensitive behavioral information about the targeted viewer. For instance, the quickness with which a viewer responds to a choice question might be indicative of their confidence in the decision or their impulsive behavior. Identifying traffic characteristics that reveal this information could be an interesting future research direction. Additionally, Netflix has recently implemented a feature in its interactive videos that allows a viewer to revisit a previous choice question. Identifying traffic features that can help an adversary infer the navigational activities of a viewer can be another interesting research direction.

Designing more powerful attacks. In the current form of the attack, we have assumed that an attacker would first use existing techniques (Reed and Kranch, 2017; Schuster *et al.*, 2017; Li *et al.*, 2018) to infer the title of the movie, and then use our proposed technique to infer the choices made by the viewer. It would be interesting to see if a more powerful attack could be designed for inferring both the title of the movie and the viewer’s choices in a single step. This would require a technique that can distinguish between different story-lines of different movies from their traffic metadata. Possible future directions would be to investigate if there exists any such metadata, and if the discovery of such metadata can be automated.

Evaluating the vulnerabilities of interactive videos at scale. At the time when this research was conducted, no other video streaming platform had developed interactive videos similar to those developed by Netflix. Also, Netflix itself had only five interactive videos hosted on its platform. However, recently Netflix has started to develop a wide range of interactive content which includes movies, games, and quizzes. An interesting future direction would be to compare the extent of information leakage from different types of video content. Additionally, we noticed that inspired by the success of Netflix, very recently, some other video streaming platforms such as MXPlayer have started to

develop online interactive movies, albeit of much shorter lengths and simple story-lines. Exploring the extent of information leakage across various video streaming platforms and techniques can be another exciting research direction.

CHAPTER 4

DEPENDING ON HTTP/2 FOR PRIVACY? GOOD LUCK!

ETA-based surveillance on targeted users of HTTP/1.x-based websites and video streaming platforms have been widely explored in the existing literature (Cheng and Avnur, 1998; Sun *et al.*, 2002; Danezis, 2010; Alan and Kaur, 2019; Ghi tte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021; Reed and Kranch, 2017; Schuster *et al.*, 2017; Li *et al.*, 2018). In Chapter 3, we exposed vulnerabilities in encrypted traffic generated by Netflix *interactive* videos, which also uses the HTTP/1.1 protocol with TLS encryption for transmitting user choices. All these attack techniques exploit a certain property of the HTTP/1.x protocol, i.e., serialized single-threaded object transmission by the web server. This method of object transmission results in deterministic traffic patterns, and helps the surveillant easily learn the traffic burst patterns for different websites/videos or infer the object sizes from the encrypted traffic. Existing defense techniques against many of these attacks suggest introducing randomness in the traffic patterns by introducing traffic padding or random inter-packet delays (Gong and Wang, 2020; Cai *et al.*, 2012; Dyer *et al.*, 2012; Ling *et al.*, 2022). However, such defense techniques are often not practical, since they either consume a lot of bandwidth or have adverse effects on the performance of the application.

The HTTP/2 protocol seemed to offer a natural resilience against contemporary ETA-based surveillance techniques that were successful on HTTP/1.x-based websites. HTTP/2 introduced multi-threaded server operation for performance improvement over HTTP/1.1. Recent research works (Morla, 2017; Lin *et al.*, 2019) have discovered that multi-threaded operation results in *multiplexed* object transmission, that can also have an unanticipated positive effect on TLS/SSL privacy. In fact, these works go on to design privacy schemes

that rely heavily on multiplexing to obfuscate the sizes of the objects, thus complicating the attempts made by the adversaries to infer sensitive information. Orthogonal to these works, we examine if the privacy offered by such schemes work in practice.

In this work, we show that it is possible for a network adversary with modest capabilities to completely break the privacy offered by the schemes that leverage HTTP/2 multiplexing. Our adversary works based on the following intuition: *restricting only one HTTP/2 object to be in the server queue at any point of time will eliminate multiplexing of that object and any privacy benefit thereof*. In our scheme, we begin by studying if (1) packet delays; (2) network jitter; (3) bandwidth limitation; and, (4) targeted packet drops have an impact on the number of HTTP/2 objects processed by the server at an instant of time. Based on these insights, we design our adversary that forces the server to serialize object transmissions, thereby completing the attack. Our adversary was able to break the privacy of a real-world HTTP/2 website 90% of the time, the code for which has been released ⁸. To the best of our knowledge, this is the first privacy attack on HTTP/2.

4.1 BACKGROUND

The Facebook-Cambridge Analytica data scandal (Chan, 2019) showed the monetary value of seemingly unimportant personal preferences (e.g., Facebook pages liked) of website users. In recent times, a more severe form of privacy attack, that does not even require access to the end user or the server devices, has been reported. This form of attack is carried out by an adversary that compromises a network device that sits between the client and the server and relies merely on the encrypted traffic exchanged between the two. Further, this attack is totally non-intrusive and does not require compromising the decryption key. Instead, the adversary analyzes the encrypted traffic using statistical and machine learning techniques to reveal *web access patterns* that are indicative of user preferences. Many research works (Cheng and Avnur, 1998; Sun *et al.*, 2002; Chen *et al.*, 2010; Zhang *et al.*, 2010; Danezis, 2010; Chapman and Evans, 2011; Gong *et al.*, 2012;

⁸https://bitbucket.org/gmit91/http2_privacy_attack/src

Miller *et al.*, 2014; Juarez *et al.*, 2014; Panchenko *et al.*, 2016; Sirinam *et al.*, 2018) have highlighted the vulnerability of HTTP/1.x (with SSL/TLS) based webserver to these encrypted traffic analysis attacks.

Traffic analysis attacks on HTTP/1.x-based websites use the sizes of encrypted objects (images, JavaScript files, etc.) transmitted by the server to uniquely identify the webpage accessed by the client. For instance, a research work (Lescisin and Mahmoud, 2018) was able to identify the political leaning of users of a popular survey website (Isidewith, 2022; Dewey, 2016; James, 2012) just based on the size of encrypted image files. Although there exist other channels through which such information can be inferred by an adversary, encrypted object size is the most reliable and widely used side-channel. Consequently, the defense techniques (Wright *et al.*, 2017; Luo *et al.*, 2011; Cai *et al.*, 2014; Cui *et al.*, 2018; Al-Naami *et al.*, 2019) against such attacks aim to obfuscate object sizes, albeit at the cost of unreasonable CPU and bandwidth overheads, which often make them impracticable.

The HTTP/2 protocol (Belshe *et al.*, 2015) is being seen by recent research works (Morla, 2017; Lin *et al.*, 2019) as an efficient alternative to such expensive defense techniques. The idea is to use the newly introduced multi-threading feature of the HTTP/2 server that can potentially multiplex multiple objects during transmission. By multiplexing the objects, the resulting TCP stream will have interleaved segments belonging to different objects, making it difficult for a network eavesdropper to discern the encrypted sizes of objects of interest. Therefore, the parallelism in object transmission, although introduced for performance improvements, has the potential to improve privacy without any additional effort. Orthogonal to these efforts, in this chapter, we investigate the following question: ‘*Can we depend on HTTP/2 Multiplexing for obfuscating encrypted object sizes?*’.

We show that a network adversary with the same privileges as a traditional on-path passive eavesdropper can completely break the privacy guaranteed by HTTP/2 multiplexing.

Our adversary leverages the simple fact that spacing consecutive GET requests sent by a client will eliminate any opportunity for the server to multiplex the objects corresponding to these requests. We begin by constructing an adversary that alters network parameters such as latency, jitter, bandwidth, and packet drops. We then observe the effect of each parameter on the multiplexing of objects within an HTTP/2 stream. Our adversary achieves inter-request spacing by adding jitter at the compromised network device immediately after the request corresponding to the object of interest is forwarded. Doing so results in non-multiplexed transmission of the targeted object, thereby breaking the privacy. However, adding jitter has the side-effect of intensified multiplexing for the subsequent objects, which would pose a challenge for the adversary if it were interested in those objects also. Note that the intensified multiplexing is mainly due to the retransmission requests sent for the delayed object (due to jitter) by the client TCP layer of the client.

To suppress the intensity of multiplexing of subsequent objects, we use the following strategies: **(1)** limit network bandwidth; and, **(2)** reset the ongoing HTTP/2 streams. The adversary limits the network bandwidth to ensure that a limited number of client requests reach the server. This reduced the scope for multiplexing at the server, albeit not completely. When this is not sufficient, the adversary may choose to force the server to reset all the ongoing HTTP/2 streams. Doing so will restart the transmission for the subsequent objects with a clean slate. To achieve this, our adversary drops the packets carrying retransmitted objects, which in turn makes the client time-out and request a server reset. Following this, the client again requests for the subsequent object of interest. At this point, the adversary repeats the attack as it did for the first object of interest. We demonstrate the effectiveness of our enhanced adversary using the website www.isidewith.com, a political survey website that was recently migrated from HTTP/1.1 to HTTP/2 for privacy reasons. Our attack was able to discern all objects of interest 90% of the time. To the best of our knowledge, this is the first traffic analysis attack on HTTP/2 technology.

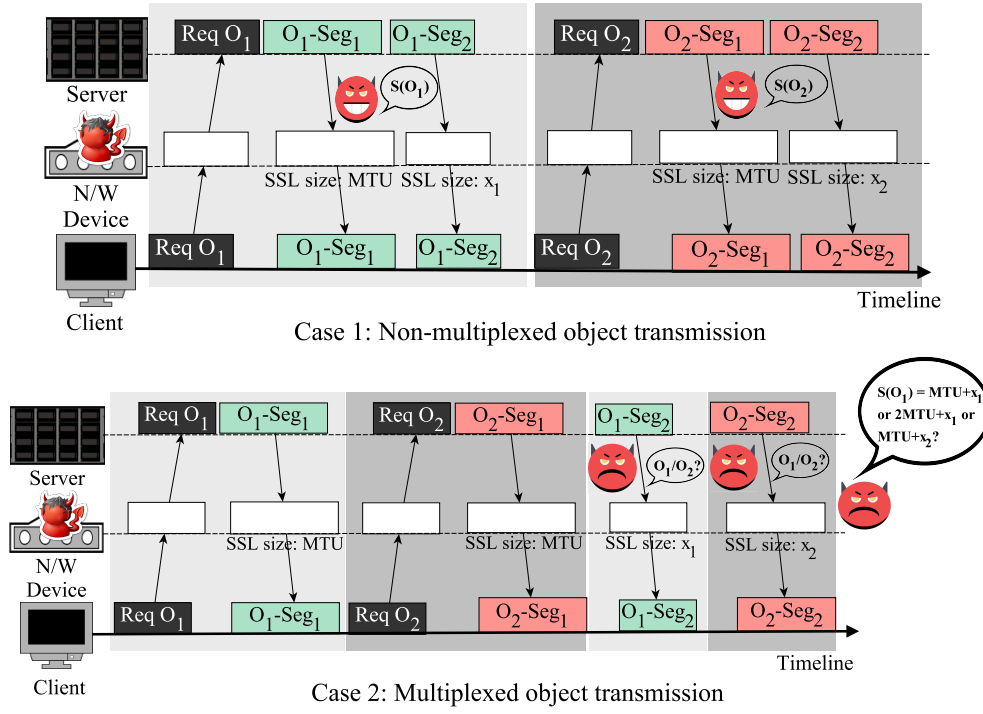


Figure 4.1: Estimating object sizes from encrypted traffic in non-multiplexed (i.e., O_2 after O_1) vs. multiplexed object transmissions (i.e., $O_1Seg_1 \rightarrow O_2Seg_1 \rightarrow O_1Seg_2 \rightarrow O_2Seg_2$)

The rest of the chapter is organized as follows: § 4.2 covers the preliminary concepts necessary to understand the chapter. § 4.3 provides a high-level overview of our attack. § 4.4 provides a comprehensive analysis of the impact of network parameters on HTTP/2 multiplexing. § 4.5 describes our adversary and evaluates its impact on a real-world website. We discuss the literature related to our work in § ???. Finally, we conclude the chapter by providing further insights for future work in § 4.6.

4.2 PRELIMINARY CONCEPTS

In this section, we discuss some preliminary concepts related to the HTTP/2 protocol that are relevant to the ideas presented in this chapter.

4.2.1 HTTP/2 Multiplexing

The advent of HTTP/2 protocol brought significant improvements over HTTP/1.1 in terms of user perception of latency during webpage loads. Contrary to HTTP/1.1, where requests are processed sequentially, HTTP/2 allows concurrent server threads to serve multiple objects on the same TCP connection, effectively *multiplexing* them. Such a design is particularly useful in avoiding Head-of-Line blocking – the scenario where a large object in the queue blocks the subsequent objects from being processed. This phenomenon is prevalent in HTTP/1.1 and has been widely exploited by adversaries for performing traffic analysis.

The multiplexed object transmission in HTTP/2 makes it difficult for a passive adversary to identify individual objects when TLS/SSL is used. Figure 4.1 illustrates how a passive network adversary perceives non-multiplexed and multiplexed object transmissions. We consider a scenario where a web client requests a web server for two web objects, say, O_1 and O_2 . We assume that the sizes of both these objects are greater than the *Maximum Transmission Unit* (MTU). Therefore, the web server splits these objects into multiple segments before transmitting them on the network. In this case, we assume that each object O_i ($i \in \{1, 2\}$) gets split into two segments, namely, $O_i - Seg_1$ and $O_i - Seg_2$. In practice, often object sizes are not exact multiples of the MTU. Therefore, the size of the final segment of an object is often less than (rarely equal to) the MTU.

Case 1 shows a simple scenario where the client requests the server for the object O_2 *after* the transmission of the object O_1 is completed. The network adversary that has access only to the size of the packets can still determine that the transmission of the object O_1 is complete the moment it receives a delimiting packet, i.e., the last packet with size that is less than the MTU. Similarly, it can determine the start and end of transmission of the object O_2 . Consequently, the adversary can sum up the packet sizes corresponding to the objects O_1 and O_2 to determine their sizes, from which the identities of the objects can be revealed.

On the other hand, in Case 2, the client requests for O_2 *before* the transmission of O_1 is completed. Since the HTTP/2 protocol supports multi-threaded server operation, the server starts transmitting segments of O_2 without waiting for the transmission of O_1 to be completed. In this scenario, two different server threads continue to transmit the segments of O_1 and O_2 in parallel. Consequently, the segments of O_1 get interleaved with those of O_2 , thereby making it difficult for the adversary to differentiate between the packets carrying segments of O_1 and those carrying segments of O_2 . This, in turn, makes it difficult for the adversary to estimate the sizes of the individual objects. This fact has been used by recent works to thwart privacy attacks that depend on the encrypted object sizes.

4.2.2 Privacy of HTTPS Communications and Related Works

Privacy, in the context of Internet communications, is a very fluid concept. In this work, we consider *privacy* as the property of a client-server communication due to which the fine details of the browsing activities of the client are protected from a passive network eavesdropper. An example of such information is the webpage(s) visited by the client within a website. Despite the use of TLS/SSL encryption, prior works have shown that it is possible to infer these fine details by analysis of encrypted network traffic. These works (Cheng and Avnur, 1998; Sun *et al.*, 2002; Chen *et al.*, 2010; Zhang *et al.*, 2010; Danezis, 2010; Chapman and Evans, 2011; Gong *et al.*, 2012; Miller *et al.*, 2014; Juarez *et al.*, 2014; Panchenko *et al.*, 2016; Sirinam *et al.*, 2018) target users of HTTP/1.x websites to predict the websites/webpages they accessed, either over anonymous communication channels such as Tor (Tor, n.d.) or over simple encrypted channels such as HTTPS. A comprehensive study of such works suggest that, for accurately identifying from encrypted traffic if a particular webpage within a website was accessed, it is sufficient if **(1)** the webpage contains at least one object which is not contained in any other webpage in the website; **(2)** the size of the object is different from those of all other objects that constitute the website; and, **(3)** the eavesdropper is able to infer the encrypted object size from the network trace. Hence, we consider

that the privacy of a webpage is completely broken if a network adversary is able to infer the size of an object that uniquely identifies the webpage (provided it exists) from encrypted traffic. So far, researchers have been successful in launching privacy attacks on HTTP/1.x.

On the other hand, the privacy of HTTP/2 in the aforementioned context is still an ongoing topic that is being investigated. Recent research works (Morla, 2017; Lin *et al.*, 2019) have ascertained the difficulty of extending the attack techniques used in HTTP/1.x to HTTP/2. Specifically, the researchers highlight the complications introduced by HTTP/2 multiplexing in the context of object size based attacks. Investigating the privacy of HTTP/2 is the primary objective of this chapter.

4.2.3 Network Parameters

We have seen that multiplexing can prevent attempts by a passive network adversary to estimate object sizes. However, we suspect that multiplexing can be affected by variations in network conditions – they may influence the TCP layer parameters such as congestion window size, which may, in turn, affect the HTTP/2 parameters such as the intensity of multiplexing. We consider the following network variations in our work:

1. **Delay.** An addition of a uniform delay in the network increases the Round Trip Time (RTT) of every packet exchanged between the client and the server. However, the time interval between the transmission of two consecutive packets on the network remains the same as in the case of the network with no delay.
2. **Jitter.** Most real-world networks experience jitter, i.e., variable network delays. Contrary to a network with uniform delay, in a jittery network, the inter-packet time intervals are not consistent, when compared to those in case of the network without jitter.
3. **Bandwidth throttling.** Bandwidth throttling is often performed by Internet Service

Providers (ISPs) to limit the number of packets transiting the network. Bandwidth limitation may result in slow loading of images and videos in webpages and degraded quality of video streaming.

4. **Targeted Packet drops.** Packets can be dropped in a targeted manner from any intermediate network device, either due to benign reasons (such as congestion control at a router) or due to an adversary residing on the router.

As we will see, changes in traffic patterns at the network layer affect the multiplexing behavior of the HTTP/2 server. The implications of this finding are particularly alarming since even a passive network adversary can control network conditions to a certain extent.

4.2.4 Measuring Privacy

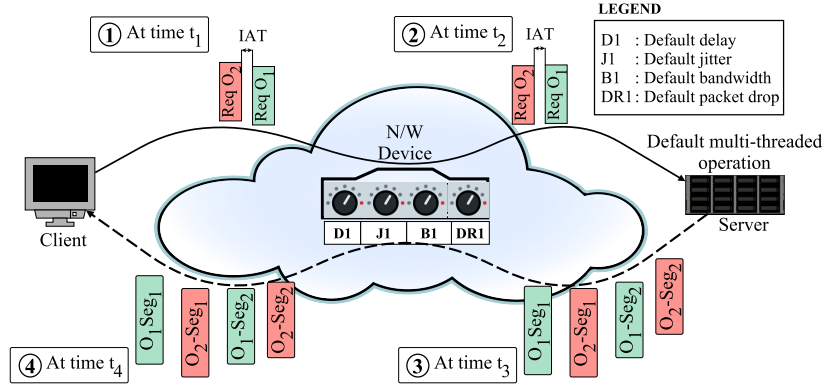
We define a metric to quantify the extent of multiplexing of objects, which, in turn, will help us in determining if the information targeted by the adversary is private or not.

Degree of Multiplexing. We define the *degree of multiplexing* of an object as the fraction of bytes of the object that is interleaved with those of another object or multiple other objects within the same TCP stream.

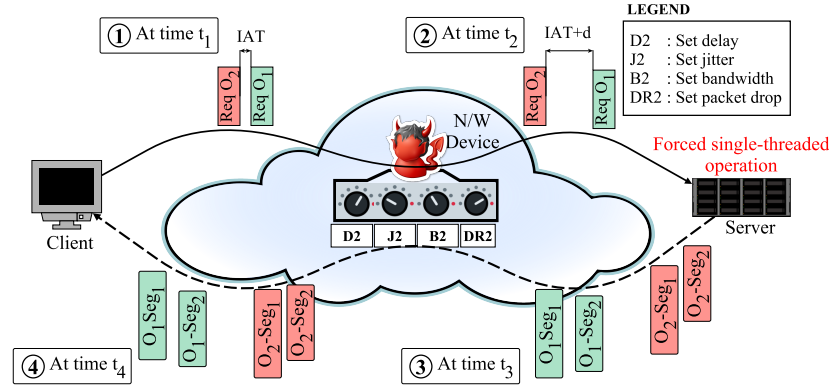
Implication on Privacy. From the definition of degree of multiplexing, we can say that, in the scenario where the adversary is able to reduce the degree of multiplexing of an object to *zero*, i.e., no byte of the object is interleaved with another object, they can estimate the encrypted object size. For instance, if two specific resources are required to uniquely identify a webpage, a scheme that prevents the multiplexing of the two objects is sufficient to break the privacy.

4.3 HIGH LEVEL OVERVIEW

In this section, we provide a high-level overview of our adversarial scheme. The objective of our adversary is to infer the webpages accessed by a targeted client (or user) within a targeted HTTP/2 website using encrypted network traffic analysis techniques. Our



(a) Multiplexed object transmission in HTTP/2 in a benign scenario



(b) Non-multiplexed object transmission in HTTP/2 due to adversarial intervention

Figure 4.2: High level overview of proposed adversarial scheme

adversary uses encrypted object sizes as a side-channel. Using encrypted object sizes in HTTP/2 is particularly challenging due to the multiplexed nature of object transmission. Our adversary is designed to overcome this challenge.

Figure 4.2 shows the capability of our adversary and its impact on HTTP/2 multiplexing. An ETA-based adversary uses the sizes of encrypted objects to uniquely identify the webpage used by the client. When a client accesses a webpage, it sends requests to the server for the objects (HTML files, style and font files, multimedia files, JavaScript files, etc.) embedded in the webpage in a sequential manner. For a given webpage in a website, the sequence in which a client requests for the objects is determined by the webpage design. This sequence is fixed for each webpage, and can be observed by the adversary when it accesses the webpage by itself. Furthermore, using existing ETA

techniques (Lescisin and Mahmoud, 2018), it is possible for the adversary to know the size of which objects in the sequences would help it discern the identity of the webpage being accessed by the client. An adversary needs to focus on only these objects. We assume that the adversary knows which objects to focus on, for breaking user privacy. Hence, it is sufficient for the adversary to prevent the multiplexing of only these *objects of interest*, which we assume to be O_1 in our example.

Figure 4.2a depicts the scenario without our adversary. At time t_1 , the client sends two successive GET requests for O_1 and O_2 . Let IAT be the time interval between the two requests, which is very small. At time t_2 , the packets are routed through the intermediate network device and reach the server. At time t_3 , the segments of O_1 and O_2 are transmitted in an interleaved manner due to server multi-threading. Finally, at time t_4 , the object segments reach the client in an interleaved manner, where they are assembled by the browser.

Figure 4.2b depicts the same scenario with our adversary controlling an intermediate networking device. Our adversary changes the network parameters to delay the GET request for O_2 by an additional time d (with respect to arrival time of O_1). This extra delay in arrival of O_2 's request ensures that the server completes transmitting O_1 before initiating the transmission of O_2 . Thus, the network adversary prevents the multiplexing of the object of interest. When the adversary is interested in more than one object in the same TCP stream, the attack gets challenging due to the repercussions of the previously introduced delay. We describe how our adversary overcomes these in the rest of the chapter.

4.3.1 Capability of Adversary

Our adversary is a compromised network device on the client-server path that can (1) access unencrypted header fields of both control and data packets; (2) monitor size of encrypted packets; (3) delay packets; (4) limit the bandwidth of the transit link; and, (5) drop packets. At the outset, this adversary model might seem stronger than the

popular passive adversary model used in prior works (Cheng and Avnur, 1998; Sun *et al.*, 2002; Chen *et al.*, 2010; Zhang *et al.*, 2010; Danezis, 2010; Chapman and Evans, 2011; Gong *et al.*, 2012; Miller *et al.*, 2014; Juarez *et al.*, 2014; Panchenko *et al.*, 2016; Sirinam *et al.*, 2018; Morla, 2017; Lin *et al.*, 2019), which would only capture network traffic in promiscuous mode. However, in reality, the privilege required (i.e., superuser access) for doing so is sufficient for tuning the network parameters. Such an adversary model is not only realistic but also common today (Matherly, 2022). Also, since we assume a non-intrusive adversary, MITM attacks that involve key hijacking and/or server impersonation, or require compromising the client’s browser are out of scope of this work.

4.3.2 Assumptions and Scope

We make the following assumptions for our adversarial scheme. **(1)** We assume that the goal of our adversary is to infer the identity of objects from the encrypted HTTP/2 stream. Once this is done, any of the techniques from the HTTP/1.x literature can be used to launch a full-fledged privacy attack. **(2)** We assume that the adversary does not have the capability to decrypt at the point of real user traffic collection. Breaking encryption is indeed a difficult problem to solve, and there exists no practical work that can do this. This assumption holds true for most authorized and unauthorized parties except maybe those involving government agencies. **(3)** We assume that the adversary’s behavior cannot be detected easily at the client or the server since it mimics network issues that are commonplace. **(4)** We also assume that the adversary has sufficient time ($\approx$ few minutes) to access the website and tune the network parameters before launching the attack. **(5)** We assume that the adversary has the knowledge about the sequence in which objects in the webpage are requested for and the specific object that is of interest.

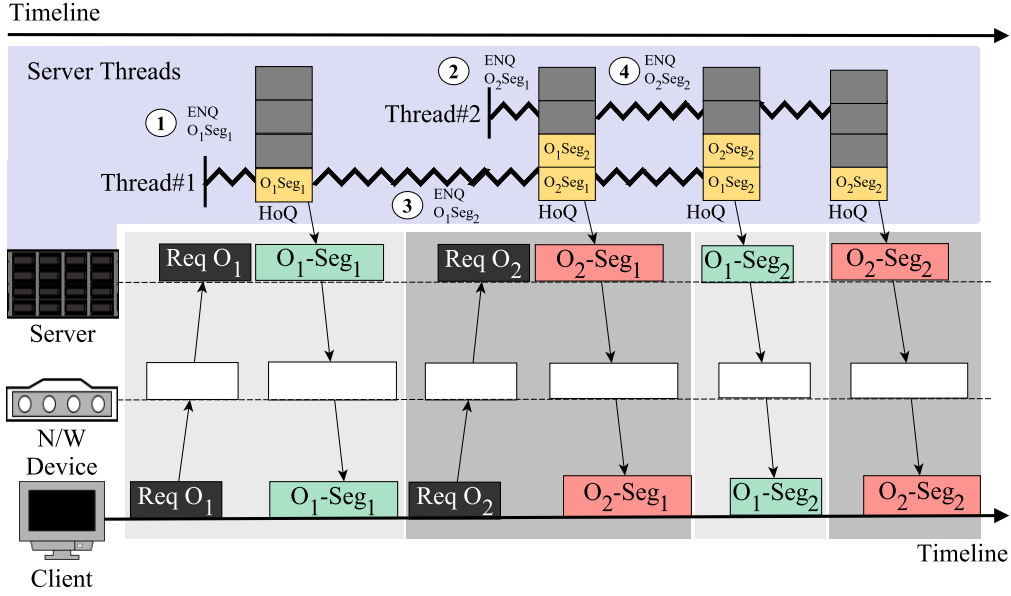


Figure 4.3: Interleaved object transmission by an HTTP/2 server

4.4 IMPACT OF NETWORK PARAMETERS ON HTTP/2 MULTIPLEXING

In this section, we explain how the network parameters affect multiplexing at the server. We start by extending Figure 4.2 to show the server side operations. Figure 4.3 shows the threads that get created at the server corresponding to the GET requests sent by the client. In this figure, we specifically focus on the multi-threaded behavior of the HTTP/2 server, when it receives requests for two web objects O_1 and O_2 within a short interval of time.

The server, on receiving the first request, spawns a thread (*Thread#1*), which starts enqueueing the first segment of O_1 on the server queue for transmission on the TCP stream, as shown in Step ①. However, before *Thread#1* could enqueue the second segment of O_1 , (i.e., $O_1 - Seg_2$), the server receives a request for the object O_2 from the client and spawns a second thread (*Thread#2*) that starts enqueueing segments of O_2 on the server queue (refer Step ②). This is followed by the enqueueing of $O_1 - Seg_2$ (refer to Step ④) and $O_2 - Seg_2$ (refer to Step ⑤) respectively. In the presence of TLS/SSL encryption, this interleaving of object segments makes it difficult for our network adversary to estimate object sizes, as discussed in § 4.3.

We reinforce our explanations with experimental results on an object in the webpage showing the results for the ‘2020 Presidential Quiz’ in the website *www.isidewith.com*. The object of interest is an HTML file of size ≈ 9500 bytes. By default, the degree of multiplexing of this object is $\approx 98\%$. We now examine the change in the degree of multiplexing of this object with change in each of the network parameters, viz., delay, jitter, bandwidth, and packet drops.

4.4.1 Delay

Introducing uniform delay for all packets on client $\rightarrow$ server path in the network cannot increase the inter-arrival time between two successive packets at the server. Hence, we do not use this parameter for our adversarial scheme.

4.4.2 Jitter

The inter-arrival time between two successive GET requests sent by a client can be increased by delaying each of the request packets by an unequal amount of time. The proposed adversary can achieve this by introducing a calculated amount of network jitter on the client-server path. For instance, in our example, the first request can be delayed by 0 ms, second by d ms, the third by $2d$ ms, and so on, to achieve an inter-arrival spacing of d ms. The amount of jitter to be introduced should depend on the size of the object of interest, the time elapsed since the previous GET request, and the time interval before the issuance of the next GET request by the client under normal network conditions.

Figure 4.4 illustrates the HTTP/2 server behavior under jittery network conditions by extending Figure 4.3. The adversary holds back the client request for O_2 at the intermediate network device for a longer time than usual. Consequently, the server thread serving O_1 (i.e., *Thread#1*) completes the transmission of all the segments of O_1 in single-threaded mode before the request for O_2 reaches the server (refer to Step ① and Step ②). Therefore, if O_1 is the only object of interest to the adversary, then its start and end points of transmission and hence its size can be estimated.

When the adversary is also interested in a subsequent object (O_2 in this example), introducing jitter in the network can introduce complications. This happens when the client request for O_2 is delayed for a considerably long period of time. This may lead the server to send duplicate acknowledgements (dup-ACKs) to a previous GET request sent by the client, which makes the client believe that the request for O_2 has been dropped in the channel. When this happens, the client sends a bunch of retransmission requests, called *TCP Fast-retransmits*, for the same object to the server within a short span of time (denoted by $ReqO_2^*$ and $ReqO_2^{**}$). The retransmitted requests cause the HTTP/2 server to spawn multiple concurrent threads (*Thread#2*, *Thread#3* and *Thread#4*) that serve multiple copies of O_2 to the client over the same TCP stream in interleaved manner, thereby resulting in intensified multiplexing.

In our experiment, the object of interest in *isidewith.com* is the 6th object downloaded by the client. Table 4.1 shows the impact of increasing jitter values on the degree of multiplexing of the object. For each jitter value shown in the table, the webpage was downloaded 100 times. Initially the number of cases where the object was not at all multiplexed increased steadily with an increase in the amount of jitter until 50 ms. However, increasing jitter further did not have any effect. This was because, at this point, the number of retransmission requests (> 130) started increasing manifold and the segments of the retransmitted objects got interleaved with those of the object of interest. Therefore, we set the jitter such that the inter-arrival time of requests is 50 ms. The next challenge for our adversary is to reduce the number of object retransmissions and object retransmission requests.

4.4.3 Bandwidth Limitation

Reducing the bandwidth during an ongoing communication session, given a particular amount of delay in the network, reduces the bandwidth-delay product (BDP). BDP is a measure of how much information (outstanding packets) the communication channel can hold at a given instant of time. When the BDP reduces, the TCP protocol at the

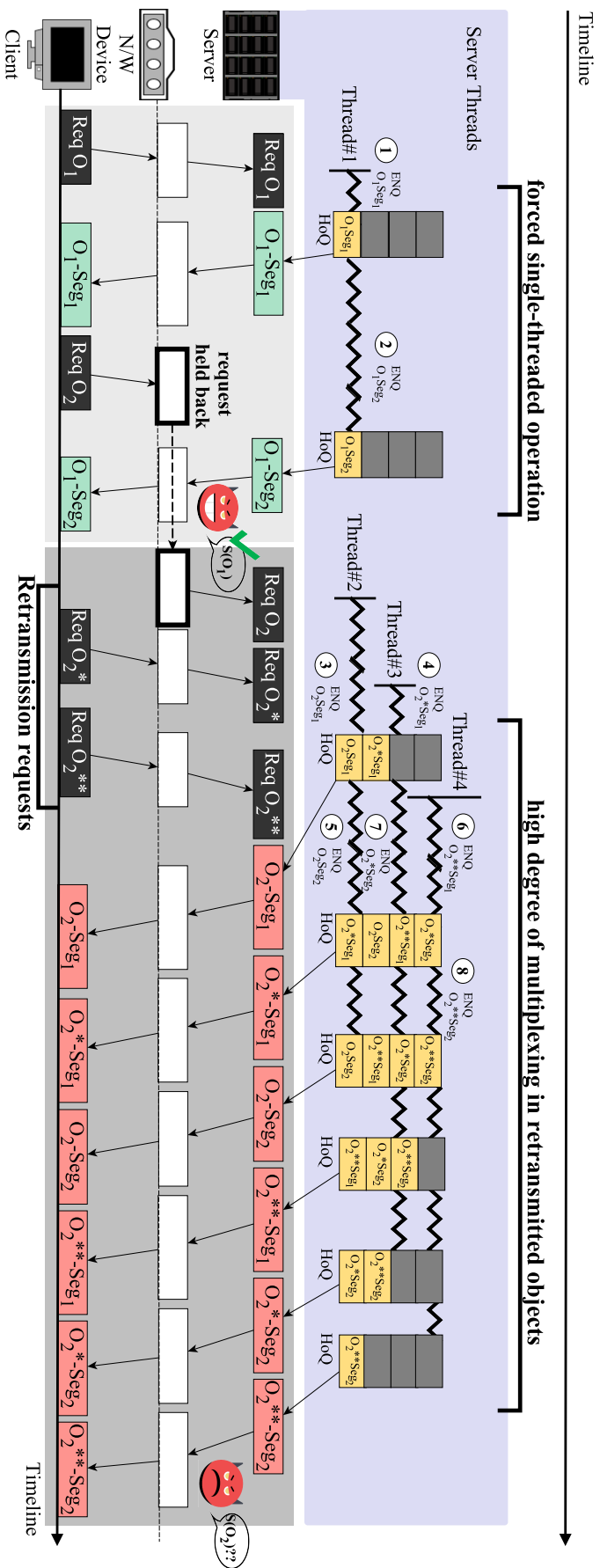


Figure 4.4: Effect of network jitter on multiplexing of objects in HTTP/2

Increase in delay per request (ms)	Cases where object of interest was not multiplexed (%)	Increase in no. of retransmissions (%)
0 (baseline)	32	0 (baseline)
25	46	≈ 33
50	54	≈ 130
100	54	≈ 194

Table 4.1: Effect of jitter on HTTP/2 multiplexing

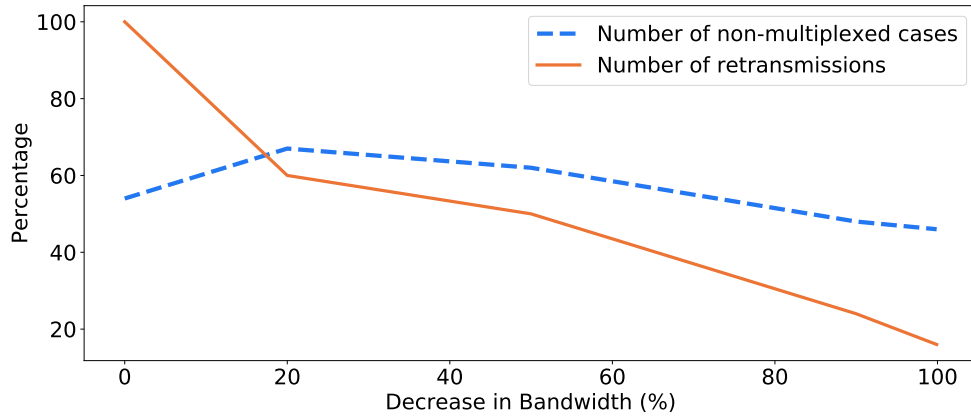


Figure 4.5: Effect of bandwidth limitation on multiplexing of objects in HTTP/2

communication endpoints responds to this change by decreasing the size of the TCP sender window. We rely on this intuition to reduce the number of fast-retransmit requests sent by the client.

We extend the setup described in § 4.4.2 with bandwidth limits applied at the compromised network device. Note that the bandwidth limits are applied for both incoming and outgoing packets. Our experiment starts with a channel bandwidth of 1000 Mbps. During the course of the experiment, we varied the bandwidth to 800 Mbps, 500 Mbps, 100 Mbps and 1 Mbps. For each bandwidth, the webpage was downloaded 100 times and we noted the corresponding degree of multiplexing of the object.

Our experimental results show that throttling the network bandwidth indeed reduces the

number of retransmissions as seen in Figure 4.5 (solid line). Contrary to our expectation, the percentage of cases in which our object of interest was in non-multiplexed state, i.e., success cases (dashed line), initially goes up sharply till 800 Mbps but reduces gradually right after. When we delved deeper, we understood that a significant percentage of successes observed till 800 Mbps can be attributed to a retransmitted version of the object and not the actual object. With a rapid decline in the number of retransmissions post 800 Mbps, the number of such success cases also reduce, thereby affecting the overall number of success cases at the lower bandwidths. Note that it was not possible to reduce the bandwidth beyond 1 Mbps because that resulted in a broken connection. Based on our observations, we limit the bandwidth of the medium to 800 Mbps. However, this can only improve the success rate to a small extent.

4.4.4 Targeted Packet Drops – forcing HTTP/2 Stream Reset

In this section, we investigate if another feature of HTTP/2 protocol, namely, the HTTP/2 stream reset, can further improve the success rate of our attack. The HTTP/2 reset stream was introduced to enhance the efficiency of a server especially when dealing with a highly lossy communication channel. Typically, the client sends the TCP fast-retransmit requests to deal with network layer delays and losses that are not persistent. However, when the communication channel is highly lossy, the client sends the HTTP/2 reset stream signal to the server (indicated by a packet with the corresponding HTTP/2 stream number and RST_STREAM flag set). Reset Stream is particularly interesting since the server closes the stream and flushes the corresponding object segments from its queue thereby reducing the load on the network immediately. In response to the lossy medium, the client's TCP stack also increases the timeout for fast-retransmits. After Stream Reset, the client resends GET requests if a high priority object is not yet received. We now exploit this feature to improve the success rate of our attack.

Figure 4.6 depicts a scenario that builds on the scenario described in Figure 4.4: firstly adding jitter (refer section 4.4.2) followed by limiting the network bandwidth (refer

section 4.4.3). To force a HTTP/2 stream reset, our adversary drops many packets on the server → client path (mimicking a lossy network) carrying the objects segments until the client resets the streams as shown in Figure 4.6. After reset, the client again sends the request for O_2 . The client's TCP also waits for a longer time before attempting to send fast-retransmission requests. Consequently, by this time, the server can completely transmit O_2 in single-threaded mode. The most challenging part of this phase of the attack is to identify when to start and stop dropping packets. This is because the adversary cannot discern the retransmitted objects from the actual ones. The adversary can either use a timer or use the number of forwarded GET requests (identified by using the filter `'ssl.record.content_type==23'`).

We experimentally verified the effect of targeted packet drop (with both jitter and bandwidth limits applied) on HTTP/2 multiplexing on the HTML file of size 9500 Bytes. Since the object is the 6th in the stream, the adversary drops 80% packets on server → client path from the time of transmission of the 6th GET request from the client. We continue the packet drops for 6 seconds until the client sends stream reset. The experiment was repeated 100 times and we observed a success rate of $\approx 90\%$, where the object of interest was transmitted in non-multiplexed state after the *Reset Stream* signal was sent by the client. However, further increasing the packet drop rate resulted in a broken connection.



Figure 4.6: Targeted packet drop by adversary to force non-multiplexed object transmission in HTTP/2

4.5 THE PRIVACY ATTACK

We now demonstrate a real-life adversarial attack ⁹ using the insights obtained from § 4.4. However, finding a real website with the complete implementation of HTTP/2 protocols is difficult today. Most HTTP/2 websites do not enable multiplexing either due to lack of awareness (disabled by default) or to avoid the extra storage required. Since our objective was to *disprove* that HTTP/2 multiplexing offers privacy, we demonstrated on the strongest HTTP/2 website known to us.

We chose *isidewith.com* (Isidewith, 2022), since it is the most widely cited website in the fingerprinting literature. This website allows users to identify their political leanings for the ‘2020 Presidential Elections’. It is also noteworthy that this is the same website that migrated from HTTP/1.1 to HTTP/2 shortly after privacy concerns were raised in a prior work (Lescisin and Mahmoud, 2018). Our evaluation suggests that currently it is resilient to most of the known attacks.

4.5.1 Target Website

The website works as follows: In response to the survey, a webpage containing the list of 8 political parties in order of preference of the user is displayed. The webpage consists of an HTML page that contains hyperlinks of 47 embedded objects, such as JavaScript files, images, and style files. One of these JavaScripts, on execution, makes the client to consecutively send request to the server for 8 images within a short span of time. Each image corresponds to the emblem of one of the 8 political parties, and the order in which the client issues requests for these images is the same as that in which they are displayed on the result webpage. The images of the political party emblems are of size ranging between 5KB to 16KB. Our adversary attempts to infer the order in which the 8 political parties appear on the webpage by encrypted traffic analysis. In short, the adversary has 9 different objects of interest – one HTML file and 8 image files. Note that our adversary has a pre-compiled list of image size to political party mapping which it leverages to

⁹Repository: https://bitbucket.org/gmit91/http2_privacy_attack

complete the attack.

4.5.2 Client setup

The experiment was conducted with the help of several volunteers (≈ 500) participating in the survey from different client systems inside our laboratory over a period of three months. They were asked to use the Firefox browser, which, to the best of our knowledge, is the only browser to have implemented HTTP/2 multiplexing efficiently. Each time, the volunteers noted down the survey results (sequence of political parties), which was later used as the golden reference to determine the prediction accuracy of our adversary.

4.5.3 Adversary Setup

We designated our lab's gateway (with 1 Gbps link) as the adversary. The gateway monitored the transiting traffic and whenever it could detect a connection to isidewith server, it launched the attack. The adversary was made up of 3 basic components – (a) the traffic monitor, which was implemented using tshark; (b) the network controller, which was implemented using bash scripts; and, (c) the object prediction module, which was implemented using Python scripts. In the first phase of the attack, the adversary introduced jitter (of 50 ms additional delay) in the client-server communication path and also started counting the number of GET requests in the client→server path. The adversary is not interested in identifying the first 5 GET requests, as these requests correspond to logos and font files that are common across all webpages of the website, and are not of interest to the adversary. However, as soon as the client sent the 6th GET request (that corresponds to the HTML file), the adversary reduced the bandwidth to 800 Mbps and simultaneously started dropping 80% application packets in the server→client path. It does so for the next 6 seconds to force the client to send a Reset Stream signal to the server. After this point, the jitter value was increased to 80 ms additional delay per GET request packet so as to force the server to transmit the 8 consecutive image files in non-multiplexed form.

Object (O_{curr})	HTML	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8
$T(\text{Req } O_{curr}) - T(\text{Req } O_{prev})$ (ms)	500	780	0.4	2	0.3	0.1	0.3	2	0.5
$T(\text{Req } O_{next}) - T(\text{Req } O_{curr})$ (ms)	160	0.4	2	0.3	0.1	0.3	2	0.5	26
Success (%) of Adversary Target: One object at a time	100	100	100	100	100	100	100	100	100
Success (%) of Adversary Target: All objects at a time	90	90	85	81	80	62	64	78	64

Table 4.2: Prediction Accuracy: O_{prev} and O_{next} denote the objects requested by the client immediately before and after O_{curr} respectively. $T(\text{Req } X)$ denotes the time of request for object X by the client.

4.5.4 Results

Table 4.2 shows the accuracy of our adversary in identifying the 9 different objects of interest from the encrypted traffic. The 8 images can appear in any order in the survey result, depending on the user response, where I_i is the i^{th} image in the sequence. In absence of any adversarial intervention, the degree of multiplexing of each of these objects range from 80% to 99%. We consider our attack to be successful only when the adversary is able to bring down the degree of multiplexing of the object of interest to 0% and identify it from the encrypted traffic. In the context of *isidewith.com*, when the adversary is interested in just one object, our adversary was able to find it with 100% accuracy all the time as shown in Table 4.2. On the other hand, if the adversary were to be interested in identifying the entire sequence of images, the accuracy was highest for I_1 but gradually drops for later images. This is because the later images required adding more jitter which resulted in a broken connection. Additional jitter was required since the previous, current, and next GET requests are issued by the client in quick succession. For instance, request for I_5 has spacing of 0.1 ms and 0.3 ms with the previous and next requests as shown in Table 4.2.

4.5.5 Ethical Considerations

Adherence to ethical standards was strictly ensured during all experiments. All the volunteers were made aware about the goal of our research work and they had absolutely

no stake in the outcome of the survey. We also informed the developers of *isidewith.com* about our findings and the need to fix their website.

4.6 COUNTERMEASURES AND FUTURE WORK

The adversarial scheme presented in this chapter affirms the fact that HTTP/2 multiplexing is not a reliable and sound enough feature for designing defense solutions against website fingerprinting attacks. Privacy attack on users of a real website showed that even though the proposed technique was unsuccessful in a few cases, it was successful in predicting sensitive information in 90% of the cases. This led us to identify the following research directions in both offensive and defensive categories respectively.

4.6.1 Improving the Adversary

The success rate of our adversary drops to 60% in cases where a large number of consecutive objects are of interest. We believe that triggering the packet drops and jitter addition accurately will alleviate this problem, possibly using machine learning. Another possible extension would be to infer the object identity even when the object is partly multiplexed. Our preliminary experiments suggest that this is indeed possible, however, at the cost of employing complex analysis techniques. In our limited experience, we have observed that variations in network speed may result in innumerable ways in which the objects can be multiplexed, all of which need to be accounted for.

4.6.2 Exploring other types of web traffic

Exploring the suitability of our technique for other types of web traffic, such as streaming traffic, is an interesting direction. We strongly believe that our attack technique can supplement the existing attacks on HTTP/2 streaming (Wu *et al.*, 2019), which is also in its nascent stages.

4.6.3 Improving HTTP/2 privacy

Several HTTP/2 features such as server push and resource prioritization (Belshe *et al.*, 2015) that are not a function of the underlying network can be leveraged for privacy.

Server push. Server push allows an HTTP/2 server to send resources to a client even before the client requests for it. Though this feature was originally developed for improving HTTP/2 performance, in practice it has been found to consume excessive bandwidth due to inefficient implementations of the protocol (Lassey, 2020). However, an effective and intelligent implementation of the server push feature could help HTTP/2 servers randomize the request and response patterns of web objects. This randomness would also manifest itself in the encrypted traffic metadata, thereby confusing the network adversary.

Resource prioritization. Resource prioritization is a feature that would allow browsers to specify the order in which the HTTP/2 server would transmit the web objects requested by the client. Similar to the server push feature, resource prioritization also did not garner much popularity due to its complex and hard-to-use implementation (Meenan, 2018). However, an efficient and intelligent implementation of this feature would have helped HTTP/2 web clients opt for a different priority/order of object delivery every time, thereby confusing the adversary.

4.7 SUMMARY

Privacy schemes that rely on HTTP/2 multiplexing are not dependable and can be attacked by our proposed adversary. Our adversary is built on the general principles of HTTP/2 server operations and can be extended to other real-world websites/scenarios. Our contribution is especially significant since HTTP/2 is in the early stages of getting adopted worldwide, and communicating the pitfalls of recent research works on its privacy is important.

CHAPTER 5

SNOOPY: A NOVEL WEBPAGE FINGERPRINTING FRAMEWORK FOR MASS-SURVEILLANCE

Internet users are vulnerable to privacy attacks despite the use of encryption. In Chapter 3 and Chapter 4, we demonstrated attacks where a network adversary can infer sensitive information from the encrypted network traffic of *targeted internet users*. Other recent research works (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghi  tte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) have also been successful in demonstrating webpage fingerprinting¹⁰ attacks on individual users. But these works have been unsuccessful in extending their attack for mass-scale surveillance. While targeted attacks help a surveillant infer sensitive information about an individual, mass-scale surveillance would help the surveillant infer cumulative behavioral patterns of a large number of users of a website. Such information is sensitive, and its leakage might prove detrimental to the organization hosting the targeted website. In this chapter, we discuss the repercussions of mass-scale surveillance and explore the vulnerability of *targeted websites* to mass-scale webpage fingerprinting conducted by network adversaries.

The key challenges for an adversary in performing mass-scale webpage fingerprinting arise from (i) the sheer number of combinations of user behavior and preferences to account for, and; (ii) the bound on the number of website queries imposed by the defense mechanisms (e.g., DDoS defense) deployed at the website. These constraints preclude

¹⁰Webpage fingerprinting is an attack that analyzes encrypted traffic to identify the webpages visited by an internet use

the use of conventional data-intensive ML-based techniques. In this work, we propose Snoopy, a first-of-its-kind framework, that performs webpage fingerprinting for a large number of users visiting a website. Snoopy caters to the generalization requirements of mass-surveillance while complying with a bound on the number of website accesses (finite query model) for traffic sample collection. We show that Snoopy achieves $\approx 90\%$ accuracy when evaluated on most websites, across various browsing contexts. A simple ensemble of Snoopy and an ML-based technique achieves $\approx 97\%$ accuracy while adhering to the finite query model, in cases when Snoopy alone does not perform well.

5.1 BACKGROUND

Leakage of private information is one of the biggest concerns for Internet users today. Recent reports (Teiss, 2021; PTI, 2020) suggest that sensitive information that could cause imminent personal harm to Internet users, including banking passwords, salary details, health records, location information, and CCTV footage, have been leaked in the dark web. While privacy loss for personal information is easily perceivable, it is not so obvious for other types of information. For example, the Cambridge Analytica scandal showed that the political leanings of an ordinary individual may not be worthy to the attackers, but the political alignment of a larger demography can help them predict the outcome of an election (Chan, 2019). Such incidents show that some user information that are seemingly unimportant to an individual might inadvertently turn sensitive when collected on a *mass scale*.

Mass surveillance and its consequences. One of the largest sources of mass-scale information about personal preferences of Internet users are their web browsing activities (Seo and Zhang, 2000; Middleton *et al.*, 2001). Information such as the *identity of websites* visited by users from a demography could be useful to an attacker, for instance, to gauge the popularity of websites in the region. However, more fine-grained information such as the *identity of webpages* (a *website* can have many

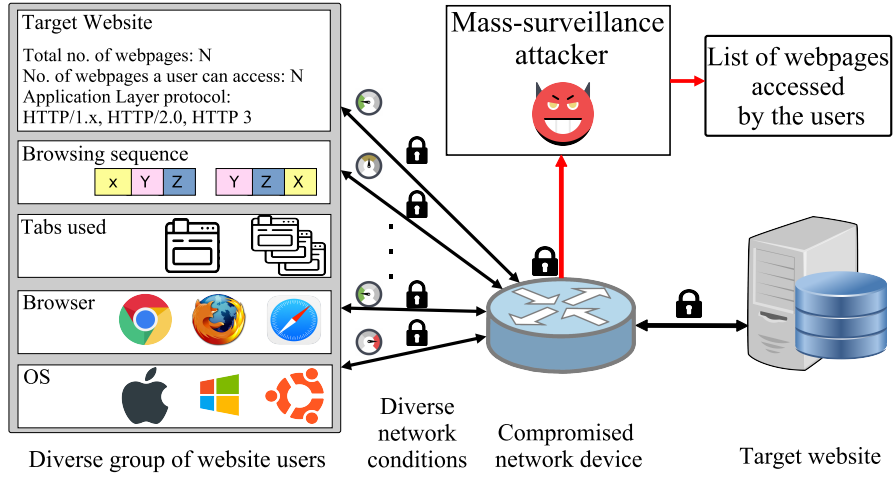


Figure 5.1: Snoopy, the proposed framework for webpage identification attack on a mass-scale

webpages) visited by the users on a targeted website could be much more useful to the attackers. For instance, a surge in the number of visitors to the “fixed-deposit” page of a bank website shortly after the announcement of a new policy is critical to the bank. This information, if leaked, could help competing banks estimate its growth and also design counter-marketing strategies. Another example is when the adversary identifies the most popular webpage of the website for placing malware. Such active attacks (phishing attacks (Hong, 2012), drive-by-download attacks (Le *et al.*, 2013) and denial-of-service attacks (Hussain *et al.*, 2003)) cause financial loss either to the hosting organization or its consumers or both. In this chapter, we focus on designing a *mass-surveillance method* that can potentially reveal such *fine-grained* sensitive information from a large number of Internet users based on their *web browsing activities*.

Straightforward methods to identify the webpages visited by the users of a website include compromising the end-user devices and the websites’ servers (e.g., extracting decryption key, installing fake certificates). However, these methods are highly intrusive and are easily detectable, making them unsuitable for long-term surveillance. Note that the importance of information grows manifold when collected over a longer period of time (Goel *et al.*, 2012; Olejnik *et al.*, 2012; Müller and Sedley, 2014; Bird *et al.*,

2020), since it ensures sufficient coverage of users and also reflects changing trends in their behavior. Encrypted Traffic Analysis (ETA) (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghiëtte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) (specifically, webpage fingerprinting) is the most promising non-intrusive method to collect such information for long periods of time by merely capturing the encrypted traffic exchanged between the websites' server and the end-users. The webpages are identified by formulating signatures that can identify them uniquely when accessed by users through encrypted channels. While several existing webpage fingerprinting attacks (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghiëtte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) target particular users, our goal is to perform mass-surveillance by observing the activities of a **large number of users** visiting a particular website. Figure 5.1 shows a high-level objective of this chapter. The main challenge with mass-surveillance is accounting for a diverse set of user behavior (e.g., number of tabs used), user preferences (e.g., OS/Browser used), and network conditions. We collectively refer to these factors as the *browsing context* in this chapter.

Existing techniques (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghiëtte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) for webpage fingerprinting are not suitable for mass-scale traffic analysis even with just HTTPS (i.e., without Tor/VPN). Practical mass-surveillance requires a *generalized model* that can identify webpages irrespective of the *browsing context* of a user. Building a generalized model that accounts for all possible browsing contexts is challenging. The primary hindrance in the generalization of existing works is the massive number of data points required, i.e., the number of

traffic samples required per webpage to account for all possible browsing contexts. For instance, building a generalized model for 3 operating systems and 3 browsers will require 9 times the traffic samples as compared to one-OS-one-browser scenario (Shen *et al.*, 2020). Moreover, for achieving the accuracy promised by DL/ML algorithms, collecting adequate number of traffic samples per scenario is also crucial. Existing ML-based (Shen *et al.*, 2020) and DL-based techniques (Sirinam *et al.*, 2018; Rimmer *et al.*, 2018) that aim at generalization of fingerprinting models have demonstrated the significance of a large number of data points in learning traffic features and building webpage fingerprints accurately. Additionally, any change in the website also requires fresh collection of traffic samples and re-training the model (Juarez *et al.*, 2014). Therefore, generalizing existing techniques requires assuming a query model that allows the adversary innumerable accesses to the website within a short span of time. However, in practice, we observed that the DDoS/DoS mitigation schemes at the websites detect and block IP addresses that make such attempts. Furthermore, IP addresses that cause anomalies in website analytics, including those involved in attempts to sample webpages repeatedly, are identified and blocked. Note that, an adversary can neither keep collecting traffic samples from a website over a long period of time to bypass these anomaly detection systems. This is because, the structure and contents of websites change significantly and frequently. For instance, the American financial services website B_5 (refer to Table 5.3) used in our experiments changed (added/deleted/modified) 65 of its webpages over a period of only 7 days. This problem has also been pointed out in a prior research work (Juarez *et al.*, 2014).

To work around the problem of collecting a massive number of traffic samples, prior works have assumed knowledge about one or more of the following, thereby compromising generalization:

1. **User interests** – Existing works (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Ghiëtte and Doerr, 2020) assume that the user

is only interested in a subset of webpages. In mass-surveillance, however, the adversary needs to account for a wide variety of user interests;

2. **User behavior** – Most of the existing works (Cai *et al.*, 2012; Gu *et al.*, 2015; Xu *et al.*, 2018) assume restricted user behavior such as single-tab browsing (Cai *et al.*, 2012; Shen *et al.*, 2020; Wang *et al.*, 2021) and traversing only a limited number of webpages per session (Ghi tte and Doerr, 2020). A few works account for the use of more than one tab (Gu *et al.*, 2015; Xu *et al.*, 2018), but they were not successful beyond 2 browser tabs. However, such assumptions are not valid in practical scenarios. Accounting for multi-tab browsing requires webpage-sequence based data collection, which in turn increases the traffic samples exponentially. Further, during multi-tab browsing, a user might even load two webpages in parallel. Existing studies (Dubroy and Balakrishnan, 2010; Huang and White, 2010) have discussed the parallel multi-tab browsing behavior of Internet users in further details;

3. **User preference** – Existing works (Miller *et al.*, 2014; Hayes and Danezis, 2016; Wang *et al.*, 2021) assume prior knowledge about the OS and the browser used, as well as knowledge about browser caching (on/off) and cookie values. However, mass-surveillance necessitates accounting for various combinations of OS and browser configurations; and,

4. **Network conditions** – Existing works build prediction models for a particular network condition, which cannot be generalized to other network conditions. However, mass surveillance requires the prediction model to work across geographical locations and service providers.

Na ve attempts at generalization without the addition of adequate number of data points for covering additional scenarios result in poor prediction accuracy. For instance, we witnessed a drastic drop in prediction accuracy (e.g., from 96% to 76% in case of

Wfin (Yan and Kaur, 2018)) of existing ML-based works when we broadened the user interest from a small number of webpages¹¹ to a large number (refer to Figure 5.7 in § 5.6.4.3). Further, we also noticed a drop in prediction accuracy (e.g., from 78% to 58% in case of ML_OPS (Chapman and Evans, 2011)) when the number of training samples is reduced from 10 to 3 per webpage (refer to Figure 5.6 in § 5.6.4.2). Likewise, when we tried to generalize existing works in terms of user preference (caching on or off), we found a drop in accuracy (as shown in Table 5.5 in § 5.6.4.4) when a model built for one scenario (caching on) was used to predict webpages accessed from a different scenario (caching off). Therefore, availability of a limited number of training samples imposes restrictions on the use of existing DL/ML based techniques.

In our work, we propose *Snoopy*, a practical webpage fingerprinting framework, that meets the generalization requirements of mass-surveillance, while assuming a *finite query model*. With limited number of traffic samples, accounting for the numerous scenarios encountered in mass-surveillance is challenging. Snoopy uses domain knowledge about the transport and application protocols to collect traffic samples in a focused manner. For instance, Snoopy uses *encrypted web resource size*, a simple feature used in ETA, that remains largely unaffected by changes in network conditions, eliminating the need to cover this scenario altogether. For generalizing across other scenarios Snoopy relies on static analysis of web object sizes, HTML code and headers to estimate the expected fingerprints. In this chapter, we analyze the capability of a simple model such as ours in terms of generalization in cases where there are practical constraints for using ML/DL techniques. We show that Snoopy was able to achieve more than 90% accuracy for most of the websites we considered, when tested on traffic samples from a diverse set of browsing contexts. For the few websites where Snoopy achieved comparatively lower accuracy ($\approx 80\%$), we show that it is possible to improve the accuracy to as high as 97% by using an ensemble of Snoopy and an ML-based technique, that complies with the constraints of a finite query model. To the best of our knowledge, ours is the first work to

¹¹We conducted these experiments on a popular bank website.

attempt webpage fingerprinting attack for mass surveillance.

The rest of the chapter is organized as follows. In § 5.2 we present the literature related to our work and motivate the need for Snoopy. Next, we state our adversarial capabilities and assumptions, followed by a high level overview of Snoopy in § 5.3. Then we describe the design choices made for Snoopy in § 5.4. This is followed by a detailed description of Snoopy in § 5.5. Thereafter, we describe the implementation and evaluation strategies of Snoopy and present a detailed analysis of the experimental results in § 5.6. We discuss the consequences of mass-surveillance and suggest some possible countermeasures to the proposed technique in § 5.7. Finally, we summarize the chapter in § 5.8.

5.2 RELATED WORK

With growing security concerns, traffic on the Internet is encrypted. Thus, our work is related to the literature on Encrypted Traffic Analysis (ETA) techniques, which can be broadly classified into **(1)** Website identification attacks – these works (Liberatore and Levine, 2006; Panchenko *et al.*, 2011; Gong *et al.*, 2012; Wang *et al.*, 2014; Zhuo *et al.*, 2017; Sirinam *et al.*, 2019; Rahman *et al.*, 2020) aim to infer coarse-grained web browsing information, i.e., the identity of websites browsed over Tor/VPN; and, **(2)** Webpage identification attacks – these works (Cheng and Avnur, 1998; Sun *et al.*, 2002; Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghi tte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) aim to infer fine-grained web browsing information of users from HTTPS traffic, i.e., the identity of webpages browsed within a website. The techniques used for webpage identification are very different from those used for website identification. On one hand, website identification require features (e.g., Round trip time) that vary across websites but remain consistent for webpages within a website. In contrast, webpage identification attacks (Cheng and Avnur, 1998; Sun *et al.*, 2002; Danezis, 2010; Chapman

Existing works	Mass-surveillance Requirements					
	Generalization Requirements					Compliance with a finite query model
	Caching	Cookies	Network conditions	User interests	Multi-tab browsing	
(Cheng and Avnur, 1998), (Sun <i>et al.</i> , 2002)	✗	✗	✓	✓	✗	✓
(Cai <i>et al.</i> , 2012), (Danezis, 2010), (Chapman and Evans, 2011), (Gong <i>et al.</i> , 2012)	✓	✗	✗	✗	✗	✓
(Miller <i>et al.</i> , 2014), (Hayes and Danezis, 2016)	✓	✓	✗	✗	✗	✓
(Gu <i>et al.</i> , 2015), (Zhuo <i>et al.</i> , 2017)	✓	✓	✗	✓	✓	✗
(Xu <i>et al.</i> , 2018)	✗	✗	✗	✗	✓	✗
(Alan and Kaur, 2019)	✗	✓	✓	✗	✗	✓
(Ghiëtte and Doerr, 2020)	✓	✓	✗	✓	✗	✗
(Panchenko <i>et al.</i> , 2016), (Sirinam <i>et al.</i> , 2019)	✓	✓	✓	✓	✗	✗
(Wang <i>et al.</i> , 2021)	✗	✓	✗	✓	✗	✓
(Shen <i>et al.</i> , 2020)	✗	✗	✗	✓	✗	✓
Snoopy	✓	✓	✓	✓	✓	✓

Table 5.1: Compliance of webpage fingerprinting techniques with the requirements for mass-surveillance

and Evans, 2011; Cai *et al.*, 2012; Miller *et al.*, 2014; Gu *et al.*, 2015; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Xu *et al.*, 2018; Yan and Kaur, 2018; Alan and Kaur, 2019; Ghiëtte and Doerr, 2020; Shen *et al.*, 2020; Wang *et al.*, 2021) require features (e.g., HTML size) that vary across the webpages within a website, which can help distinguish between them. Blurring the lines between the two classes, some website identification attacks (Liberatore and Levine, 2006; Panchenko *et al.*, 2011; Wang *et al.*, 2014; Zhuo *et al.*, 2017; Sirinam *et al.*, 2019; Rahman *et al.*, 2020) internally use webpage fingerprinting techniques to distinguish between the homepages of thousands of websites. We note that such techniques do not inherit the same set of challenges that a webpage identification attack faces (e.g., restrictions on number of accesses to the website). In this work, we focus on webpage identification attacks on a given website, subject to constraints on the number of webpage accesses.

In this section, we revisit the literature on webpage fingerprinting and assess their suitability for mass-scale HTTPS traffic analysis. Webpage fingerprinting on mass-scale traffic requires a generalized classifier which could account for diversity in client behavior and network conditions. Diversity in client behavior, for example, in terms of browser/OS/device used, number of tabs open, and the sequence in which webpages are accessed, results in different fingerprints for the same webpage (Alan and Kaur, 2019). Diversity in network conditions in terms of jitter, bandwidth and packet drop rates also affects the webpage fingerprints (Sirinam *et al.*, 2019). Accounting for these diversities across so many factors is challenging due to practical restrictions on the number of website accesses. In this context, Table 5.1 classifies the prior works in terms of their generalization capabilities. The earliest webpage fingerprinting techniques (Cheng and Avnur, 1998; Sun *et al.*, 2002) could successfully generalize webpage fingerprints for simple contemporary websites that did not use caching, cookies, and used static webpages with limited number of resources. Such works used basic features such as size of client request packets and sequences of encrypted object sizes, coupled with elementary statistical methods.

The prevalence of web caching on the Internet prompted subsequent research works (Danezis, 2010; Chapman and Evans, 2011; Cai *et al.*, 2012) to start the use of webpage-sequence fingerprinting to account for variations in traffic patterns based on the previously accessed webpage(s). However, the applicability of webpage-sequence fingerprinting techniques is restricted only to targeted attacks and are not suitable for mass surveillance due to the following reasons – **(1)** It necessitates an enormous number of website accesses as compared to individual webpage based fingerprinting techniques, even for medium-sized websites; and, **(2)** these works used features such as resource ordering (Danezis, 2010) and traffic burst patterns (Chapman and Evans, 2011; Cai *et al.*, 2012) for fingerprinting webpage sequences. Such features are not consistent across varying network conditions and user behavior (e.g. number of browser tabs open), and hence, require an estimate of the network speed and browsing behavior of the victim.

Such fingerprinting techniques were useful in the context of surveillance on a small set of targeted users, since the attackers had knowledge about the targeted victim's browsing behavior and network conditions. However, assuming interests of website users, their browsing behavior (sequential or single-tab browsing) and their network conditions is impractical for mass-surveillance in realistic scenarios (Juarez *et al.*, 2014).

The use of cookies by the next generation websites further complicated the process of webpage fingerprinting, even for targeted attacks. Tracking cookies embedded inside web resources and session cookies included in application layer headers result in resource size variations. To account for these variations, subsequent research works (Miller *et al.*, 2014; Hayes and Danezis, 2016) either used complex techniques (combination of clustering algorithms, Gaussian distribution, Hidden Markov Model and Viterbi algorithm) complemented by huge datasets (Miller *et al.*, 2014) or simplified their assumptions about user behavior (Hayes and Danezis, 2016). The former technique (Miller *et al.*, 2014) is too restrictive about user interests since they consider only a small set of webpages to make data collection practical. On the other hand, the latter one (Hayes and Danezis, 2016) assumes impractical user behavior such as single tab browsing without caching. However, these techniques are restricted only to targeted attacks, and are not suitable for mass surveillance due to the following reasons – **(1)** The goal of mass-surveillance is to understand the interests of the users of a website. Therefore, restrictive assumptions about user interests are unreasonable in the context of mass-surveillance, and; **(2)** Restrictive assumptions regarding the browsing behavior (for e.g., the number of tabs used, OS/Browser used, and the browser configuration) of a diverse set of website users are also unreasonable. Therefore, these techniques cannot be used for analysis of encrypted traffic on a mass scale.

Recent works (Gu *et al.*, 2015; Panchenko *et al.*, 2016; Zhuo *et al.*, 2017; Xu *et al.*, 2018; Sirinam *et al.*, 2019; Alan and Kaur, 2019; Shen *et al.*, 2020; Ghi  tte and Doerr, 2020; Wang *et al.*, 2021) on webpage fingerprinting have recognized the importance

of practical mass surveillance. Most of these works acknowledge the necessity of accounting for diversity in user interests (Panchenko *et al.*, 2016; Ghi tte and Doerr, 2020; Wang *et al.*, 2021), user behavior (particularly, a wide variety of user interests and multi-tab browsing) (Gu *et al.*, 2015; Zhuo *et al.*, 2017; Xu *et al.*, 2018), OS/browser settings (Alan and Kaur, 2019) and network conditions (Sirinam *et al.*, 2019) while performing encrypted traffic analysis on mass-scale web traffic. However, it is to be noted that each of these works can only generalize for at most one of these factors. This is because, for most of these works (Zhuo *et al.*, 2017; Alan and Kaur, 2019; Sirinam *et al.*, 2019), generalizing for even one of these factors using the features and/or the techniques presented in these works requires collecting a massive amount of traffic samples, coupled with a heavyweight ML algorithm. This would result in a large number of website queries (i.e., the number of website accesses done by the adversary), as well as a high bootstrap time every time the website contents change or a new OS or browser or firmware becomes popular. For instance, one of these works (Alan and Kaur, 2019) account for diversity in browser, OS and device used by the client at the cost of an immensely high bootstrap time.

As summarized in Table 5.1, existing ETA techniques have complied with a finite query model at the cost of restrictive assumptions about the interests and browsing behavior of the users, and vice-versa, and are suitable for mass surveillance. We propose *Snoopy*, an ETA technique that is designed with the goal of meeting the generalization requirements and complies with a finite query model, both of which are essential for mass-surveillance.

5.3 THE SNOOPY FRAMEWORK

In this section, we present *Snoopy*, our proposed adversarial framework for mass-surveillance through webpage fingerprinting. We first describe the capabilities of the adversary, then define the scope of our work, and finally provide a high level overview of *Snoopy*.

5.3.1 Adversary Capabilities

Our adversary is a compromised network device on the client-server path that can (1) access unencrypted header fields of both control and data packets; and, (2) observe the traffic characteristics, such as the size of encrypted packets. Such an adversary model is not only realistic but also common today. In the real world, this translates to a rogue Internet Service Provider (ISP) or a malicious entity who compromises an ISP router.

5.3.2 Assumptions and Scope

We make the following assumptions about the context in which Snoopy is to be employed.

- We do not consider websites that host dynamic or highly personalized content for each user, for e.g., social media websites and web search engines, as targets for our attack;
- The adversary does not have the capability to decrypt the traffic of real website users. Most real-world adversaries cannot decrypt web traffic except maybe those involving government agencies or authorized middle-boxes (Shahbaz, 2018); and,
- Snoopy must compromise a network device that has access to all the encrypted Application layer packets (referred to as *traffic trace*) exchanged between the client and the server for the entire duration of a browsing session. For instance, our framework cannot be used when route flapping (dynamic change of route due to, for instance, unreliable connections) occurs on the server-client path.
- The scope of our work does not include inferring the intent and interest of the website visitors from the webpages inferred by Snoopy. Rather, we believe that existing works (Garimella *et al.*, 2021) on behavioral analysis of website users can be used by an adversary for this purpose.

5.3.3 High Level Overview

We now provide a high level overview of Snoopy – our mass-surveillance framework. Snoopy comprises the *Snoopy Database* and two functional modules, namely, *Webpage*

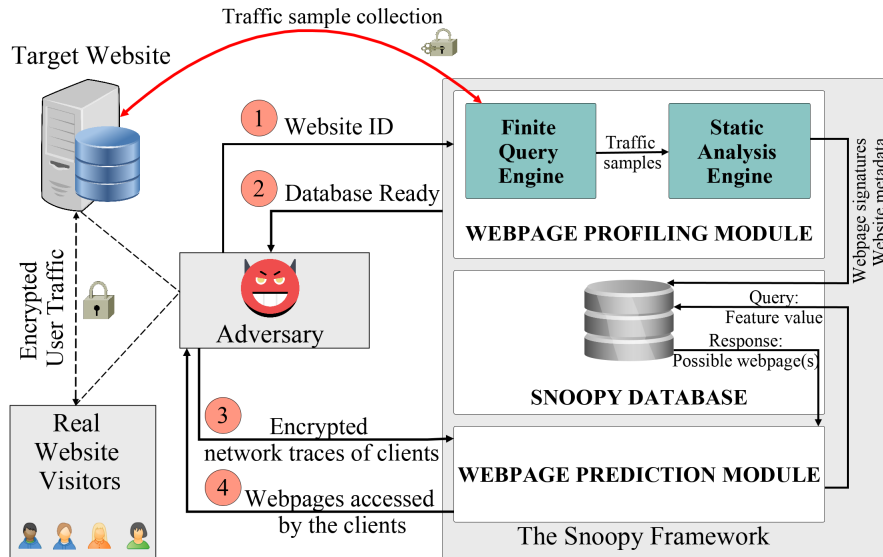


Figure 5.2: The components of Snoopy

Profiling Module and Webpage Prediction Module. The Snoopy Database is meant to store information (i.e., webpage fingerprints and additional website metadata) required for carrying out the predictions, and the two functional modules are used by the adversary for profiling webpages and predicting the identities of webpages accessed by the users. Figure 5.2 shows the different components of Snoopy and how an adversary interacts with them.

To populate the Snoopy Database, the adversary first triggers the Webpage Profiling Module of Snoopy with the Website ID (IP address or homepage URL) of the target website, as shown in Step 1 of Figure 5.2. The goal of this module is to gather information about the website that will be useful to the Webpage Prediction Module. The input to this module is the website identifier (IP address or homepage URL) and the set of features to be used for webpage fingerprinting. First, the finite query engine in this module performs a focused traffic sample collection from the target website. Subsequently, this traffic is decrypted and both the encrypted and decrypted versions of the traffic samples are passed on to the static analysis engine. While Snoopy does not require the decryption keys of users during the attack, it still needs to decrypt the traffic samples that it generates on its own during the profiling stage. This is necessary to identify the plaintext resources

(e.g., HTML, javascript, images) of each webpage from their encrypted counterparts. The final output of the Webpage Profiling module includes information pertaining to (1) the structure of the website; (2) resource download sequences for each webpage in the website; (3) signature of these web resources; and, (4) other relevant website metadata, such as cache-ability and cookie information of the web resources.

Once the database has been populated, Snoopy notifies the adversary that it is ready to be used for prediction, as shown in Step 2 of Figure 5.2. When the adversary wants to predict the webpages accessed by a user from their encrypted traffic trace, it triggers the *Webpage Prediction Module* of Snoopy. This module takes an encrypted traffic trace T of a user as input, as shown in Step 3 of Figure 5.2, and predicts the webpages that are accessed in T using the information stored in the Snoopy database. Snoopy performs this prediction in two steps. First, it extracts the feature values from the input trace T and performs a lookup on the Snoopy Database to retrieve the sequence of candidate web resources present in T . Next, Snoopy uses this sequence and the website metadata to predict the final set of webpages, which it returns to the adversary (Step 4 of Figure 5.2).

The most unique and crucial feature of Snoopy is its ability to comply with a finite query model, while retaining its generalization capability at the same time. Achieving this balance between generalization and a finite number of queries is extremely challenging. Snoopy attempts to solve this problem using the following:

(1) Predictable features: It uses a feature that exhibits negligible or predictable variation across browsing contexts. The predictability of the feature values allows Snoopy to fingerprint webpages in different browsing contexts even with a finite number of traffic samples, complying with a finite query model. The feature used in Snoopy and the justification behind this choice are discussed in further details in § 5.4; and,

(2) Focused data collection and feature value estimation: For collecting traffic samples from the targeted website, Snoopy uses a focused data collection method to

account for certain browsing contexts, which has been discussed in details in § 5.5.1. Snoopy estimates the variations in feature values in different browsing contexts using static analysis of HTTP headers and payload. At the time of prediction, Snoopy uses these estimated feature values (from the Snoopy Database) based on the user’s browsing context. The feature value estimation and prediction techniques used by Snoopy are detailed in § 5.5.2.

5.4 SNOOPY DESIGN: PREDICTABILITY OF FINGERPRINTS

Snoopy needs an encrypted traffic feature that either exhibits no variation or predictable variation across different browsing contexts, in order to minimize the number of traffic samples required during webpage profiling. In other words, the traffic features used for webpage profiling need to be *stable* across different browsing contexts. For this, we evaluated different classes of traffic features (e.g., timing side-channels and traffic burst patterns) that are widely used in prior ML-based works (Gong *et al.*, 2012; Miller *et al.*, 2014) for targeted attacks and assessed their stability. It is known and also shown in our evaluations presented in the Appendix that such traffic features have poor stability and are therefore not predictable. For instance, the traffic burst pattern corresponding to a webpage download changes with variations in network conditions and the number of parallel browser tabs used by a user.

Snoopy uses *sequence of encrypted web resource sizes* as the fingerprint of a webpage. The size of an encrypted resource is computed as the sum of TLS segment sizes of all packets carrying the resource. Since each user might have a different browsing context, Snoopy first focuses on *understanding* the effect of various browsing contexts on the fingerprint. While some of the factors of the browsing context affect the encrypted resource size, some others affect the sequence in which the resources are downloaded. An analysis of these factors helps Snoopy *statically* estimate the variations caused by each of these factors, thereby eliminating the need for collecting traffic samples for all

possible browsing contexts. We now explain how the size of encrypted resources and their download sequence change based on the browsing context of a user:

(1) Operating system (OS) – The choice of OS affects the TLS segment size of packets in multiple ways. First, the TLS segment size of a web resource depends on the TLS implementation on the corresponding OS. Different OS'es start with the same TLS record but they break them into segments of different sizes. Although one might not expect this to affect the TLS segment size, we noticed minor differences when the sum of TLS segment sizes is calculated. A deeper analysis revealed that the number and size of segments affects the metadata associated with every TLS segment in a number of ways, which are, to our advantage, predictable. First, there are some TLS headers added to every TLS segment. Therefore, if a record is broken into several small TLS segments, the total number of TLS header bytes for all the segments would be greater than the total number of TLS header bytes added in case the record was broken into fewer larger segments. Second, a variable field on the HTTP header, namely, the user agent string that carries the name of the OS also affects the TLS record size.

(2) Browser – The browser name indirectly affects the TLS segment size since it is also a part of the User Agent string. As different browser names have different lengths, they affect the length of the User Agent string, which in turn, results in variation in the TLS record sizes.

(3) Browsing sequence – The sequence in which a user browses the webpages affects both the TLS record sizes and the download sequence of the web resources, particularly when caching and cookies are allowed (enabled in most browsers by default).

- **Caching:** When a user visits a webpage containing a resource that was previously downloaded, the resource may not get downloaded if resource caching is enabled. This results in variation in resource download sequence. For instance, if a user visits

webpage W_X (composed of resources r_1 and r_2), followed by webpage W_Y (composed of resources r_1 and r_3), the overall resource download sequence would be $r_1-r_2-r_3$. On the other hand, if caching was disabled, it would result in the following sequence: $r_1-r_2-r_1-r_3$.

- **Cookies:** When a user allows a website to use cookies, the server sends a *session cookie* in the HTTP header along with the first resource delivered during the browsing session. Addition of the session cookie increases the TLS record size of the first resource downloaded during a browsing session. Therefore, a resource would have a larger TLS record size if it gets downloaded at the beginning of a browsing session, as compared to its TLS record size when it gets downloaded at a later point during the browsing session. Furthermore, another type of cookie, called the *tracking cookie*, affects the payload size of the transmitted resource. Tracking cookies hold information about the browsing behavior of a user such as the URL of the previously browsed webpage(s) in the session. Since different webpage URLs have different lengths, the variation in TLS record size due to tracking cookies depend on the webpage(s) last visited by a user. Also, tracking cookies hold a null value for the first resource delivered during the browsing session. Again, this causes the TLS record size of a resource to vary depending on the user's browsing sequence. For instance, if the size of a resource r_1 is s_1 and the size of the session cookie is s_c , then the TLS record size of r_1 would be $(s_1 + s_c)$ if r_1 is the first resource to be downloaded. On the other hand, if r_1 is downloaded after the user has visited a webpage of URL length t_c (also, the tracking cookie size) in a browsing session, the TLS record size of r_1 would be $(s_1 + t_c)$.

(4) Application layer protocol – Grouping packets that carry a resource is critical for computing the encrypted resource size. For HTTP/1.x, packets belonging to a resource have the same TCP ACK number, making the process straightforward. However, it is not the same for HTTP/2 and HTTP/3 websites due to pipelining and multi-threaded server

operations. With multi-threading, packets belonging to two different resources could get interleaved within the same TCP stream (in HTTP/2 (Belshe *et al.*, 2015)) or QUIC stream (in HTTP/3 (M. Bishop, 2020)). To handle such complex scenarios, Snoopy adopts the technique described in our recent work (Mitra *et al.*, 2020) for computing encrypted resource sizes in HTTP/2 websites. For HTTP/3 websites, Snoopy drops the QUIC connection establishment packets so that the communication protocol falls back to HTTP/1.1 or HTTP/2.

(5) *Parallel tabs* – Browsing on concurrent tabs affects the sequence of downloaded resources. For instance, when a user browses two webpages, say W_X and W_Y one after another on two tabs, the resources of W_X get downloaded first, followed by the resources of W_Y . On the other hand, if the user opens the two webpages in two parallel browser tabs, the resources of W_X and W_Y would get downloaded in an interleaved fashion. In such cases, the attacker faces the additional challenge of identifying the resources download sequence corresponding to each webpage from the interleaved sequence of encrypted resources, which makes the webpage prediction process more challenging.

(6) *Network conditions* – A congested link results in packet transmission delays, packet drops, and blocking of new connections. Among these, packet drops affect the encrypted size of resources. The dropped packets may or may not get re-transmitted, based on the nature of the resource. In the event of packet re-transmissions, the re-transmitted packets can be easily assembled using state-of-the-art network protocol dissectors so that Snoopy can still work. Therefore, computation of the encrypted resource size by Snoopy is not affected in this case. However, in rare situations when a large number of dropped packets are not re-transmitted or results in a broken connection, Snoopy cannot compute or predict the resource size correctly. Severe network congestion may also result in route flapping. In that case, the attacker would not be able to access the complete network trace unless it has control over the new route

too. Such circumstances cannot be handled by Snoopy.

Presently, Snoopy has been configured to use *sequence of encrypted web resource sizes* as the feature for fingerprinting webpages. We incorporate the aforementioned knowledge in the design of Snoopy to make it compliant with a limited query model while allowing it to be generalized at the same time. However, if a feature that is more stable is discovered, Snoopy can be configured to use that instead.

5.5 SNOOPY DESIGN: A DEEP DIVE

In this section, we provide a detailed description of the two functional modules of Snoopy, viz., the Webpage Profiling Module and the Webpage Prediction Module, and how they use the insights from § 5.4. As discussed in § 5.4, the present implementation of Snoopy uses only one feature – sequence of encrypted resource sizes. Therefore, in the rest of this section, we describe the functional modules of Snoopy in the context of this feature.

5.5.1 Webpage Profiling Module

The input to this module is the target website identifier *WebsiteIP*, the encryption function *EF*, and the webpage fingerprinting feature *F* (i.e., encrypted resource size sequence). The output of the webpage profiling module includes the following information **(1)** the *structure of the website*, represented by a graph $\mathbb{G} = (\mathbb{W}, \mathbb{E})$, where, $\mathbb{W} = \{w_1, w_2, \dots, w_n\}$ is the set of n distinct webpages in the website and $\mathbb{E} = \{(w_i, w_j) \mid w_j \text{ is directly navigable from } w_i\}$ is the set of directed edges; **(2)** a *resource-map* $\mathbb{RM} : \mathbb{W} \rightarrow \mathbb{R}$ that shows the relationship between webpages in $\mathbb{W}$ and the web resources in $\mathbb{R}$, where $\mathbb{R} = \{r_1, r_2, \dots, r_m\}$ is the set of web resources that constitute the targeted website ; **(3)** the *set of resource download sequences* for all n webpages in the website, denoted by $\mathbb{S} = \{S_1, S_2, \dots, S_n\}$, where S_i be the sequence of resources requested by the client when a user accesses the webpage w_i ; **(4)** *signatures of web resources*, i.e., the encrypted size of each resource; and, **(5)** other *relevant website metadata*, such as the cache-ability and cookie information of the web resources. We now describe the working of the Webpage Profiling Module using

Algorithm 1: PROFILE_WEBSITE

input : $WebsiteIP$, EF , F **output** $\mathbb{G}$, $\mathbb{S}$, $\mathbb{RM}$, $reverseFeatureDB$, $cookie_var$

```
:  
/* automated website crawling */  
1  $\mathbb{G} = GET\_WEBSITE(WebsiteIP);$   
/* parsing hypertext */  
2  $\mathbb{S}, \mathbb{R} = GET\_RESOURCES(\mathbb{W});$   
/* webpage-resource relation */  
3  $\mathbb{RM} = MAP\_RESOURCES(\mathbb{W}, \mathbb{R});$   
/* traffic sampling for webpage access */  
4  $TSamples = COLLECT\_SAMPLES(\mathbb{W}, F);$   
/* unique identifiers for encrypted resources */  
5  $featureDB = BUILD\_SIGNATURES(F, \mathbb{R});$   
/* look-up table for prediction */  
6  $reverseFeatureDB = CONSTRUCT\_DICTIONARY(featureDB);$   
/* variable cookie fields */  
7  $cookie\_var = COMPUTE\_COOKIE\_VAR(\mathbb{G}, featureDB, EF)$ 
```

Algorithm 1 in the following sections.

5.5.1.1 Extracting Structure of the Website

The $GET_WEBSITE$ procedure in Step ① in Algorithm 1 uses automated website-crawling and webpage parsing techniques to construct $\mathbb{G}$, the graph representation of the website of interest. A vertex u in $\mathbb{G}$ has a directed edge to another vertex v if u represents a webpage that has embedded hyperlinks to help users navigate directly to the webpage represented by v . Following this, the $GET_RESOURCES$ procedure in Step ② parses each webpage in $\mathbb{W}$ to extract its resource download sequence and compiles $\mathbb{R}$, the list of embedded web resources present in the website. The set of resource download sequences thus retrieved is denoted by $\mathbb{S}$, and the unique set of embedded resources is denoted by $\mathbb{R}$. The $MAP_RESOURCES$ procedure in Step ③ creates a bipartite graph $\mathbb{RM}$ where the vertices consist of elements in $\mathbb{W}$ and $\mathbb{R}$. A webpage w_i is mapped to a web resource r_j when r_j is embedded in w_i , and this mapping is denoted by an edge between the vertices corresponding to w_i and r_j . Figure 5.3 shows an example of $\mathbb{R}$, $\mathbb{W}$, $\mathbb{RM}$, and $\mathbb{G}$ for a website.

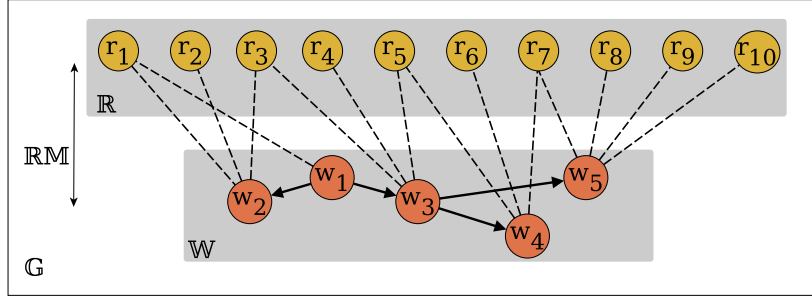


Figure 5.3: Representation of the structure of a sample website

5.5.1.2 Focused Traffic Sample Collection and Formulating Signatures of Web Resources

For building webpage fingerprints Snoopy requires a dataset comprising traffic samples that capture how different web resources manifest themselves on an encrypted communication channel in different browsing contexts. The COLLECT_SAMPLES procedure in Step ④ of Algorithm 1 builds this dataset. The inputs to this step are the set of webpages $\mathbb{W}$ and the fingerprinting feature F , i.e., sequence of encrypted resource sizes. As discussed in § 5.4, the values of this feature vary with different browsing contexts. We now discuss how the COLLECT_SAMPLES procedure handles these different scenarios.

- (1) **Factors causing no variation in feature values.** As discussed in § 5.4, the values of encrypted resource sizes remain invariant to changes in factors such as network conditions. Snoopy does not require traffic samples to account for such factors that do not affect the feature values.
- (2) **Factors causing predictable variation in feature values.** As discussed in § 5.4, the variations in encrypted resource sizes due to factors such as operating system, browser, and number of parallel tabs used are deterministic in nature. Differences in feature values due to these factors can be estimated by Snoopy from the domain knowledge of network and browser protocols incorporated in its design. Therefore, no extra traffic samples need to be collected for accounting for such browsing contexts.

The COLLECT_SAMPLES procedure collects traffic samples for any one browsing context (for example, Firefox browser). Snoopy can estimate the feature values for the other browsing contexts (for example, Google Chrome browser) using static analysis techniques described in § 5.5.2.

- (3) Factors causing website-specific variation in feature values.** Snoopy requires traffic samples to estimate feature values due to variations in factors that depend on website design – the resources that can be cached and the information that the cookie carries. To handle this, the COLLECT_SAMPLES procedure collects traffic samples from each webpage once with caching on and cookies allowed and once with caching off and cookies prohibited.

To collect the traffic samples for estimating the feature values, the COLLECT_SAMPLES procedure spawns multiple dummy clients to access the webpages, one at a time, in separate sessions. During each webpage access, the encrypted traffic of each client is captured by a network monitoring tool from the beginning till the end of the browsing session. Subsequently, Snoopy processes every traffic trace and splits them into multiple sub-traces, corresponding to each resource r_i . Note that the packets carrying the same resource can be easily identified from an encrypted trace, since they have the same TCP acknowledgement number. The actual resource that is carried by the packets in a sub-trace are found by decrypting the sub-trace with the Transport Layer Security (TLS) keys used by the dummy clients. The sub-traces and the resources they correspond to are stored in the dataset named *TSamples*, where each entry is of the form $\{sub-trace, r\}$. Once the *TSamples* dataset is built, the feature values of the resources can be extracted for building the resource signatures.

5.5.1.3 Constructing Feature and Resource Databases

In Step ⑤ of Algorithm 1, the BUILD_SIGNATURES procedure extracts the F value (encrypted size) of each resource from its corresponding sub-trace, which essentially captures the effect of the encryption function EF. For instance, if a sub-trace contains

two packets of size 50 bytes each, the encrypted size of the corresponding resource would be 100 bytes. For every extracted sub-trace in *TSamples*, an entry of the form $\langle r_i, sig_i \rangle$ is made in the signature database *featureDB*, where sig_i is the feature value of the resource r_i .

During prediction, an inverse mapping (i.e., EF^{-1}) from an observed feature value (f_i) to the corresponding resource (r_i) will be required. However, constructing the inverse mapping EF^{-1} is not straightforward.

Challenges in constructing EF^{-1} . Constructing EF^{-1} is challenging because it is not possible to perform a one-to-one mapping between encrypted resource sizes and the corresponding resources. This is because, as we discussed in § 5.4, the sizes of encrypted resources vary due to variations in user browsing contexts. Therefore, a single resource might have different sizes in different browsing contexts. Likewise, a given size value might correspond to multiple resources in a given browsing context. Moreover, due to the finite query model, Snoopy cannot collect traffic samples (data points) for all possible browsing contexts.

The `CONSTRUCT_DICTIONARY` procedure builds a reverse database *reverseFeatureDB* out of *featureDB* by taking into account the many-to-many mapping between encrypted resource sizes and resources. Each entry in *reverseFeatureDB* is of the form $\langle f_i, L_i \rangle$, where f_i corresponds to a unique FS value and L_i is a list of resources that have this value. Note that L_i may include (1) multiple resources that have the same signature and (2) multiple instances of the same resource (each corresponding to a dummy client access). Figure 5.4 shows an instance of *featureDB* and *reverseFeatureDB* constructed from the *TSamples* collected corresponding to a website.

During prediction, to narrow down on the possible resources corresponding to a sub-trace, Snoopy uses the meta information (i.e., user-agent string, cache-ability, and

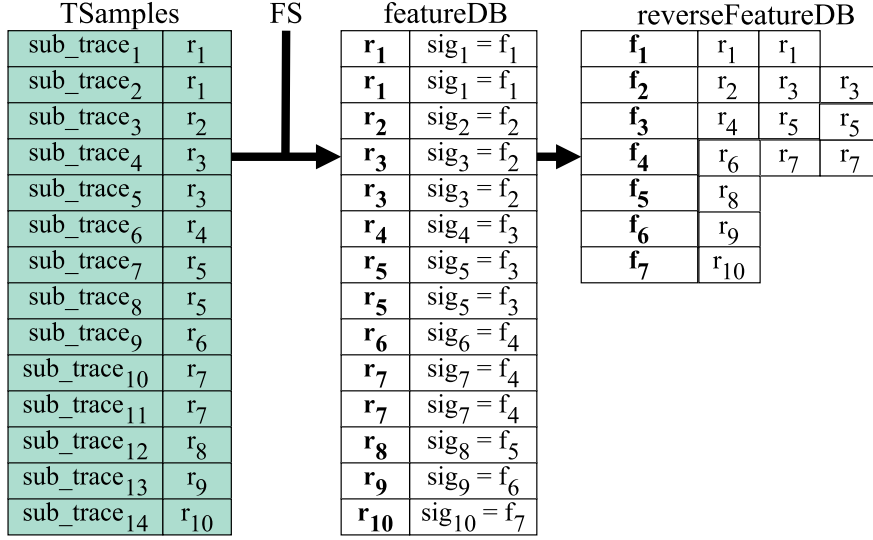


Figure 5.4: Constructing featureDB and reverseFeatureDB from TSamples and FS

cookies) to estimate the possible encrypted resource sizes in a given browsing context. Although the estimations for cache-ability and user-agent string can be determined easily (refer § 5.5.1.2), for cookies, we need to collect additional traffic samples. The COMPUTE_COOKIE_VAR procedure (Step ⑦ of Algorithm 1) processes these traffic samples to extract and store the meta information pertaining to cookies as follows:

(1) Characterizing tracking cookies. Tracking cookie is an integral part of the source code of a webpage that records the sequence of webpages previously visited by a user. To account for the impact of tracking cookies on the size of encrypted resource r_i , we model the parameter $c_t(r_i)$. Note that there is no tracking cookie during profiling time, since each webpage is accessed in an individual session. The possible variation in feature values that arises due to embedded tracking cookies in a user trace is calculated by parsing the source code of the web resource (if it is in text format). Recently developed tools such as CookieCheck (Trevisan *et al.*, 2019) can also be used for automated detection of tracking cookies in web resources. Note that only resources that are in text format carry tracking cookies. For each web resource that carries a tracking cookie, the COMPUTE_COOKIE_VAR procedure first identifies the set of URLs from which a user

can navigate to the resource. Subsequently, for each of these URLs, it first computes the size of the resource *with* the tracking cookie when it is not encrypted. To compute the corresponding resource size when encrypted, we use linear interpolation of known plain-text and encrypted-text pairs. Finally, for each resource r_i , the estimated variation is stored as

$$c_t(r_i) = \{ \langle \text{URL}_1, c_t(r_i, 1) \rangle, \langle \text{URL}_2, c_t(r_i, 2) \rangle, \dots, \langle \text{URL}_x, c_t(r_i, x) \rangle \}. \quad (5.1)$$

Here, $c_t(r_i, j) (1 \leq j \leq x)$ denotes the increase in encrypted size of resource r_i due to an embedded tracking cookie carrying the string URL_j .

(2) Characterizing session cookies. Session cookies are transmitted when a page is accessed for the first time in a session. During profiling, the resources will always have the session cookie. The parameter $c_s(r_i)$ accounts for the discrepancy in the encrypted size of resource r_i due to the possible absence of session cookie in the real user's trace. This happens when the resource is not the first resource to be accessed by the real user. The value of $c_s(r_i)$ can be estimated by considering all the information carried in the header field (e.g., user ID, user agent, max-age of the cookie, its expiry date, etc.) and their possible values. Most of these information are of fixed length, except for the user agent field that contains the browser and OS name. Much like tracking cookies, the `COMPUTE_COOKIE_VAR` stores this information for each resource r_i as

$$c_s(r_i) = \{ \langle bo_1, c_s(r_i, 1) \rangle, \langle bo_2, c_s(r_i, 2) \rangle, \dots, \langle bo_x, c_s(r_i, x) \rangle \}. \quad (5.2)$$

Here $c_s(r_i, j) (1 \leq j \leq x)$ denotes the increase in encrypted size of resource r_i due to the session cookie carrying browser-OS identifier string bo_j .

The cookie induced variations thus estimated, are stored in a lookup-table *cookie_var*, where each entry is of the form $\langle r_i, \{c_t(r_i), c_s(r_i)\} \rangle$. For resources that do not contain tracking cookies, $c_t(r_i)$ contains NULL value.

Notation	Meaning
G	Graph representation of the target website
RM	Webpage to resource map of the website
F	Feature used for webpage fingerprinting
$\mathcal{S}$	Set of resource download sequences of all webpages of the target website
<i>reverseFeatureDB</i>	mapping between encrypted resource sizes and possible resource names
$c_t(r_i, j)$	Increase in size of resource r_i due to tracking cookie when the user navigates from URL j
$c_s(r_i, k)$	Increase in size of resource r_i due to session cookie carrying the Browser/OS identifier bo_k
$max(c_t)$	Maximum possible increase in size of any resource in the website due to tracking cookies
$max(c_s)$	Maximum possible increase in size of any resource in the website due to session cookies

Table 5.2: Information fetched from the Snoopy Database for resource prediction

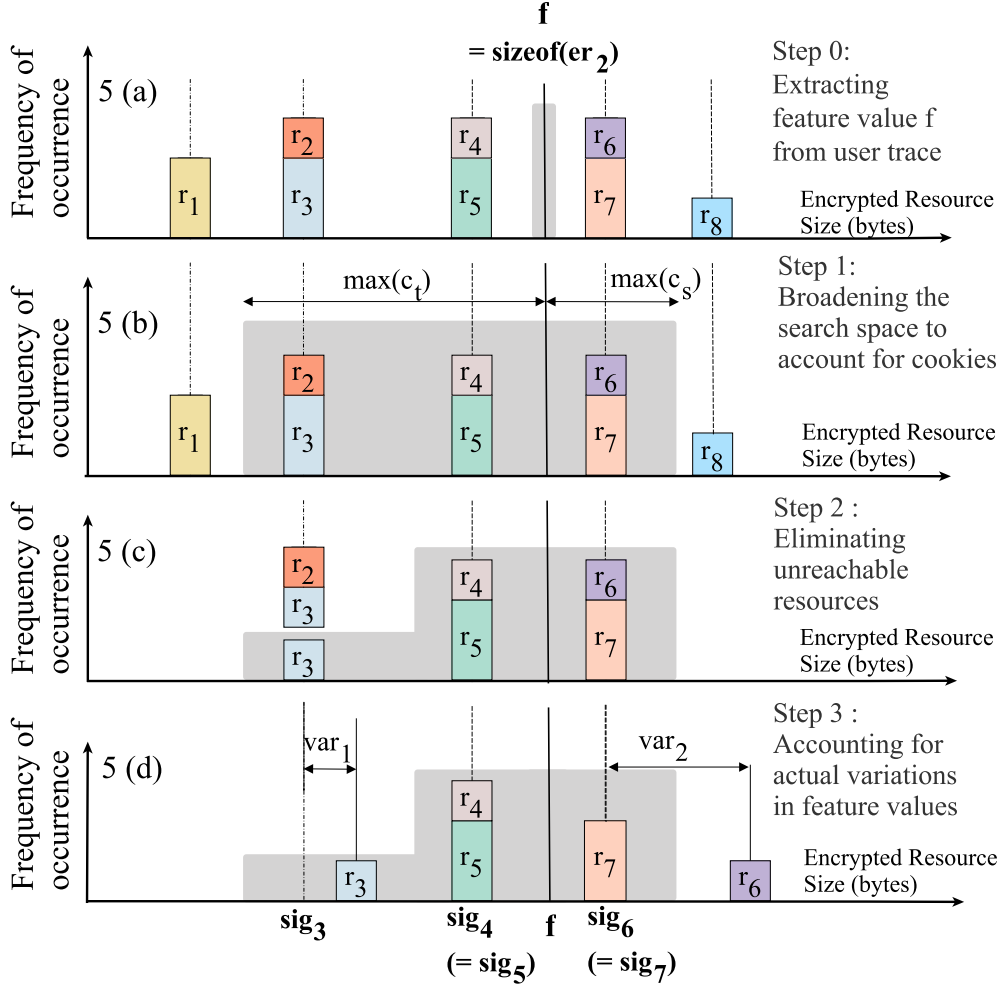
5.5.2 Webpage Prediction Module

The *Webpage Prediction Module* of Snoopy takes an encrypted traffic trace T as input, as shown in Step 3 of Figure 5.2, and predicts the webpages that are accessed in T using the information stored in the Snoopy database (refer to Table 5.2). Snoopy first predicts the web resources accessed and subsequently uses them for predicting the webpages accessed in T .

5.5.2.1 Predicting Web Resource Sequence

For predicting the resources in the input trace T , Snoopy first splits T into sub-traces, each corresponding to an encrypted resource. From each sub-trace, the feature values are extracted and stored, in the order they appear in T , into an array *FValues*, where *FValues*[i] contains the feature value of an encrypted resource er_i . Recall that the feature used is the encrypted resource size.

As a next step, Snoopy identifies the actual web resource that may correspond to each feature value in *FValues*, serially. It does so by looking-up the dictionary *reverseFeatureDB*. Apart from the fact that multiple resources may have the same



$$\text{var}_1 = (\text{sig}_3 - c_t(r_3, 1) + c_s(r_3, 2)) \quad \text{var}_2 = (\text{sig}_6 - c_t(r_6, 2) + c_s(r_6, 2))$$

Figure 5.5: Steps involved in selecting resources that might correspond to a given feature value

feature value, the look-up procedure is also not straightforward due to variations in feature values induced by tracking and session cookies.

To start with, we organize the information available (for the website shown in Figure 5.3) in the *reverseFeatureDB* (refer to Figure 5.4) as a stacked bar plot shown in Figure 5.5(a). The X-axis represents encrypted resource size and the Y-axis represents the frequency of occurrence of the resources with a particular size in the target website. For e.g., in Figure 5.5(a), r_7 and r_6 are stacked together and have the same size but r_7 (embedded

in two webpages) is twice as likely to occur as compared to r_6 (embedded in only one webpage), as indicated by the height of the stacked bar. This is because, r_7 is embedded in two webpages, whereas r_6 is embedded in only one webpage. We begin the look-up process by considering a shaded region (around $x = f$ as shown in Figure 5.5(a)) that represents the area in which the actual resource accessed by the user in trace T could be present. Subsequently, we refine our search by expanding and narrowing down the shaded region in each step.

We now illustrate the look-up with a concrete example.

Example: Consider two encrypted resources er_1 and er_2 extracted from T such that er_1 is accessed by the user before er_2 . The feature values extracted from these two resources are stored in $FValues[1]$ and $FValues[2]$ respectively and processed sequentially. We assume without loss of generality that Snoopy has predicted that the resource, say, r_4 , corresponds to er_1 , after processing $FValues[1]$. We now describe how Snoopy processes the feature value of the second resource er_2 , stored in $FValues[2]$. Let us assume $FValues[2] = f$.

Step 1: Broadening the search space – The goal of this step is to broaden the space in which we search for the resource corresponding to f (shown in Figure 5.5(a)), since tracking and session cookies may have changed the resource sizes. If f is inclusive of the tracking cookies, it needs to be decremented (by $c_t(r_i, j)$) before performing a *reverseFeatureDB* look-up. Likewise, if f does not contain session cookies, it needs to be incremented (by $c_s(r_i, k)$) before performing a look-up. However, the presence or absence of these cookies cannot be determined by Snoopy at this stage, since it does not yet have the knowledge of the webpage from which the user navigated to this encrypted resource. That is, both $c_t(r_i, j)$ and $c_s(r_i, k)$ cannot be determined since i (i.e., resource ID), j (i.e., previously accessed URL), and k (i.e., user agent string) are not known. To account for these unknown parameters, we use the maximum possible values for $c_t(r_i, j)$ and $c_s(r_i, k)$, retrieved from *cookie_var*. Therefore, Snoopy performs a look-up of all

values in the range $[(f - \max(c_t)), (f + \max(c_s))]$ and adds all the retrieved resources to the *multiset relevant_resources*. Multiple instances of a resource are added to *relevant_resources* if the resource is associated with multiple webpages.

Step 1 of Figure 5.5 shows that although f is closest to the sizes of r_4 , r_5 , r_6 , and r_7 , it could also possibly belong to r_2 or r_3 due to the aforementioned variations. These resources form the multiset $relevant_resources = \{r_2, r_3, r_3, r_4, r_5, r_5, r_6, r_7, r_7\}$, and is indicated by the shaded region in Step 1 of Figure 5.5.

Step 2: Reachability checking – In the real-world, a user starts browsing from a particular webpage and navigates using links embedded in the initial and subsequent webpages. Leveraging this, Snoopy eliminates the resources (in *relevant_resources* from previous step) that are not navigable from previously accessed webpages. For checking reachability, Snoopy leverages the website structure $\mathbb{G}$ and the resource map $\mathbb{RM}$. While performing reachability checking, unlike prior works (Chapman and Evans, 2011; Miller *et al.*, 2014), Snoopy also considers multi-tab browsing. Snoopy eliminates those resources that are not accessible from *any* of the previously accessed webpages from any of the open tabs.

In the example, $er_1 (= r_4)$ is the previously accessed resource, which is uniquely associated to the webpage w_3 , as seen from the website structure in Figure 5.3. Through reachability check, we can infer that the resource er_2 can come from either of w_3 , w_4 , or w_5 and not from w_1 or w_2 . If any resource in the *relevant_resources* set that are associated with w_1 (we do not have r_1 in this example) or w_2 (i.e., r_2 and r_3), its frequency should be reduced accordingly, i.e., the shaded region shrinks in size. Therefore, the set of reachable resources now contains $\{r_3, r_4, r_5, r_5, r_6, r_7, r_7\}$, denoted by *reachable_resources*, indicated by the shaded region in Step 2 of Figure 5.5, which is scaled-down as compared to that in Step 1.

Step 3: Accounting for cookie-induced variations in reachable resources – In this

step, the goal is to compute $(sig_i - c_t(r_i, x) + c_s(r_i, y))$ to estimate the actual encrypted feature value for each resource r_i in the aforementioned step, based on information from *reverseFeatureDB* and *cookie_var*. Based on this, we re-plot Figure 5.5(d) to shift all the reachable resources. Note that the shaded region does not change in this step. Snoopy estimates (1) $c_t(r_i, x)$ – which is the length of the URL contained in its tracking cookie. When the resource is associated with multiple URLs, some can be eliminated through reachability checking; and, (2) $c_s(r_i, y)$ – the browser and OS information contained in the session cookie in the Application layer header can be used. This information (i.e., bo_y from § 5.5.1), can be estimated by using existing browser fingerprinting techniques (Husák *et al.*, 2016) and OS fingerprinting techniques (Laštovička *et al.*, 2020).

This step further narrows down the search space. This process is illustrated in Step 3 of Figure 5.5, which shows the case where the resource r_6 moves out of the shaded region whereas r_4 , r_5 and r_7 remain unchanged since they have no cookie induced variation. However, note that r_3 is still under consideration, since it lies within the broadened search space even when its cookies are accounted for.

Step 4: Assigning Weightage to Resources – In this step, Snoopy identifies the resource that is most likely to correspond to $er_2 = f$. For this, Snoopy assigns a weightage to each resource in *reachable_resources* from the previous step. We measure the proximity of each of these resources to f , given that their cookie induced variations are accounted. We do so by taking a weighted average of their distance from f . Finally, Snoopy considers the resource with the highest weightage to be the one corresponding to er_2 . In this example, er_2 is predicted as r_7 . The output of this step is the sequence of resources in T as predicted by Snoopy, denoted as *predicted_resources*.

5.5.2.2 Predicting Webpages Accessed by the User

Snoopy uses the longest common sub-sequence matching algorithm for predicting accessed webpages from *predicted_resources*. Snoopy starts with the first resource (say, r_{p1}), and from the sequence set $\mathbb{S}$, it identifies all the webpages where r_{p1} is the first

resource in the resource download sequence. The same process is followed recursively for the subsequent resources in *predicted_resources* until the webpage with the longest sub-sequence match is found. When Snoopy identifies the webpage, it adds the webpage to a set denoted as *predicted_webpages*. Note that *predicted_resources* may not be empty yet since it could have the resources corresponding to multiple webpages. In such a case, the process is repeated.

Webpage identification becomes more challenging when caching is enabled in the browser. When caching is enabled, all the resources in the sequences in $\mathbb{S}$ for a webpage would not get downloaded, and thereby forming a gap in the sequence *predicted_resources*. To handle such cases, during matching, we allow the algorithm to skip resources in $\mathbb{S}$ of a webpage as long as it is a cache-able resource and it has been downloaded in the past.

5.6 IMPLEMENTATION AND EVALUATION

We first describe our implementation and define the metrics used for evaluating Snoopy. Thereafter, we describe the experimental setup, which includes the websites used for evaluation, the user browsing scenarios considered for evaluation, and the traffic sample collection methodology. Subsequently, we evaluate Snoopy by answering a set of research questions.

5.6.1 Implementation

We implemented the functional modules of Snoopy as Python libraries. We also compared Snoopy with relevant state-of-the-art works on webpage fingerprinting such as ML_PS (Chapman and Evans, 2011), ML_IPS (Chapman and Evans, 2011), ML_OPS (Chapman and Evans, 2011), ML_BoG (Miller *et al.*, 2014), ML_LL (Liberatore and Levine, 2006), ML_KFP (Hayes and Danezis, 2016), ML_Wfin (Yan and Kaur, 2018), and ML_CUMUL (Panchenko *et al.*, 2016). For this, we implemented these techniques to the best of our abilities with the help of open-source Python libraries (Alan, 2019). While most of these ML-based techniques (Chapman and

Evans, 2011; Miller *et al.*, 2014; Liberatore and Levine, 2006) were solely designed for the purpose of webpage fingerprinting over HTTPS, few others were not designed with the exact same goal. Techniques such as k-FP (Hayes and Danezis, 2016) and Wfin (Yan and Kaur, 2018) were originally used for both website fingerprinting (over Tor) and webpage fingerprinting (over HTTPS). For our purpose, we only consider the following scenario that was presented in the k-FP (Hayes and Danezis, 2016) and Wfin (Yan and Kaur, 2018) papers: *webpage* fingerprinting over HTTPS. For instance, in this scenario, k-FP uses features such as packet size, packet direction, and packet ordering. We used the same features and classification algorithms in our implementation of k-FP. Another technique, namely CUMUL (Panchenko *et al.*, 2016), attempted to perform both website and webpage fingerprinting over Tor (not HTTPS). For our purpose, we were only interested in the *webpage fingerprinting* part of the paper. The paper proposed the use of (i) a novel ML-based technique as well as (ii) the use of a novel set of features (Tor and TLS features). The paper concedes that CUMUL was unsuccessful in performing webpage fingerprinting over Tor. This was mainly attributed to Tor’s obfuscation of resource sizes, which rendered the TLS features useless. We use CUMUL as benchmark in our experiments but on *HTTPS traffic* (i.e., a much weaker defense than Tor). We expected CUMUL to perform better against this weaker defense due to the absence of Tor’s obfuscation of resource sizes. In our implementation, we used the same ML-based technique and the TLS features proposed in CUMUL but used them on HTTPS dataset.

Encrypted network traffic traces used for webpage fingerprinting and testing was captured using TShark(v2.6.6) packet capture tool, which was running on the network gateway, although the same could be done by the attacker at any intermediate network router in the real world.

5.6.2 Metrics

We define the metrics that we use for evaluating Snoopy and existing state-of-the-art webpage identification techniques as follows:

Sl No	Website	HTTP version	TLS version	No of pages	Type of webpages
1	RS	1.1	1.2	20	plain HTML
2	SBC	2.0	1.3	27	HTML scripts
3	IC_1	1.1	1.2	95	aspx
4	B_1	1.1	1.2	444	aspx
5	B_2	1.1	1.2	458	HTML scripts
6	B_3	1.1	1.2	549	aspx
7	B_4	1.1	1.2	965	HTML + JS
8	IC_2	1.1	1.2	849	HTML + JS
9	B_5	1.1	1.2	1964	plain HTML
10	PS	2.0	1.2	40,323	HTML + JS

Table 5.3: Characteristics of Profiled Websites

RS - Retail Store website, SBC - Service Based Company website, IC - Insurance Company website, B - Bank website, PS - Political Survey website

Generalization factors (GF): The set of factors related to user's browsing behavior that the webpage identification technique aims to generalize across. $\mathbf{GF} \subseteq \{I, BC, T, O, B, N\}$, where I denotes user interest and thereby the set of webpages in the website that the attacker needs to fingerprint, BC denotes the set of browser configurations (cache and cookie settings) considered by the attacker, T denotes the number of tabs used by a user in a browsing session, O indicates the set of Operating Systems that the attacker generalizes across, B denotes the set of Browsers, and N denotes network conditions.

Number of Queries (N_q): The maximum number of website queries (accesses) that an attacker can perform on the targeted website for collecting traffic samples for training (webpage profiling).

Fingerprinting Accuracy (FA): The percentage of webpages accessed by a user (of the target website) in a browsing session that are correctly identified by a webpage identification technique.

5.6.3 Experimental setup

Websites Used. We evaluate Snoopy and related works (Liberatore and Levine, 2006; Chapman and Evans, 2011; Miller *et al.*, 2014; Hayes and Danezis, 2016; Panchenko *et al.*, 2016; Yan and Kaur, 2018) on 10 websites that include some of the Fortune 100 companies, financial organizations, and service-based companies from different parts of the world. These websites have been listed in Table 5.3. The scale of our experiments is at par with existing works on webpage identification (Cheng and Avnur, 1998; Sun *et al.*, 2002; Danezis, 2010; Miller *et al.*, 2014; Hayes and Danezis, 2016; Zhuo *et al.*, 2017; Xu *et al.*, 2018; Shen *et al.*, 2019; Alan and Kaur, 2019). As mentioned earlier, this chapter deals with *webpage identification* performed *on a website* of interest to the adversary, in contrast to website identification that concerns thousands of websites. The names of these websites are anonymized to protect the websites from being targets of this attack.

User Browsing Scenarios. We consider different combinations of (1) Operating Systems viz.; Ubuntu 18.04, and Microsoft Windows 7, (2) Browsers viz.; Mozilla Firefox 63.0.3, and Google Chrome 67.0, (3) cache settings viz.; ON and OFF, (4) cookie preferences viz.; Allowed and Prohibited, and (5) network conditions. Users can start browsing from any webpage in the website that they would be interested in, and browse the pages in any order. Further, we also allow users to freely browse up to 15 different webpages in each browsing session, either sequentially or by using multiple parallel tabs, with no restriction on the transition time from one webpage to another. Most existing works do not consider cases where the users browse more than one page in a session and even if they do, they restrict it to a small number (for e.g., 4 in (Xu *et al.*, 2018) and 2 in (Gu *et al.*, 2015)). These numbers are much lower than the average number of parallel browser tabs used by website visitors, as pointed out in a study¹² conducted by Mozilla (Singer-Vine, 2010).

Traffic Sample Collection. We developed a Python bot for collecting encrypted traffic

¹²Mozilla has removed the dataset compiled for the Test Pilot study conducted in 2010 from public domains. But we can speculate that the trend of tabbed webpage browsing has only gone up in the last decade.

samples for evaluating Snoopy as well as the existing works. Our traffic collection bot used Selenium for simulating behavior of real website users while collecting test traffic samples. For creating webpage fingerprints, our bot accessed the webpages as many times as required by the different webpage fingerprinting techniques, within the limits of the finite query model. Depending on the requirements of our experiments, the bot performs either sequential or single-page traffic sample collection. We introduced a delay of 1 minute between subsequent browsing sessions. We observed that repeated website accesses from the same network to the same website needed to be separated by this time in order for it to not be flagged and blocked by the website. For evaluations that required sequential browsing traffic samples, the browsing sequence length was varied from 3 to 15 webpages per session. For evaluations that required traffic samples collected over various network conditions, the traffic sample collection was conducted over several months and from different geographical locations to ensure variations in network conditions. In addition to the webpage(s) accessed in a browsing session (which is needed for computing the accuracy), our bot also recorded the sequence of web objects downloaded in each browsing session, which we use for additional evaluation.

5.6.4 Results

In this section, we first evaluate the suitability of Snoopy in the context of practical mass-surveillance. In the prior sections, we have discussed about the two key requirements for conducting practical mass-surveillance – generalization and compliance with a finite query model. Therefore, we primarily evaluate Snoopy on these two parameters, through experiments that intend to answer the following questions.

- (Q1)** How effective is Snoopy in different browsing contexts? (Refer to § 5.6.4.1)
- (Q2)** How is the effectiveness of Snoopy and related works affected with changes in the value of N_q (number of queries allowed)? (Refer to § 5.6.4.2)
- (Q3)** How well does Snoopy generalize across user interests? (Refer to § 5.6.4.3)
- (Q4)** How well does Snoopy generalize across different configurations of a browser? (Refer to § 5.6.4.4)

- (Q5) How well does Snoopy generalize in terms of number of tabs used by the end user while browsing? (Refer to § 5.6.4.5)
- (Q6) How well does Snoopy generalize in terms of variations in Operating Systems? (Refer to § 5.6.4.6)
- (Q7) How well does Snoopy generalize in terms of variations in Browsers? (Refer to § 5.6.4.7)
- (Q8) How well does Snoopy generalize across various network conditions? (Refer to § 5.6.4.8)

5.6.4.1 Average prediction accuracy of Snoopy across diverse browsing scenarios

We now show the prediction accuracy (FA) of Snoopy when it builds webpage fingerprints (trains) for any one browsing scenario and uses this information to predict user activities across diverse browsing scenarios. For this experiment, we considered all the 10 websites. For each website, we built a training dataset where we consider $GF = \{I, BC, T, O, B, N\}$ such that, $I = \{\text{all webpages in the website}\}$, $BC = \{\{\text{Caching ON, Cookies Allowed}\}, \{\text{Caching OFF, Cookies Prohibited}\}\}$, $T = 1$, $O = \{Ubuntu\}$, $B = \{Firefox\}$. N was kept constant by collecting all traffic samples from the same system within a short span of time. We collect 10 such traffic samples for each webpage.

For each website, we also build a test dataset, where we consider $I = \{\text{all webpages in the website}\}$, vary BC across the values $\{\{\text{Caching ON, Cookies Allowed}\}, \{\text{Caching ON, Cookies Prohibited}\}, \{\text{Caching OFF, Cookies Allowed}\}, \{\text{Caching OFF, Cookies Prohibited}\}\}$, vary T from 1 to 15, vary O across the values $\{Windows, Ubuntu\}$, and vary B across the values $\{Chrome, Firefox\}$. The network conditions N were varied by collecting traffic samples over several months from different geographical locations (refer to the test dataset described in § 5.6.3).

Table 5.4 shows the webpage prediction accuracy (FA) of Snoopy on the 10 websites. The FA value is more than 90% for most of the websites, indicating the ability of Snoopy to generalize across different browsing contexts. In the rest of this section, we show how the different factors in GF individually influence the FA value of Snoopy, when subjected

Website	No. of webpages	Webpage identification accuracy		
		Accurately identified (%)	Not identified (%)	Wrongly identified (%)
IC_1	95	93	7	0
IC_2	849	89	11	0
B_1	444	99	1	0
B_2	458	99	1	0
B_3	549	97	0	3
B_4	965	88	11	1
B_5	1964	90	3	7
SBC	27	81	7	12
RS	20	75	21	4
PS	40323	83	17	0

Table 5.4: Webpage prediction accuracy of Snoopy

to constraints on the value of N_q , the maximum number of queries allowed to a website.

5.6.4.2 Compliance with a finite query model: Snoopy vs. ML-based solutions

We now evaluate Snoopy on its ability to comply with a finite query model with respect to existing works. For this experiment, we considered the website B_4 and generalization factors $GF = \{I, BC, T, O, B, N\}$, where I is a set of 200 random webpages of the website B_4, $BC = \{\{\text{Caching OFF, Cookies Allowed}\}\}$, $T = 1$, $O = \{Ubuntu\}$, $B = \{Firefox\}$. N was constant since the traffic samples were collected within a short span of time. Our test dataset comprised traffic traces from the website B_4 collected using the above configuration. We built three different training datasets with traffic samples collected using the same configuration, but varying the number of samples collected per webpage. We set N_q , the maximum number of queries allowed, as $w \times s$, where w is the number of webpages to be fingerprinted and s is the number of traffic samples collected per webpage. Keeping $w = 200$ constant, we vary the value of s as $s = 10, 5$, and 3 .

Figure 5.6 plots the fingerprinting accuracy (FA) for Snoopy and existing works for different values of s . From the figure, we can observe that (1) When $s = 10$, the prediction accuracy of Snoopy (77.75%) is at par with the existing ML-based techniques, and; (2)

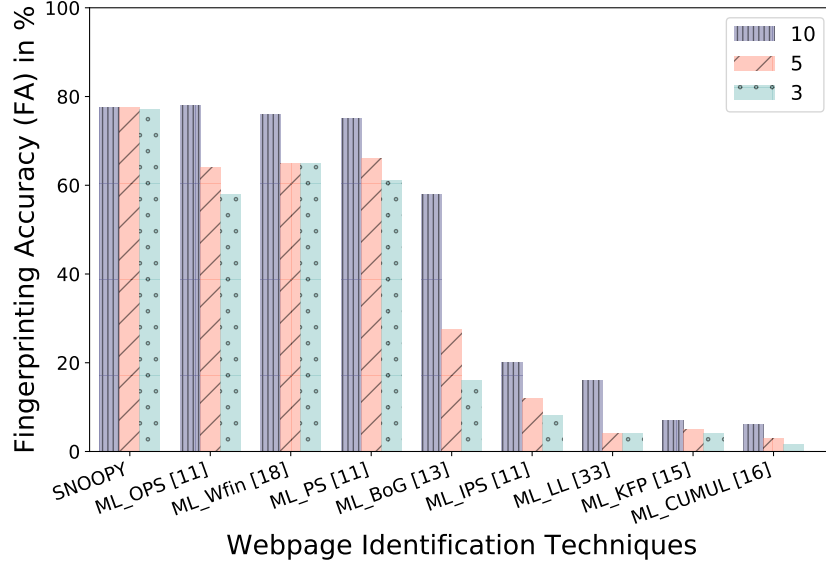


Figure 5.6: Importance of training set size N_q on fingerprinting accuracy FA (in %) of existing ML models vs. Snoopy (for 200 web pages)

When $s = 5$ or $s = 3$, we observed a drop in FA of ML techniques (for e.g., from 78% to 58% in case of ML_OPS (Chapman and Evans, 2011)). On the other hand, we can see that FA for Snoopy remains unchanged even after lowering the value of s . Since we have seen that for the three different s values considered in this experiment, the ML models have the highest FA value when $s = 10$, we use $s = 10$ to constrain the value of N_q for the finite query model assumed in the rest of our experiments (shown in § 5.6.4.3–§ 5.6.4.8).

5.6.4.3 Generalization across User Interests

We now show how the fingerprinting accuracy (FA) of Snoopy and existing works change with the number of webpages of interest (I). For this experiment, we consider the website B_4 and generalization factor $GF = \{I, BC, T, O, B, N\}$, where we set $BC = \{\{\text{Caching OFF, Cookies Allowed}\}\}$, $T = 1$, $O = \{Ubuntu\}$, $B = \{Firefox\}$. N was constant since the traffic samples were collected within a short span of time. We varied I as sets of 50, 100, and 200 webpages for building three different training datasets. For this experiment, $N_q = |I| \times 10$, which is the constraint imposed by our finite query model. For each of the training datasets, we built test datasets using encrypted traffic traces with

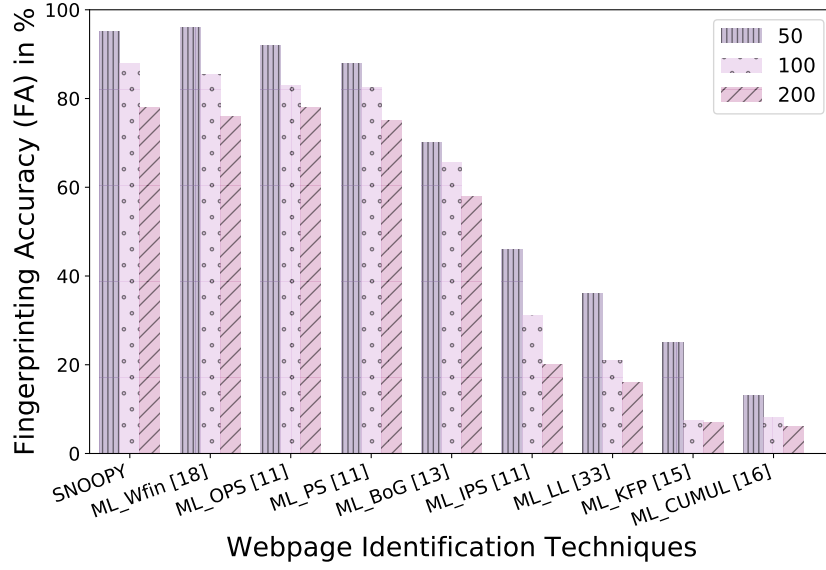


Figure 5.7: Fingerprinting accuracy FA (in %) of Snoopy vs. existing techniques for different number of webpages of interest

the same constraints as the training dataset.

Figure 5.7 shows the fingerprinting accuracy (FA) of Snoopy and existing works when the number of webpages of interest increases. From the figure we can observe that **(1)** Despite using a simple feature and a simple approach, Snoopy gives the same prediction accuracy as some of the best-performing ML-based techniques (e.g., ML_Wfin (Yan and Kaur, 2018)), and; **(2)** When $|I|$ was increased from 50 to 100 and 200, there was a fall in the value of FA for Snoopy as well as the existing ML-based techniques. Even then, Snoopy gave the same prediction accuracy as the best-performing ML-based techniques (for e.g., ML_Wfin (Yan and Kaur, 2018) and ML_OPS (Chapman and Evans, 2011)).

A detailed inspection revealed that the drop in the FA value of Snoopy when the size of I was gradually increased was due to the fact that many of the webpages in the larger sets had similar fingerprints. In § 5.6.4.10, we will discuss the reason behind this in more details, and in § 5.6.5 we will discuss possible ways to improve the fingerprinting accuracy (FA) of Snoopy.

Classifier	Browser Cache Configuration	
	Training: OFF Testing: ON	Training: OFF Testing: OFF
SNOOPY	72	78
ML_PS (Chapman and Evans, 2011)	61	75
ML_OPS (Chapman and Evans, 2011)	60	78
ML_Wfin (Yan and Kaur, 2018)	51	76
ML_BoG (Miller <i>et al.</i> , 2014)	50	58
ML_LL (Liberatore and Levine, 2006)	14	16
ML_KFP (Hayes and Danezis, 2016)	4.5	7
ML_IPS (Chapman and Evans, 2011)	3	20
ML_CUMUL (Panchenko <i>et al.</i> , 2016)	2.5	6

Table 5.5: Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different browser configuration (for 200 webpages).

5.6.4.4 Generalization across Browser Configurations

Next, we evaluate Snoopy and existing works on their ability to generalize across different configurations of caching and cookie settings for a given browser (BC). For this experiment, we consider the website B_4 and $GF = \{I, BC, T, O, B, N\}$, such that I is a set of 200 random webpages of the website B_4, $T = 1$, $O = \{Ubuntu\}$, $B = \{Firefox\}$, and N was constant. While building our training dataset, we keep $BC = \{\text{Caching OFF, Cookies Allowed}\}$ constant, and collect 10 samples of browsing traffic from each of the 200 webpages of B_4 that we considered. For this experiment, $N_q = 200 \times 10$, which is the constraint imposed by the finite query model. In our test dataset, we vary BC across the values $\{\{\text{Caching OFF, Cookies Allowed}\}, \{\text{Caching ON, Cookies Allowed}\}\}$, with all other factors similar to the training dataset.

Table 5.5 shows a comparison of the fingerprinting accuracy (FA) of Snoopy and related works. For the straightforward case, where training and testing used the same BC configuration, Snoopy achieves an FA that is comparable ($FA \approx 75\% - 78\%$) to the best performing ML models as expected. However, when the BC configurations for testing and training were different, Snoopy outperforms even the best ML techniques. For example, Snoopy achieves an $FA = 72\%$ as compared to ML_PS (Chapman and

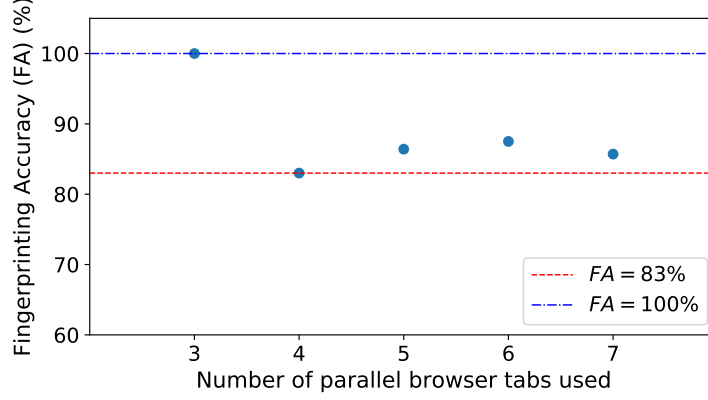


Figure 5.8: Fingerprinting accuracy (FA) of Snoopy in multi-tab browsing scenario (for website B_4)

Evans, 2011) that achieves $FA = 61\%$.

5.6.4.5 Support for prediction on multi-tab browsing traffic

We now evaluate Snoopy in the context of a real-world scenario where the users of the targeted website open the webpages simultaneously in multiple parallel browser tabs (T). For this experiment, we considered the website B_4 and $GF = \{I, BC, T, O, B, N\}$, such that I is a set of 200 random webpages of the website B_4, $BC = \{\text{Caching OFF, Cookies Allowed}\}$, $O = \{Ubuntu\}$, $B = \{Firefox\}$, and N was constant. While building our training dataset, we keep $T = 1$ constant, and collect 10 samples of browsing traffic from each of the 200 webpages of B_4 that we considered. For this experiment, $N_q = 200 \times 10$, which is the constraint imposed by the finite query model. In our test dataset, we vary T from 3 to 7 with all other factors similar to the training dataset.

Figure 5.8 shows average the fingerprinting accuracy (FA) of Snoopy for browsing sessions for the website B_4, involving different number of parallel browser tabs. We observe that even for 7 parallel browser tabs, the average fingerprinting accuracy of Snoopy is more than 85%.

The traffic samples collected for building the training dataset for Snoopy in this case correspond to single-tab single-webpage browsing sessions. This is in contrast to

Website	Number of unique webpage sequences (Sequence Length: 3-7) (l)	Minimum number of website accesses required by Snoopy (w)
IC_1	586	95
IC_2	170,538	849
B_1	9,291,522	444
B_2	1199	458
B_3	13,617	549
B_4	7,000	965
B_5	103,116	1964
SBC	3,271	27
RS	1,159	20
PS	11	11

Table 5.6: Snoopy vs. Existing works: Number of website accesses required for fingerprinting in multi-tab browsing scenario

existing works (Chapman and Evans, 2011; Danezis, 2010; Miller *et al.*, 2014; Xu *et al.*, 2018) that fingerprint sequences of webpage browsing traffic instead of single webpage accesses. While existing techniques have been shown to result in a high fingerprinting accuracy (FA), it magnifies the number of website accesses required (N_q) to a great extent. Table 5.6 shows a comparison of the number of webpage sequences of length 3 – 7 for the 10 websites we selected for our experiments, with the number of individual webpages that Snoopy would access for building fingerprints. Existing works would require $N_q = l \times$ number of traffic samples for training their models, where l is the number of webpage sequences possible. On the other hand, Snoopy would require only $N_q = w \times$ number of traffic samples, where $w (\ll l)$ is the number of webpages in the website. Note that the number of webpage sequences shown in Table 5.6 do not consider multiple occurrences of the same webpage in a sequence. Considering such cases would have further increased N_q for existing works. This might have caused the website to block the adversary from collecting traffic samples, and would have definitely escalated the time required for collecting traffic samples, and training the model. On the other hand, this would not impact the N_q value for Snoopy. Latest works (Gu *et al.*, 2015) that are closest to Snoopy in terms of working principle have not been able to obtain a considerable FA

Classifier	Operating System	
	Training: Windows Testing: Ubuntu	Training: Ubuntu Testing: Ubuntu
SNOOPY	97.8	97.9
ML_Wfin (Yan and Kaur, 2018)	54.8	93.7
ML_CUMUL (Panchenko <i>et al.</i> , 2016)	21.5	78
ML_KFP (Hayes and Danezis, 2016)	15.0	65.2
ML_BoG (Miller <i>et al.</i> , 2014)	2.1	77.9
ML_PS (Chapman and Evans, 2011)	1.1	77.9
ML_IPS (Chapman and Evans, 2011)	1.1	62.1
ML_OPS (Chapman and Evans, 2011)	1.1	55.8
ML_LL (Liberatore and Levine, 2006)	1	40.0

Table 5.7: Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different operating system (for 95 webpages).

value beyond 2 parallel tabs.

5.6.4.6 Generalization across various Operating Systems

We now evaluate Snoopy and existing works on their ability to generalize across different operating systems (O). In this experiment we consider the website IC_1 and $GF = \{I, BC, T, O, B, N\}$, such that I is the set of 95 webpages of the website IC_1, $BC = \{\{\text{Caching OFF, Cookies Allowed}\}\}$, $T = 1$, $B = \{\text{Firefox}\}$, and N was constant. We built two different training datasets, one where $O = \{\text{Ubuntu}\}$ was kept constant, and another where $O = \{\text{Windows}\}$ was kept constant. For building each training dataset, we collected 10 samples of browsing traffic from each of the webpages in I. For this experiment, $N_q = 95 \times 10$, which is the constraint imposed by the finite query model. In our test dataset, we considered $O = \{\text{Ubuntu}\}$, with all other factors similar to the training datasets.

Table 5.7 shows a comparison of the fingerprinting accuracy (FA) of Snoopy and related works. When training and testing were performed using traffic samples from the same OS, Snoopy achieves an FA that is comparable ($FA \approx 94\% - 98\%$) to the best performing ML model ML_Wfin (Yan and Kaur, 2018), as expected. However, when the OS used for testing and training were different, Snoopy outperforms even the best ML techniques

Classifier	Browser	
	Training: Chrome Testing: Firefox	Training: Firefox Testing: Firefox
SNOOPY	85.9	97.9
ML_Wfin (Yan and Kaur, 2018)	13.8	93.7
ML_PS (Chapman and Evans, 2011)	8.0	77.9
ML_OPS (Chapman and Evans, 2011)	8.0	55.8
ML_IPS (Chapman and Evans, 2011)	5.7	62.1
ML_LL (Liberatore and Levine, 2006)	5.7	40.0
ML_CUMUL (Panchenko <i>et al.</i> , 2016)	3.4	78
ML_BoG (Miller <i>et al.</i> , 2014)	2.3	77.9
ML_KFP (Hayes and Danezis, 2016)	2.3	65.2

Table 5.8: Accuracy (in %) of Snoopy vs. ML models when tested on data points from a different browser (for 95 webpages).

by a huge margin. For example, Snoopy achieves an FA = 97.8% as compared to ML_Wfin (Yan and Kaur, 2018) that achieves FA = 54.8%.

5.6.4.7 Generalization across various Browsers

We now evaluate Snoopy and existing works on their ability to generalize across different browsers (B). In this experiment we consider the website IC_1 and GF = {I, BC, T, O, B, N}, such that I is the set of 95 webpages of the website IC_1, BC = {{Caching OFF, Cookies Allowed}}, T = 1, O = {Ubuntu}, and N was constant. We built two different training datasets, one where B = {Google Chrome} was kept constant, and another where B = {Firefox} was kept constant. For building each training dataset, we collected 10 samples of browsing traffic from each of the webpages in I. For this experiment, $N_q = 95 \times 10$, which is the constraint imposed by the finite query model. In our test dataset, we considered B = {Firefox}, with all other factors similar to the training datasets.

Table 5.8 shows a comparison of the fingerprinting accuracy (FA) of Snoopy and related works. When training and testing were performed using traffic samples from the same browser, Snoopy achieves an FA that is comparable (FA $\approx$ 94% – 98%) to the best

performing ML model ML_Wfin (Yan and Kaur, 2018), as expected. However, when the OS used for testing and training were different, Snoopy outperforms even the best ML techniques by a huge margin. For example, Snoopy achieves an FA = 85.9% as compared to ML_Wfin (Yan and Kaur, 2018) that achieves FA = 13.8%.

5.6.4.8 Generalization across network conditions

We did not observe any changes in the fingerprinting accuracy of Snoopy due to small-scale natural network fluctuations. The results shown in Table 5.4 reflects the prediction accuracy of Snoopy across diverse network conditions. The traffic samples that were used for profiling were collected over a steady network connection within a short span of time. On the contrary, the test traffic traces were collected over a month from different geographical regions to ensure sufficient diversity in network conditions.

To further test the limits of Snoopy, we introduced artificial perturbations in the compromised network device over an hour by randomly adding delays of 50ms to 80ms and throttling the bandwidth by 20%. In this case, we encountered significant packet drops. This resulted in incomplete download of web resources on several instances, and consequently Snoopy could not identify the webpages correctly. The percentage of cases where the webpages could not be identified by Snoopy has been indicated in Table 5.4.

5.6.4.9 Resource-level prediction accuracy of Snoopy

We now present more detailed evaluation results about the performance of Snoopy. This includes individual resource-level prediction accuracy of Snoopy and the correlation between resource-level prediction accuracy and webpage-level prediction accuracy. We also analyze the cases where Snoopy fails to accurately identify the webpages accessed, and propose potential solutions to effectively handle such scenarios.

Table 5.9 shows the resource prediction accuracy of Snoopy on the ten websites. While for most of the websites Snoopy had a prediction accuracy of more than 70%, in a few cases, we witnessed an accuracy of less than 50%. For websites with a very low number

Website	No of resources	Resource identification accuracy		
		Accurately identified (%)	Unidentified resources Incomplete download (%)	Conflict (%)
IC_1	187	94	5	1
IC_2	19,163	71	19	10
B_1	865	69	7	24
B_2	12,435	47	52	1
B_3	688	91	5	4
B_4	3472	83	1	16
B_5	3,998	68	26	6
SBC	277	81	7	12
RS	204	94	6	0
PS	55 (11 [#])	15 (76 ^{##})	-	0

Table 5.9: Prediction accuracy of resources in bot traces

[#]The number indicates the resources that are of interest to the adversary

^{##}Accuracy computed with respect to the number of resources of interest

of resources (for instance, IC_1 and RS) Snoopy had a high prediction accuracy since the encrypted size of most of the resources were very distinct from each other. On the other hand, Snoopy had a low prediction accuracy for most websites with a high number of resources (for instance, B_2 and B_5). This is because, in practice, most of the web resources in such websites have similar sizes. However, the website IC_2 was an exception where Snoopy had a relatively high prediction accuracy despite a large number of resources. This was because, most of the resources in this website had distinct sizes. Also, note that out of the 55 resources in the website PS, only 11 resources were of interest to the adversary, and their download sequence was sufficient to identify all the 40323 webpages uniquely. Details about this can be found in our recent work (Mitra *et al.*, 2020).

5.6.4.10 Relation between resource prediction accuracy and webpage prediction accuracy

As seen from Table 5.4 and Table 5.9, the relationship between web resource prediction and webpage prediction accuracy is not straightforward. We now discuss three different

scenarios that we encountered during the evaluation of Snoopy.

Case 1: High resource prediction accuracy and high webpage prediction accuracy.

In case of websites IC_1 and B_3, we observed a high correlation between web resource prediction accuracy (91% – 94%) and webpage prediction accuracy (93% – 97%). While such results are quite intuitive, such direct correlation was not observed in case of some other websites;

Case 2: Low resource prediction accuracy and high webpage prediction accuracy.

In case of IC_2, B_1, B_2 and B_5, we see low values of resource prediction accuracy (47% – 75%) but relatively high values of webpage prediction accuracy (89% – 99%). The website B_2 exhibits an extreme case of this behavior with only 47% resource prediction accuracy but 99% webpage prediction accuracy. The reason for this is though the overall resource prediction accuracy was low, Snoopy was able to identify the critical resources that are unique to a webpage; and,

Case 3: High resource prediction accuracy and low webpage prediction accuracy.

In case of the website RS, though the resource prediction accuracy was high (94%), the webpage prediction accuracy was relatively low (75%). This was because most of the web resources that Snoopy could predict correctly were non-critical resources that were shared by multiple webpages.

Failure Analysis – Our analysis reveals that the most common reasons due to which Snoopy could not perform well are (1) when the identified resources are associated with a lot of other webpages; (2) when the resources critical for identifying a webpage are already cached at the browser and do not get downloaded; and, (3) increase in number of possible webpage transitions in case of multi-tab browsing. For instance, Snoopy may correctly identify a web resource but may not be able to determine which tab it came from in case webpages open on multiple tabs at the same time share the resource. This increases the confusion and leads to incorrect predictions. To improve the webpage

prediction accuracy in cases where Snoopy has a poor accuracy, we explore an ensemble of Snoopy and a ML-based technique in § 5.6.5, that complies with the constraints of a finite query model.

5.6.5 Snoopy-ML Ensemble

Our experiments (Refer to § 5.6.4) show that there are certain cases where the webpage prediction accuracy of Snoopy is approximately 80%. This is comparatively lower than most of the other websites where Snoopy could achieve a prediction accuracy of more than 90%. This motivated us to design a small experiment where we analyze the effectiveness of ML-based webpage identification techniques in classifying only those webpages that Snoopy fails to classify.

5.6.5.1 Experiment 1

For our experiment we consider the website B₄, and the scenario described in § 5.6.4.3, where we study the generalization capability of Snoopy and ML-based techniques in terms of user interests. We had observed that for 200 webpages of website B₄ and 10 traffic samples per webpage, Snoopy had achieved a prediction accuracy of 78%. We first examine how the best-performing ML-based technique, ML_Wfin (Yan and Kaur, 2018), performs when trained and tested on only those webpages that Snoopy failed to identify.

We first inspect the prediction results of Snoopy and identify 70 out of 200 webpages that Snoopy could not predict correctly. Next, we train the ML_Wfin model using 10 samples from each of these 70 webpages. During validation, the ML model assigns a probability to each of these 70 classes (webpages) for each validation point. For a given validation point (say, t_j), the webpage that gets assigned the highest probability (denoted as P_j) is the prediction output. For the 70 webpages, we observe a validation accuracy of $\approx 94\%$. The results of this experiment motivated us to further explore if there is a way to combine Snoopy and an ML-based technique into an ensemble that outperforms each of the aforementioned webpage identification techniques individually; subject to the limitations on the number of queries.

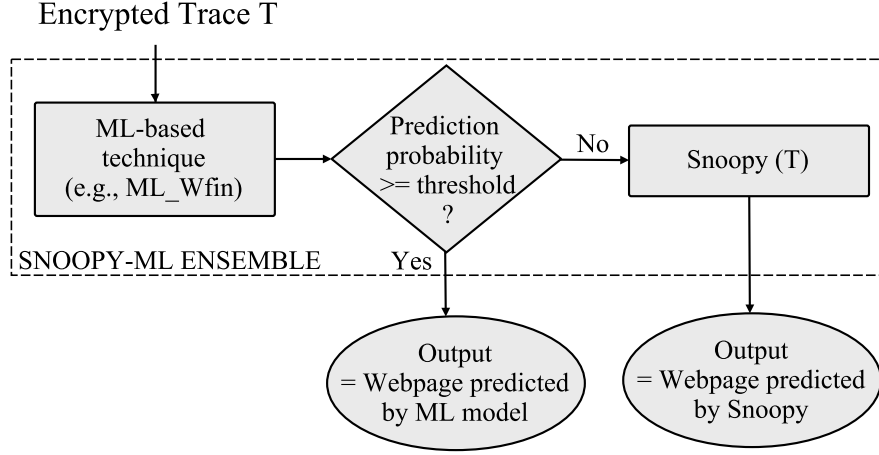


Figure 5.9: Webpage prediction by Snoopy-ML ensemble

5.6.5.2 Experiment 2

We build a very basic ensemble model with Snoopy and ML_Wfin (Yan and Kaur, 2018) as sub-modules. We train (webpage profiling) Snoopy on 130 webpages of the website B_4, while we train ML_Wfin on the remaining 70 webpages of B_4. We use 10 traffic samples from each webpage for this training. Figure 5.9 shows how this ensemble is used for predicting webpages accessed in a given encrypted trace T. The trace T is first passed to ML_Wfin for prediction. ML_Wfin predicts one out of the 70 webpages it has been trained with, with a probability P_{ML} . Based on our insights from Experiment 1 (refer to § 5.6.5.1), we calculate a threshold probability $P_v = \min(P_j), \forall_j$ (where, P_j is the maximum probability assigned to a class by ML_Wfin for validation point t_j) to determine if T belongs to one of the 70 webpages that ML_Wfin was trained with.

If $P_{ML} \geq (P_v - 10)$ (error margin = 10%), we consider the webpage predicted by ML_Wfin as the ensemble output. If $P_{ML} < (P_v - 10)$, the trace T is passed to Snoopy for prediction, and we consider the class predicted by Snoopy as the ensemble output. For the scenario we considered in Experiment 1, the observed value of P_v was 32.5%.

For testing this basic ensemble, we use a dataset comprising encrypted traffic traces from the website B_4, collected using the same browsing context as the training dataset. Our

ensemble achieved a prediction accuracy of 97%, which is much higher than the accuracy achieved by Snoopy (78%) or ML_Wfin (76%) separately. Since in this experiment we considered the case where generalization is performed only with respect to user interests, we did not need additional training samples for ML_Wfin. However, note that we would have to collect more training samples for ML_Wfin if we considered other factors in GF as well. While this might not be the best possible ensemble, the fact that even such a basic ensemble achieved a high accuracy shows an interesting future research direction.

5.7 COUNTERMEASURES

Existing countermeasures (Gong and Wang, 2020; Cai *et al.*, 2012; Dyer *et al.*, 2012; Ling *et al.*, 2022) perform traffic pattern obfuscation to prevent website/webpage fingerprinting attacks. Deployment of these countermeasures require the participation and co-operation of the website users. Website user participation was acceptable in these cases since these countermeasures were primarily designed for protecting the privacy of website users. However, the privacy leak presented in our work is more critical to organizations than the website users. In such a case, existing countermeasures cannot be deployed. Instead, we explore alternatives that can be deployed by the organizations, even without the participation of their website users.

Snoopy, although presented as a webpage fingerprinting technique, can also be seen as a tool that can be leveraged by organizations to design websites that are resilient to mass surveillance, before they are deployed in the public domain. Once a website has been designed, the web developer of the organization can set up a local test-bed to generate encrypted traffic traces corresponding to different browsing contexts that mimic real website users. The developer can then use Snoopy to determine if these traces are vulnerable. If so, the detailed prediction reports of Snoopy can help the developer understand which resources in the website leak information. Following are some of the ways in which the detailed report can be used for designing a countermeasure:

Altering the web resources. The web developer can alter the size of the vulnerable web resources (e.g., images, videos, texts) and/or change the sequence in which they get downloaded. Web resource sizes can be changed either by padding them with extra bytes or by re-rendered them from the scratch. For changing the download order, the developer can either modify the HTML or, when HTTP/2 deployment is available, change the download priority of resources. The amount by which the size of a resource should be changed can also be estimated from the reports of Snoopy.

Restructuring the website. Restructuring websites can sometimes help minimize the impact of surveillance on the privacy of the organization. For instance, a news website that features one news article per webpage can be restructured by grouping multiple articles in a webpage. Although Snoopy may still infer the webpage accessed, the information may not be useful in estimating the popularity of news articles on that webpage.

5.8 SUMMARY

In this chapter, we proposed Snoopy, a webpage fingerprinting framework for performing mass surveillance while complying with a finite query model. Snoopy achieves this objective with $\approx 90\%$ accuracy when evaluated on most websites, across various browsing contexts, while requiring only 3 – 10 traffic samples per webpage. We also presented some preliminary findings on the possibility of a Snoopy-ML ensemble model for websites where Snoopy alone could not achieve the best results. In such a case, when we tried to generalize across various user interests, a simple ensemble of Snoopy and an ML-based technique achieved $\approx 97\%$ accuracy while adhering to the finite query model.

This work shows that mass surveillance is possible even when the web traffic is encrypted. Today, the websites of the most popular banks, insurance companies, retail stores, and even political survey websites are vulnerable to such mass surveillance, without the knowledge of the organizations or the website visitors. This is irrespective of whether

the surveillance is conducted by a Government agency or by a malicious entity. We hope that this work motivates security researchers to investigate countermeasures for the network side channel leaks, and opens a dialogue on mass-scale privacy attacks and defenses. We have suggested few potential countermeasures that can be explored further for protecting against the proposed privacy attack.

CHAPTER 6

NOT JUST ANOTHER PRIVACY LEAK: AN ACTIVE ETA-BASED ATTACK ON A *LEAKY* WATER TREATMENT PLANT

In Chapters 3- 5, we saw how network adversaries can infer fine-grained information about the browsing activities of internet users, when they visit a website. In these attacks, the surveillant is only interested in obtaining sensitive information about its targets, and does not cause any immediate damage to the target. Rather, the adversary collects sensitive information about its target over a significant amount of time to understand its characteristics or behavioral patterns. The adversary might eventually use this knowledge for launching a deadly active attack (e.g., DoS attack on the most popular service offered by a targeted website), or might simply sell the information to competitive business entities or political organizations in exchange for monetary benefits. Contrary to these passive privacy attacks, in this chapter, we explore the potential of encrypted traffic analysis in designing an active attack, where the adversary aims to cause immediate and significant damages to its target. For this, we explore a different type of internet-connected system, i.e., an industrial control system (ICS). Industrial Control Systems have always been a lucrative target of cyber attackers, due to the critical nature of the infrastructure they control and the large number of people that they cater to. Attacks on these systems not only cause huge losses to the targeted company, but also results in disruption of human lives.

The architecture, communication protocols and applications used in an ICS are completely different from that of a web server. Furthermore, the operational constraints of an ICS are much more stringent than that of web applications. For instance, ICS communications

are much more time-sensitive than webpage loads. However, one similarity between web servers and internet-connected ICS devices is that they use the same TLS encryption standard for protecting the application data from network adversaries. In this chapter we investigate if ICS communications are also vulnerable to ETA-based network surveillance.

The growing trend of interfacing control communications of industrial control systems (ICS) with the Internet has resulted in exposure of these sensitive communications to adversaries on the Internet. This has made the Operational Technology (OT) components of the ICS vulnerable to malware infiltration, data/command modification attacks and man-in-the-middle attacks. To prevent such attacks, industries are enforcing end-to-end encryption of ICS traffic, strong endpoint security, and strong intrusion detection systems.

In this work, we show that despite such security measures it is possible for an on-path network adversary to cause damage to an ICS. We propose an adversarial scheme that works on the principle that *delaying or dropping packets carrying critical ICS commands/data can disrupt ICS operations*. We show that characteristics of encrypted ICS traffic can be used as side-channels by an adversary for identifying these critical packets. We demonstrate our attacks on an emulation of the Secure Water Treatment (SWaT) testbed at the Singapore University of Technology and Design (SUTD), that uses state-of-the-art ICS protocols on top of Transport Layer Security (TLS). Using our proposed techniques we demonstrate 4 different attacks that could potentially damage the mechanical components or degrade the performance of the ICS. To the best of our knowledge, this is the *first traffic analysis based attack on encrypted ICS control communications*.

6.1 BACKGROUND

Cyber attacks on Industrial Control Systems (ICS) can damage critical infrastructure such as water treatment plants and power grids, disrupt the lives of civilians and cause financial losses. Well-known ICS attack strategies include implanting malware

for injecting spurious data/commands (Symantec, 2011), disabling ICS components and services (Holloway, 2015; Zetter, 2015), and launching denial-of-service (DoS) attacks (Stanger, 2021). In *traditional* ICS systems, these attacks would invade the ICS through the Internet-connected Information Technology (IT) network in order to propagate to the private Operations Technology (OT) networks. Strategies to detect such attacks and protect the OT networks have been extensively explored (Lin *et al.*, 2018; Gao and Morris, 2014; Gualandi and Casalicchio, 2019; Kirrmann *et al.*, 2009).

In recent times, the threat surface has expanded significantly in *modern ICS systems*. This is due to the growing trend of interfacing the OT networks directly with the public Internet (Galloway and Hancke, 2012; European Network and Information Security Agency, 2011; Andreeva *et al.*, 2016). Internet connectivity exposes the OT network to adversaries on the Internet, enabling them to infiltrate the OT network of the ICS without infiltrating the IT infrastructure (European Network and Information Security Agency, 2011), or to modify data and commands from compromised on-path network devices (Maynard *et al.*, 2014; Chen *et al.*, 2015; Urbina *et al.*, 2016; Sanchez, 2019).

To alleviate these security risks, administrators of modern ICS systems have adopted measures for ensuring strong endpoint security, strengthening intrusion detection systems (IDS), and end-to-end encryption of messages (data and commands) using Internet-compatible encryption protocols such as the Transport Layer Security (TLS) protocol (Rescorla, 2018; Rinaldi, 2019; Schneider Electric USA, Inc., 2018). *In this work, we propose a novel network attack technique that can disrupt the operations of a modern ICS despite the aforementioned security measures.*

We show that it is possible for an adversary with *minimal privileges* to accomplish her goals even when strong security mechanisms *prevent her from compromising the ICS devices, inspecting the live system logs, injecting malware, and decrypting, modifying and replaying the ICS traffic*. In particular, we consider an adversary who compromises a network device that routes the traffic of the targeted ICS, so that it can monitor the

encrypted ICS traffic. The adversary might also be located outside the perimeter of the ICS. We design an adversarial scheme that leverages the observation that certain messages (data updates/commands) exchanged within an ICS are *highly sensitive* in nature, and if such messages are delayed or not received by the intended recipient, an ICS may incur significant damage. Although an Internet-connected ICS is designed to tolerate natural network fluctuations, it is not resilient to delay or loss of sensitive messages beyond a threshold (Krotofil *et al.*, 2014). Therefore, a network adversary can delay/drop these sensitive messages at the compromised device to disrupt the ICS operations. In the real world, any rogue ISP or an adversary who compromises an ISP router can have these capabilities (Bar-Yanai *et al.*, 2010; Okada *et al.*, 2011; Kazlouski *et al.*, 2021; Barbieri *et al.*, 2021).

In this work, we design an adversarial technique that introduces *targeted network perturbations* to delay/drop *only* the packets that carry sensitive messages. This technique is much stealthier compared to existing ICS network attacks that indiscriminately introduce network perturbations (Korkmaz *et al.*, 2016), and could be detected by the ICS administrators (Zhang *et al.*, 2000). However, *identifying these sensitive messages is challenging for a network adversary when the ICS traffic is encrypted*. To alleviate this problem, we design *encrypted traffic analysis* (ETA) techniques for identifying the sensitive messages from the encrypted ICS traffic. The proposed technique uses *encrypted traffic metadata to fingerprint the sensitive ICS messages*. The fingerprints can be used by a network adversary to distinguish sensitive messages from non-sensitive ones while eavesdropping on the encrypted ICS traffic. Traffic metadata that act as side-channel information include the number of packets, size of packets, packet source, and destination. By delaying or dropping only the sensitive packets, the adversary can often cause damage to an ICS in ways *that are as devastating as modifying the packet payloads*. While ETA techniques are popular in the context of website fingerprinting (Cai *et al.*, 2012), *to the best of our knowledge, ours is the first ETA-based attack on a modern ICS that uses end-to-end encryption*.

Designing ETA-based techniques for identifying and fingerprinting sensitive messages is challenging because of the following reasons – **(1) *Lack of real-time ICS state information*** - The sensitivity of ICS messages varies with the state of its internal components (operational state) at a given point of time. For instance, a *plant_shutdown* message is more critical in case of an electrical hazard, than in the case of a routine check. However, the adversary we consider does not have access to real-time system logs that record the operational states of the ICS, and; **(2) *Lack of knowledge about implementation details*** - In the absence of real-time system logs, determining the sensitivity of ICS messages becomes even more challenging for an adversary who is not aware of the implementation details of the targeted ICS. These challenges deter an adversary from using traditional ETA-based techniques (Wang *et al.*, 2014; Papadogiannaki and Ioannidis, 2021) designed for fingerprinting publicly accessible web applications that do not exhibit highly time-sensitive behavior, and allow the adversary to use the system for collecting traffic samples corresponding to known browsing activities. On the other hand, an ICS is a private system, where the lack of visibility into the real-time system state information makes it difficult for an adversary to fingerprint the sensitive messages.

In this work, we address the aforementioned challenges while considering the following two different adversarial scenarios.

- **Scenario 1:** This adversary does not have access to the real-time device logs and message decryption keys, but is *aware of the system design, implementation details, communication protocols used, and the network configuration of the targeted ICS*. This models adversaries who have insider information/connections, e.g., a disgruntled former employee of the organization who has knowledge about the plant. Note that many real-world attacks have been launched by such adversaries, for instance, the Maroochy Shire cyber attack (2000), and the recent attack on a water plant in Ellsworth, Kansas (Shorman and Vockrodt, 2021). For such adversaries, we design an ETA technique that exploits the system design and implementation details (static information) to predict the encrypted

network characteristics corresponding to the dynamic operational states of the targeted ICS. The traffic fingerprints of the operational states are thereafter used by the proposed technique for identifying sensitive messages and building their fingerprints.

- **Scenario 2:** We consider a *more common and realistic* scenario, where the adversary has *no knowledge about the system design or implementation details* of the targeted ICS. However, it is *aware of the general properties of ICS* systems, such as, **(1)** Components of an ICS are designed to execute only a fixed set of operations, **(2)** ICS operations are repetitive and periodic in nature. For example, the ICS controlling a power generation plant performs a fixed set of operations every 24 hours to generate 100 MW of power per day, and; **(3)** Each operation executed by an ICS requires exchange of messages (data/commands) between different ICS components. For this scenario, we design a technique for identifying sensitive messages based on the idea that encrypted ICS communications can be modeled as a time-series sequence of periodically occurring repetitive patterns, where each pattern corresponds to the communications involved in executing a fixed set of operations.

In this work, we show that an adversary using the proposed technique can attack an ICS with the goal achieving a specific objective (for instance, damaging a mechanical component) in Scenario 1. Furthermore, even in Scenario 2, the adversary can cause significant damage to an ICS by making it behave in an arbitrary manner.

The proposed adversarial technique has the following three unique advantages – **(1)** *Stealthy* – It would be challenging for an ICS administrator to distinguish the small amounts of targeted network perturbations from natural network fluctuations, **(2)** *Difficult to locate point of attack* – The adversary would not leave footprints on the ICS-internal system and network logs. Even if the attack is detected, locating the adversary on the network path outside the ICS perimeter would be challenging, and; **(3)** *Difficult to prevent and recover from* – A targeted ICS can prevent or recover from the proposed attack only if there is a redundant communication path on the Internet that is free of compromised

network devices. Ensuring this would be challenging if the adversary is powerful, for instance, a rogue Internet Service Provider (ISP).

We successfully demonstrate four different attacks using the proposed adversarial techniques on an emulated version of the 6-stage Secure Water Treatment Plant (SWaT) setup at Singapore University of Technology and Design (SUTD) (SUTD iTrust, n.d.). In this test-bed, we consider that the OT network traffic is encrypted using Transport Layer Security (TLS) (Rescorla, 2018) protocol, and transmitted over the Internet. The test-bed has been shown to mimic real industrial water treatment plants, and uses state-of-the-art commercial communication protocols. Our experiments show that by using network side-channel information, it is often possible for adversaries on the Internet to cause as much damage to an ICS as an adversary who can compromise ICS devices. For instance, the adversary was able to cause a tank in our ICS set-up to overflow by 6.5% of its normal capacity by incorporating packet drops over a duration of only 45 seconds. This attack resulted in only 0.006% packet drops and 1.8% of the packets getting delayed. Even an adversary who was oblivious of the ICS implementation details was able to reduce the output of an ICS sub-process by 5.4%, while dropping only 0.002% of the packets and delaying only 5% of the packets over a period of 90 seconds. We conclude by suggesting potential countermeasures for these attacks, and the associated challenges.

We summarize the contributions of our work as follows.

1. We present the *first ETA technique* that can be exploited by network adversaries for attacking modern ICS systems, without infiltrating the ICS network or breaking the encryption.
2. We develop an emulated version of the SWaT setup at SUTD (SUTD iTrust, n.d.) for studying its encrypted network characteristics. While the original test-bed uses only unencrypted CIP on ENIP communication, our emulated test-bed can be easily configured to use either MODBUS/TCP or CIP on ENIP. These are two of the most widely used

communication protocols in ICS OT networks (Real Time Automation, n.d.b,n). We also consider that the ICS communications are encrypted using the TLS protocol (Rescorla, 2018; Schneider Electric USA, Inc., 2018; Rinaldi, 2019). For this, we plug in a packet size translator that converts unencrypted packet sizes to TLS-encrypted packet sizes. We intend to release the test-bed implementation and artefacts for interested researchers.

3. We have analysed our experimental results to show the impact of different factors (communication protocol, system design, etc.) on leakage of side-channel information from ICS traffic. Understanding the impact of these factors might aid security researchers develop best practices for ICS designers, and develop side-channel leakage-proof encryption techniques that are compatible with legacy ICS systems.

6.2 PRELIMINARY CONCEPTS

In this section, we discuss some preliminary concepts related to industrial control system communications, that are relevant to the ideas presented in this chapter.

6.2.1 Modern Industrial Control Systems (ICS)

6.2.1.1 ICS and its components

An industrial control system (ICS) is a cyber-physical system that controls an industrial process. An ICS typically consists of an Operational Technology (OT) network that controls the mechanical components of an ICS, and an Information Technology (IT) network that consolidates and secures the ICS data and is responsible for data center and cloud computing tasks. In this work, we are only concerned with the *Operational Technology (OT) components of an ICS*, that control and interact directly with the physical components of the ICS. The physical components include **(1) mechanical components**, for example, a water tank for storing sewage water in a water treatment plant, **(2) sensors**, for example, a water level sensor, and; **(3) actuators**, for example, a pump to draw out water from a tank. The cyber components of an ICS OT network typically include **(1) Programmable Logic Controllers (PLCs)** - These are rugged computing devices that are

responsible for making local decisions based on sensor inputs. For instance, a PLC might be used for controlling turning a water pump on or off based on the water level detected by a sensor fitted to a tank, **(2) Supervisory Control and Data Acquisition unit (SCADA)** - The SCADA consists of networked systems that collect data from the PLCs, and analyzes the data to supervise and synchronise the operations of different components of the plant. A human-machine interface (HMI) is often co-located with the SCADA to help the plant operators monitor the functioning of the plant, **(3) Engineering workstation** - This system is mainly used for pushing code and software updates to the PLCs and SCADA, and; **(4) a Historian server** that is used for storing device logs such as sensor readings, actuator states, etc. Such information collected over long periods of time can be used for monitoring process statistics and debugging errors in the system.

A modern ICS is highly automated, modularized, and distributed in nature. Large-scale modern ICSes are distributed geographically and monitored remotely by the plant operators. The entire industrial process controlled by such a modern ICS is often decomposed into smaller sub-processes. Most of the computations necessary for process-control are performed locally for each sub-process. The SCADA only performs the computations necessary for synchronizing all the sub-processes.

6.2.1.2 Communications in a Modern ICS

The communication system of industrial control systems can be modeled using a layered architecture. The basic communication layers in a modern ICS network are as follows – **(1) Layer-0** corresponds to the communication between the PLCs and the sensors and actuators within a sub-process. These communications are done over private networks that are isolated from the Internet, **(2) Layer-1** corresponds to the *plant control network*, i.e., the communication among the PLCs and the SCADA, and; **(3) Layer-2** corresponds to the Information Technology (IT) system communication. These systems have been conventionally connected to the Internet.

To facilitate distributed process control and remote monitoring in modern industrial

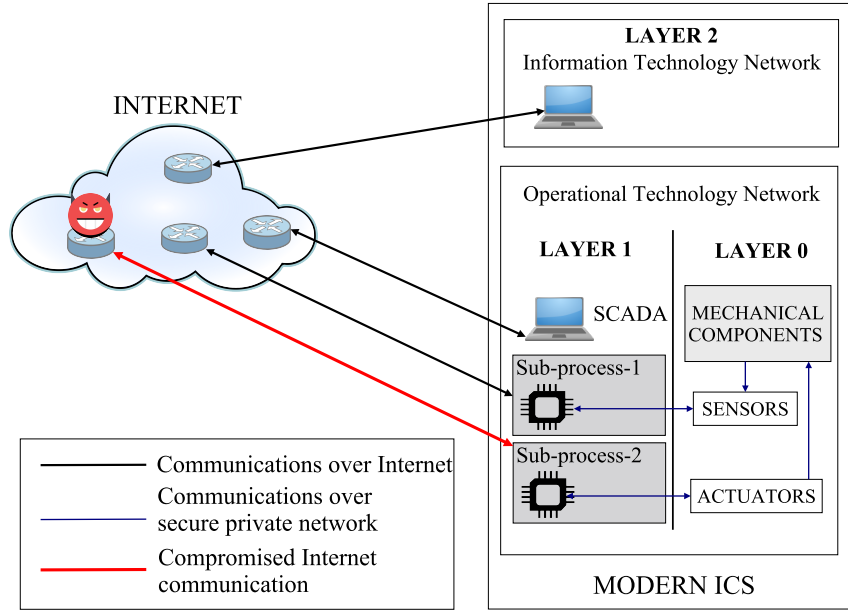


Figure 6.1: Modern ICS communications and emergence of new threat surface in the form of Internet communications.

control systems, the *Layer-1* communications, i.e., communications among the PLCs and the SCADA, have been connected over the Internet in recent years (Galloway and Hancke, 2012; European Network and Information Security Agency, 2011; Andreeva *et al.*, 2016). *In this work, we only focus on these Internet-connected Layer-1 communications.*

To facilitate the integration of Layer-1 communications with the public Internet, some of the legacy ICS communication protocols (such as Modbus (Real Time Automation, n.d.a) and CIP (Common Industrial Protocol) (ODVA, 2016)) have been interfaced with the TCP/IP stack of the Internet (for instance, Modbus/TCP (Real Time Automation, n.d.b) and CIP over ENIP (Common Industrial Protocol over Ethernet IP) (Real Time Automation, n.d.c)).

Security of modern ICS communications. Figure 6.1 shows the layered communication system of a modern ICS, and the threat surface that we consider in this work. An adversary requires physical access to the ICS devices or access to the private network for compromising Layer-0 communications. Therefore, this layer is secure against

attackers on the Internet. Layer-2 communications are connected to the Internet and directly exposed to human interactions. These communication systems have several known vulnerabilities as well as standard security protocols for protection against cyber attacks (Thomas and Chothia, 2020).

We focus on the security of Layer-1 communications, that have been traditionally isolated, but are now being integrated with the Internet. It is challenging to integrate the security protocols used in legacy ICSes with the Internet Protocol (IP) stack used by modern ICSes. This is because protocols used for securing private communications in legacy systems (for instance, IPSec (Kent and Seo, 2005)) do not provide end-to-end security on the Internet (Fauri *et al.*, 2017). Therefore, these communication paths can be vulnerable to network adversaries on the Internet. To facilitate the interfacing of legacy ICS communication systems to the Internet, SCADA industries have adopted the Transport Layer Security (TLS) (Rescorla, 2018) protocol for securing ICS traffic over the Internet (Fauri *et al.*, 2017; Schneider Electric USA, Inc., 2018; Rinaldi, 2019). TLS is the state-of-the-art protocol for securing web traffic on the Internet, and is designed to provide data privacy and security against man-in-the-middle (MiTM) attacks.

6.2.2 Related Works

6.2.2.1 Network Perturbation-based ICS Attacks

Existing adversarial techniques that work by disrupting sensitive ICS communications (Korkmaz *et al.*, 2016; Rao and Kesidis, 2003; Krotofil *et al.*, 2014) either need to compromise an ICS-internal device (PLC or network device) from which the adversary can observe the message contents (Korkmaz *et al.*, 2016; Rao and Kesidis, 2003; Krotofil *et al.*, 2014), or drop *all* packets corresponding to a particular protocol for a certain amount of time (Zhang *et al.*, 2000). The first category of attacks can be detected by malware intrusion detection systems (Long *et al.*, 2005). Existing works (Pal *et al.*, 2014) that address the second category of attacks identifies an instance of packet loss/delay as adversarial if **(a)** an ICS service exhibits discriminatory behavior towards a

particular component, or; **(b)** the traffic volume and timing patterns exhibit significant deviation from normal behavior.

Existing encrypted traffic analysis-based adversarial techniques have only focused on distinguishing encrypted ICS control traffic from all other types of traffic on the Internet (Barbieri *et al.*, 2021). To the best of our knowledge, no prior work has performed fine-grained analysis on encrypted ICS control traffic using ETA-based techniques.

Impact of network perturbations. Modern ICS OT communications use the Transmission Control Protocol (TCP) (Information Sciences Institute of USC, 1981), owing to its reliability. When a packet is dropped or delayed, TCP automatically attempts to re-transmit the packet. Therefore, a packet dropped/delayed by an adversary might still be re-transmitted at a later time. However, the number re-transmission attempts is limited by a Re-transmission Timeout (RT) period.

6.2.2.2 Challenges in performing ETA attack on ICS

Existing ETA techniques used for web applications (Wang *et al.*, 2014; Papadogiannaki and Ioannidis, 2021) cannot be directly used for analyzing encrypted ICS control traffic. Unlike web applications that are publicly accessible and available to an adversary for fingerprinting, an ICS is a closed system that is not accessible to unauthorized personnel. Therefore, the adversary cannot access the device logs, and hence, it cannot fingerprint ICS operations and device states. Moreover, modeling the behavior of complex large-scale ICSes is challenging. This makes it challenging for the adversary to predict the internal operations and device states of an ICS from its encrypted network traffic.

6.2.2.3 ETA-based Active Attacks

ETA-based attacks that cause immediate damages to a targeted internet user / internet-connected systems by means of traffic manipulation (delaying/dropping packets) have not yet been widely explored. The most relevant work (Setayeshfar *et al.*, 2022) in this area targets smart home devices and disrupts their operations by dropping/delaying sensitive

signals. However, there is a fundamental difference between attacking a commercial smart device and attacking an ICS. In this work (Setayeshfar *et al.*, 2022), the attacker itself has a smart device (commercially available), which it first uses to generate dummy signals and fingerprint them based on their encrypted traffic characteristics. Unlike industrial PLCs, smart devices generally do not transmit continuous streams of data. They transmit only a finite number of different types of signals. This makes it easy for the attacker to fingerprint them. Later, it targets a user who uses the same device and predicts the user's activities and sensitivity of signals based on traffic patterns. The attacker also drops any highly sensitive signal. On the other hand, an attacker who targets an ICS does not have access to the ICS devices and cannot profile them apriori. Even if the attacker buys a PLC, it would not be able to profile the ICS traffic, since the process parameters and PLC algorithms are proprietary and unknown to the attacker. Therefore, we cannot use the conventional fingerprint-then-predict technique presented in this paper.

6.3 PROPOSED ADVERSARIAL SCHEME

6.3.1 Attacker Capabilities and Assumptions

We consider an adversary whose objective is to disrupt the operations of a targeted ICS. The adversary can either make the ICS behave in an arbitrary manner, or force it to behave in a specific manner so as to achieve a targeted impact.

Adversary position and capabilities - The adversary compromises a network device carrying the traffic between two or more ICS components. It can **(i)** observe unencrypted header fields of all network packets using common traffic monitoring tools such as tshark (Wireshark, n.d.) or deep packet inspection tools, and; **(ii)** drop and delay packets – at the outset this might seem to be more intrusive as compared to conventional ETA attacks. However, introducing network perturbations requires the same level of privilege (i.e., super-user access) as monitoring traffic on a network device. There are a large number of network devices on the Internet that are susceptible to getting

compromised (Matherly, 2022).

Assumptions and scope - (i) The adversary cannot compromise any of the internal ICS devices. Therefore, it cannot access device logs, nor observe the mechanical components of the ICS; (ii) the adversary can neither hijack the decryption key nor break the encryption of the ICS traffic; (iii) the adversary can passively observe the encrypted traffic characteristics from the compromised device for a sufficiently long time so that it can learn the communication patterns of the ICS; (iv) the adversary can introduce network perturbations. However, the identification of targeted ICS traffic from compromised on-path network device is out of scope of our work. Rather, we assume that the adversary can compromise an Internet router that routes the targeted traffic, and can identify the ICS traffic using known techniques (Barbieri *et al.*, 2021) that exploit encrypted traffic metadata; and, (v) we also assume that the adversary cannot replay packets. This is a reasonable assumption since most modern time-sensitive applications use time-bound session keys and timestamped messages to prevent packet replay attacks. Therefore, in the real world, if an adversary attempts to replay packets without changing the timestamp, the server/client would consider it as corrupt data and/or trigger an alarm.

6.3.2 Attack overview

A distributed ICS process can be modeled as a collection of synchronized sub-processes. The control logic of each sub-process can be represented as continuous-loop state-transition system, where the state transitions are driven by message exchanges (Chow and Tipsuwan, 2001; Witsch *et al.*, 2010). An ICS sub-process can be disrupted by delaying/preventing state transitions to take place at their scheduled time. We propose an adversarial technique where the adversary disrupts a targeted sub-process by delaying/dropping messages that drive state transitions in the targeted sub-process. Such messages are considered to be *sensitive* by the adversary.

Figure 6.2 shows an overview of the proposed adversarial technique. Figure 6.2(a) depicts the ICS behavior in the absence of the adversary. We consider a scenario where

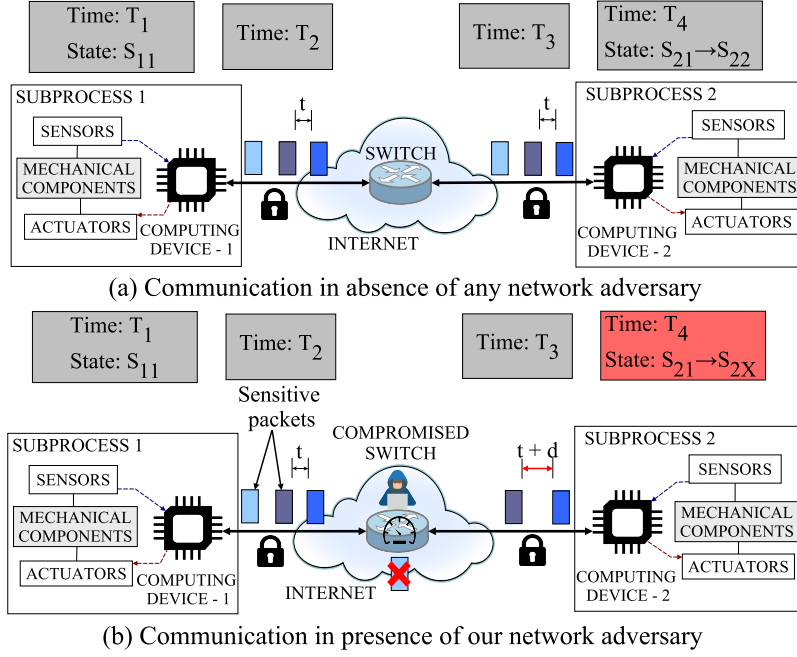


Figure 6.2: Overview of proposed adversarial scheme

the contents of the packets sent by subprocess-1 triggers a state transition ($S_{21} \rightarrow S_{22}$) in subprocess-2 at time T_4 . Figure 6.2(b) depicts the same scenario in the presence of the adversary controlling one of the on-path network devices. In this case, the adversary delays (by d time units) and/or drops one or more packets that are sensitive to the operation of the ICS. This causes subprocess-2 to transition from state S_{21} to state S_{2X} at time T_4 , where S_{2X} is different from S_{22} .

Not all ICS messages are responsible for a state transition. Therefore, to disrupt a state transition, the adversary needs to precisely delay/drop packets that are crucial for the state transition. However, inferring the current state of the targeted process and identifying these sensitive packets is challenging when the ICS traffic is encrypted. Moreover, the adversary does not have access to device logs or visibility into the ICS components. To address these challenges, we propose encrypted traffic analysis-based techniques for fingerprinting ICS states and the sensitive messages. The proposed techniques are discussed in the subsequent subsections.

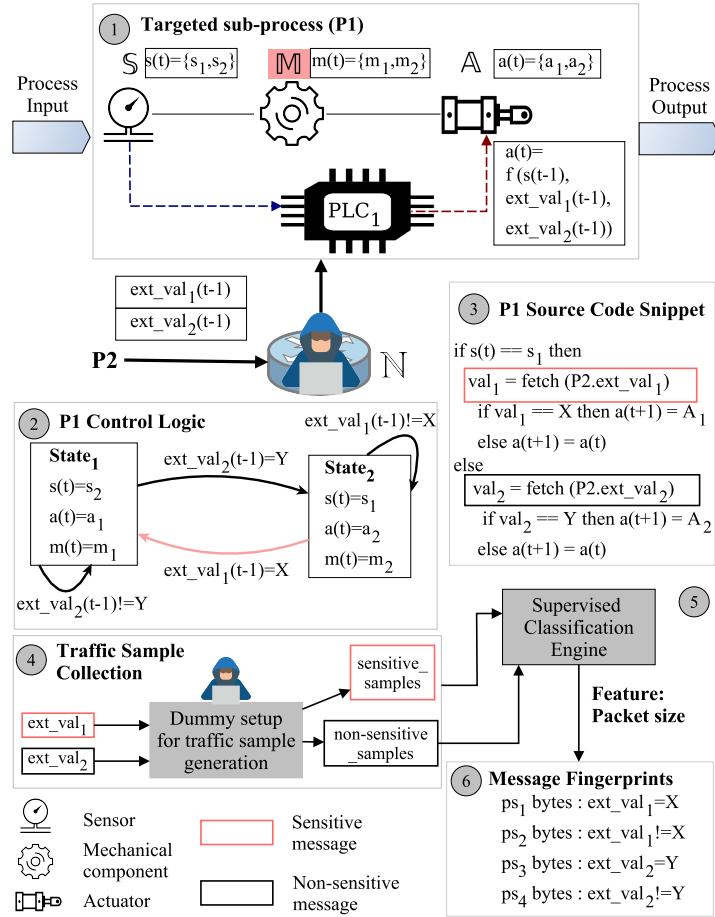


Figure 6.3: Process analysis and message fingerprinting

6.3.3 Encrypted Traffic Analysis Techniques

We consider two scenarios – one where the adversary is aware of the design and implementation details of the targeted ICS (Scenario 1), and another where it is oblivious of the same (Scenario 2). However, in both cases, the adversary does not have access to the real-time device logs and the network traffic decryption key.

6.3.3.1 Scenario 1 - ICS design and implementation details are known to adversary:

This attack has the following two phases – (i) *Process analysis and message fingerprinting*, and; (ii) *active attack*.

Process analysis and message fingerprinting.

In this section, we describe how the adversary uses the information available to her for characterizing sensitive messages when they are transmitted over an encrypted communication channel. We illustrate this algorithm with a sample ICS set-up, as shown in Figure 6.3. Figure 6.3 shows a sub-process P1 targeted by the adversary. The sub-process comprises a mechanical component $\mathbb{M}$, that operates on the process input to generate the process output. Let us assume, the mechanical action produced by $\mathbb{M}$ at time t is denoted by $m(t)$, where $m(t) \in \{m_1, m_2\}$. $\mathbb{M}$ is controlled by an actuator $\mathbb{A}$, and $m(t)$ is a function of the state of the actuator, denoted by $a(t)$, where $a(t) \in \{a_1, a_2\}$. The value of $a(t)$ is computed by PLC_1 . The value of $a(t)$ is a function of the reading of the sensor $\mathbb{S}$ in the previous clock cycle (denoted by $s(t-1)$, where $s(t) \in \{s_1, s_2\}$), and two other values, viz., $ext_val_1(t-1)$ and $ext_val_2(t-1)$, that are fetched from a different sub-process P2. The values of $ext_val_1(t-1)$ and $ext_val_2(t-1)$ are communicated through the network device $\mathbb{N}$ over an encrypted communication channel. Timely operations of the sub-process is crucial for ensuring the quality of the process output.

We consider an adversary that compromises the network device $\mathbb{N}$ to disrupt the sub-process P1. The adversary performs the following steps for process analysis and message fingerprinting.

Step 1: *Identifying attack objective ($attack_obj$)* - First, the adversary inspects the ICS design and layout to identify the attack objective, $attack_obj$. In this example, let us assume that $attack_obj$ is to disrupt the operations of the mechanical component $\mathbb{M}$.

Step 2: *Identifying sensitive process parameters ($attack_params$) and attack time (T)*

- To execute an attack, the adversary needs to identify the process parameters crucial for achieving $attack_obj$, and the time of attack T . For this, the adversary parses the control logic (state-transition representation) of the targeted process to identify (i) the state transition that needs to be disrupted for achieving $attack_obj$, (ii) the process parameters that are responsible for the state transition ($attack_params$), and; (iii) the

time of the attack T . In the example shown in Figure 6.3, let us assume that the adversary needs to prevent the process $P1$ from transitioning to $State_1$ ($\{a = a_1, m = m_1\}$) from $State_2$ ($\{a = a_2, m = m_2\}$) when $s = s_1$ and $ext_val_1 = X$. Therefore, the adversary considers $attack_params = \{ext_val_1\}$, and $T = t$, such that $ext_val_1(t) = X$.

Step 3: *Identifying sensitive and non-sensitive messages in process implementation -*

The adversary needs to ensure that the targeted process does not receive the updated value of $attack_params$ at time T . For this, it needs to identify the packets that carry the value of $attack_params(T)$. For this, the adversary first parses the source code of the targeted process and extracts all the network communication function calls. The sequence of network function calls and the messages they carry are also important for the adversary, since it represents the state of the process at a given instant of time. Next, the adversary establishes a correlation between the process parameters and the network messages that communicate the parameter values. The network messages that carry the values of $attack_params(T)$ are considered to be *sensitive_messages*, while the messages that carry the values of the other process parameters are considered to be *non – sensitive_messages*. In the given example, the adversary needs to identify the packets carrying the message ' $ext_val_1(t) = X$ '. For the source code snippet shown in Figure 6.3, $sensitive_messages = \{P2.ext_val_1 = X\}$, and $non – sensitive_messages = \{P2.ext_val_1 \neq X, P2.ext_val_2 = Y, P2.ext_val_2 \neq Y\}$. Note that, sometimes a sequence of messages can also be considered sensitive, particularly when there are multiple parameters in $attack_params$, or a state transition of caused by a sequence of message communications.

Step 4: *Collection of encrypted traffic samples (sensitive_samples and non-sensitive_samples) -*

The adversary needs to know how different messages are manifested on the encrypted communication channel. For this, it collects network traffic samples for both sensitive and non-sensitive messages (identified in Step 3) over a local network set up with a pair of devices that communicate using the same application

protocol, transport protocol and encryption protocol as the targeted ICS. The traffic samples corresponding to the *sensitive_messages* are denoted as *sensitive_samples* and those corresponding to the *non – sensitive_messages* are denoted as *non – sensitive_samples*. In this example, the adversary would require encrypted traffic samples corresponding to all possible values of ext_val_1 and ext_val_2 . The traffic samples corresponding to ' $ext_val_1 = X$ ' are considered as *sensitive_samples*, while all other traffic samples are considered as *non – sensitive_samples*.

Step 5: *Identifying feature for message fingerprinting (fp_features)* - Next, the adversary determines the encrypted traffic features that distinguish *sensitive_samples* from *non – sensitive_samples*. For this, the adversary begins by extracting all the commonly known traffic metadata, such as, packet size, time interval between previous and next packets, and traffic burst patterns, from both *sensitive* and *non – sensitive* samples. Using these traffic metadata as features, this method attempts to classify *sensitive* and *non – sensitive* messages using supervised learning techniques. The feature(s) that result in the highest classification accuracy are selected for fingerprinting *sensitive_messages*. The set of such features is denoted by *fp_features*. Note that, the adversary might also use more complex message-sequence based features for fingerprinting sensitive messages. In the example shown in Figure 6.3, we assume that $fp_features = \{packet\ size\}$.

Step 6: *Building fingerprints for sensitive messages (sensitive_msg_fp) and process states* - Finally, the adversary builds the encrypted traffic fingerprints for the *sensitive_messages*. For this, it extracts the values of the features in *fp_features* (identified in Step 5) from the *sensitive_samples* collected in Step 4. These feature values constitute the encrypted traffic fingerprint *sensitive_msg_fp*. Using the same features, the adversary can also fingerprint each state by concatenating the feature values of *all* the messages communicated within a state in order of their occurrence. The fingerprints are stored in a database, and used later during the active attack phase. In this example, we assume that packets carrying the *sensitive_message* have a size of ps_1

bytes, which is different from the size of all other packets.

Active attack. In this phase the adversary monitors the live encrypted traffic of the targeted ICS from the compromised device. Packets carrying encrypted application data can be easily identified using protocol headers (for instance, TLS payload header when TLS encryption is used). From these packets, the adversary extracts the values of the features in $fp_features$. The adversary compares these values with the sensitive message fingerprint(s) $sensitive_msg_fp$ and state fingerprints stored in the database to predict if any sensitive message is being communicated. If a match is detected, it introduces network perturbations to drop/delay the corresponding packets. Note that, in case of connection-oriented protocols such as TCP, the adversary would need to drop all the re-transmission requests/responses as well. In the example considered in Figure 6.3, the adversary might start delaying all packets of size ps_1 bytes, for as long as it wants to prevent the transition of the process P1 to $State_2$.

6.3.3.2 Scenario 2 - ICS design and implementation details are NOT known to adversary:

In this scenario, the adversary is oblivious of the design and implementation details of the targeted ICS. To compensate for this lack of insight, the adversary relies on her knowledge about the general properties of any modern ICS, which include (but are not limited to) the following.

1. ICS control logic can be represented as a continuous-loop state-transition system, where the state transitions are driven by message exchanges. An ICS state can be represented by a sequence of instructions and message exchanges that are executed repetitively and periodically. This leads the adversary to assume the following.

Assumption₁: An ICS state can be characterized by the encrypted traffic patterns generated by the message exchanges within that state. Let the traffic pattern corresponding to an ICS state be denoted by $pattern_1$. Repetitive execution of operations within an ICS state would result in repetitive occurrences of $pattern_1$ in the network traffic, until

there is a state transition.

Assumption₂: State transitions can be identified from the network traffic if the encrypted traffic pattern corresponding to an ICS state (say, $pattern_1$) is different from the pattern corresponding to the next state (say, $pattern_2$). In such cases, the adversary would observe repetitive occurrences of $pattern_1$, followed by repetitive occurrences of $pattern_2$ and so on. The transition of the network traffic from $pattern_1$ to $pattern_2$ would indicate a state transition.

Assumption₃: Due to the periodicity of state transitions, the traffic patterns would also exhibit periodicity in occurrence. However, the state transitions might not always be perfectly periodic. This might happen when the condition for state transitions are dependent on environmental factors. However, even the periodicity of such states do not fluctuate drastically within a short period of time.

2. **Assumption₄:** A state transition can also be caused by an infrequent event. Such events are often critical (for instance, shutdown message sent by the SCADA due to an electrical hazard). Such state transitions would generate traffic patterns that occur infrequently. This leads to the assumption that infrequently observed traffic patterns might correspond to critical messages.

An adversary in this scenario identifies sensitive communications from the encrypted traffic based on the aforementioned assumptions. The sensitive packets are then dropped/delayed for disrupting the ICS operations. This attack has the following two phases – (i) *passive observation and pattern extraction*, and; (ii) *active attack*.

Passive observation and pattern extraction. This phase involves the following steps.

Step 1: *Recording encrypted traffic flow (TF)* - The adversary begins with recording the encrypted traffic flows of the targeted ICS from the compromised network device. The adversary should record the network traffic for at least one complete cycle of the process.

Recording the network traffic for a longer period of time would increase the chances of encountering even the infrequent events.

Step 2: *Extracting encrypted traffic features (S)* - Next, the adversary extracts the characteristics of the traffic flow (TF). Encrypted traffic characteristics comprise values of features such as packet size, direction of packet flow, inter-packet timing intervals, packets burst sizes, traffic volume per second, etc. The adversary might also be interested in extracting values corresponding to a combination of features. Let us assume the adversary considers a set of n such features (including feature combinations) $F = \{f_1, f_2, \dots, f_n\}$. The output of this step would be n sequences of values corresponding to each feature (for instance, a sequence of packet sizes or a sequence of $\langle \text{inter} - \text{packet_arrival_timing}, \text{packet_size} \rangle$ values). Let the set of these sequences be denoted by $S = \{s_1, s_2, \dots, s_n\}$, where s_i is the sequence corresponding to feature f_i .

Step 3: *Identification of repetitive and periodic patterns* - Based on *Assumption₁* and *Assumption₂*, the adversary models each of the sequences in S (refer to Step 2) as a time-series featuring cyclic behavior, composed of repeating sub-sequences. For extracting these repetitive sub-sequences, the adversary might use time-series data modeling techniques similar to those used for modeling industrial processes from system logs (Galbrun *et al.*, 2018). For each sequence $s_i \in S$ ($\forall i \in 1 \dots n$), the output of this step would be (i) a set of patterns that are repeated, denoted as $P_i = \{\text{pattern}_{i_1}, \text{pattern}_{i_2}, \dots, \text{pattern}_{i_m}\}$, where pattern_{i_x} ($\forall x \in 1 \dots m$) is a sub-sequence of s_i , i_m is the number of repetitive patterns in s_i , (ii) the number of times pattern_{i_j} ($\forall j \in 1 \dots m$) is repeated before the next pattern $\text{pattern}_{i_{j+1}}$ occurs in s_i , denoted by num_{ij} , and; (iii) the periodicity of pattern_{i_j} ($\forall j \in 1 \dots i$) in s_i , denoted by per_{ij} .

Step 4: *Determining the timing of attack* - Finally, to determine the attack timing T ,

the adversary studies the pattern characteristics (set of constituent patterns, number of times each pattern is repeated, and the periodicity of each pattern) for each of the feature value sequences in S . The adversary might target any pattern, say $pattern_{i_x}$, in any sub-sequence s_i , and drop/delay packets when $pattern_{i_x}$ is just about to transition to $pattern_{i_{x+1}}$. The adversary can estimate the time of transition from the number of times $pattern_{i_x}$ is consecutively repeated in s_i , and its periodicity in s_i . Note that, selecting the pattern to target is not trivial. This is because (i) the corresponding state transition might not be too crucial for the ICS, and; (ii) if the number of repetitions per cycle or the periodicity of the pattern are not consistent, then the estimated attack timing might be wrong. The adversary would need to adopt a trial and error approach for addressing the first challenge. For addressing the second challenge, the adversary records traffic flows for several complete operational cycles in Step 1, and repeats Steps 2-3 for all the traffic flows. It then selects the patterns that exhibit the minimum variation in number of repetitions and periodicity across the maximum number of features in F (refer to Step 2). The output of this step is the targeted pattern $pattern_{i_x}$, the traffic feature (f_i) that it corresponds to, the average periodicity of $pattern_{i_x}$, denoted as per_{i_x} , and the number of successive repetitions of $pattern_{i_x}$ in s_i , denoted as rep_{i_x} .

Active attack. In this phase the adversary monitors the live encrypted traffic of the targeted ICS process, and extracts the values of the traffic feature f_i selected in Step 4 of the pattern extraction phase. Sequences of these extracted values are then compared with $pattern_{i_x}$. Once a match has been found, the adversary starts counting the number of successive repetitions of $pattern_{i_x}$. The adversary starts introducing network perturbations (drop/delay) as soon as the number of repetitions reaches $(rep_{i_x} - 1)$. The adversary would know that the state transition has been successfully prevented as long as the $pattern_{i_{x+1}}$ is not seen during the active attack. The adversary might also disrupt the communication when it detects the infrequently occurring packets on the network. Such packets might either comprise patterns with the highest variation in periodicity, or a previously unseen pattern.

It must be noted that the maximum tolerable delay of an ICS towards a sensitive message might range between a few milli-seconds to hours. In the proposed technique, the adversary (in the second attack) would need to adopt a trial-and-error approach for understanding how much delay needs to be introduced to cause damages to the targeted ICS. The adversary can start with a small amount of delay and see if that is enough for changing the behavior of the ICS. If the initial attempt is not successful, it might keep increasing the delay in subsequent attempts, until it is successful.

6.4 A CASE STUDY

In this section, we demonstrate the adversarial scheme proposed in § 6.3.3 with a case study. For our case study, we consider the 6-stage Secure Water Treatment (SWaT) Plant test-bed hosted at SUTD, Singapore (SUTD iTrust, n.d.).

6.4.1 Experimental Setup

6.4.1.1 Test-bed.

The Secure Water Treatment (SWaT) Plant test-bed purifies waste water in six stages. Each of the six stages performs a sub-process (raw water storage, chemical dosing, ultrafiltration, dechlorination, reverse osmosis, and backwash) of the entire water treatment process. This test-bed has been designed to mimic real industrial processes, and uses commercial cyber-physical components and state-of-the-art communication protocols.

The test-bed consists of six Programmable Logic Controllers (PLCs), one for each of the six sub-processes of water treatment, and a Supervisory Control and Data Acquisition (SCADA) unit. These components are connected to each other over the Internet. Due to lack of access to the original test-bed, we used an emulated version of the test-bed, where the PLCs and the SCADA had been connected over a mesh topology to emulate the Internet. Figure 6.4 shows the high-level overview of the test-bed.

We emulated the ICS setup using MiniCPS (Antonioli and Tippenhauer, 2015), a

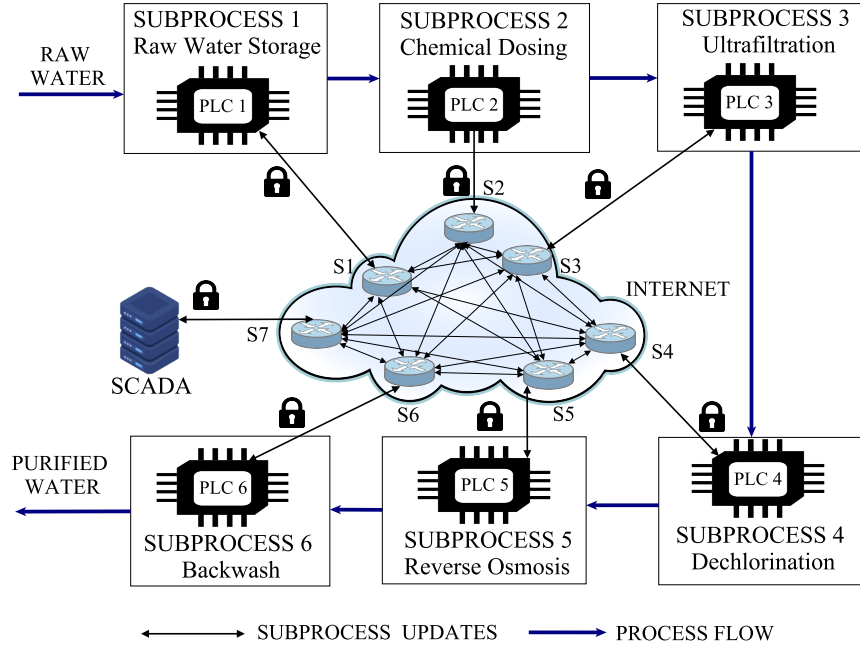


Figure 6.4: High-level overview of the emulated SWaT test-bed

framework for emulating distributed industrial control systems. MiniCPS is built on top of Mininet (Mininet, n.d.), a well-known network emulator. The PLCs and SCADA are implemented as 7 virtual machines (VMs), each connected to a Mininet switch that acts as the Internet gateway for the corresponding sub-process. The SCADA and PLC logic and the values of operational parameters used in our implementation have been taken from the original open-source implementation of the SWaT test-bed (Chen, 2020). We also ensured that the traffic generated by our test-bed is similar to that generated by the original test-bed (based on traffic traces released by the testbed authors (SUTD iTrust, n.d.)). All the functional modules of the SWaT test-bed that run on the PLCs and the SCADA have been implemented using Python.

Communication System and Protocols. The original SWaT test-bed uses a layered communication system, as described in § 6.2.1. *Our proposed adversary can only intercept the communications among PLCs and SCADA.* PLCs in the original test-bed uses Common Industrial Protocol (CIP) over EtherNet/IP (ENIP). We also extend it to use the Modbus/TCP for communications, as these are the two most common protocols

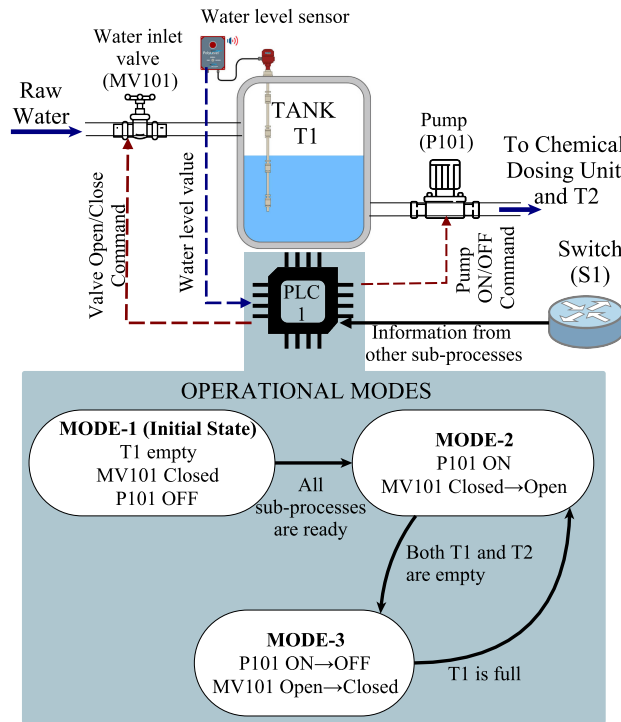


Figure 6.5: Schematic representation of Sub-process 1 (Raw Water Storage) and operational modes of PLC1

in this domain. We further consider that the communication is encrypted using the TLS protocol, as this is one of the most widely adopted protocols for securing ICS OT communication on the Internet (Schneider Electric USA, Inc., 2018; Rinaldi, 2019).

6.4.1.2 Targeted process.

We consider an adversary whose objective is to cause disruptions in Sub-process 1 (raw water storage).

Sub-process 1. Sub-process 1 is responsible for storing raw waste water in a Tank T1 and pumping it out to the subsequent sub-process for chemical dosing. T1 is fitted with a water level sensor, a water inlet valve (MV101) that fills water into T1 when open, and a pump (P101) that draws water out of T1 so that the tank in the next sub-process (i.e., T2) gets filled. The height of water level in T1 has a maximum threshold of 800cm. The components MV101 and P101 are controlled by PLC1 (refer to Figure 6.5), based on the

water level in T1 and the information received from the other PLCs.

Figure 6.5 shows the control logic of PLC1. PLC1 has the following 3 modes of operation - **(i) Mode-1:** This is the initial stage of operation when Tank T1 is empty, valve MV101 is closed and pump P101 is turned off. PLC1 switches to Mode-2 as soon as it receives confirmation from the other PLCs that the entire plant is ready to function, **(ii) Mode-2:** In this mode, initially PLC1 communicates with PLC3 to check if T2 is empty or full. If both T1 and T2 are empty, PLC1 switches to Mode-3. If both T1 and T2 are full, PLC1 closes MV101 and turns off P101. Then, it continuously communicates with PLC3 to check the status of tank T2. If T2 is not full, PLC1 first sets the pump P101 on. If the water level in T2 is below the maximum threshold but not empty, and T1 is not full, then MV101 is also opened, and PLC1 continues in this state until both T1 and T2 are empty at the same time; **(iii) Mode-3:** In this mode, initially MV101 is open and P101 is turned on until T1 is full. Meanwhile, if T2 is full, PLC3 notifies PLC1. If T1 is full, PLC1 checks the status of Tank T2. If T2 is also full at the same time, PLC1 switches to Mode-2. If T2 is not full, PLC1 continues to be in Mode-3 and does not close MV101, since the water would be continuously consumed by the next sub-process.

Goal of the adversary. The goal of the adversary is to make Sub-process 1 deviate from its normal behavior by delaying the mode (state) transitions in PLC1.

6.4.1.3 Adversary setup

We place the adversary on the switch S1 (refer to Figure 6.5 and Figure 6.4) to emulate an adversary who compromises a network device on the communication path connecting PLC1 with the other PLCs. This adversary can only access the metadata, and control the flow of the encrypted traffic routed by S1.

The adversary has the following components - **(1)** a network traffic monitor and packet capturing tool implemented using tshark (Wireshark, n.d.), **(2)** a process analysis module and a pattern analysis module, both of which are implemented in Python, **(3)**

a prediction module implemented using Python for analysis of the captured encrypted traffic, and; (4) a network controller for dropping/delaying packets, implemented using the *ovs-ofctl* (Openvswitch.org, n.d.) and *tc qdisc* (Linux, n.d.) tools.

6.4.2 Evaluation Metrics

Prediction Accuracy. We define the *prediction accuracy* of the adversary as the percentage of the total number of sensitive messages that the adversary can accurately identify.

Stealthiness of an Attack. We compare the relative stealthiness of the adversary during an attack A_1 with another attack A_2 as the ratio of the number of packets dropped or delayed during attack A_2 to the the number of packets dropped or delayed during attack A_1 .

6.4.3 Results

To establish the baseline, we first show the impact of *intermittent random network perturbations* caused by a network adversary on our experimental ICS setup. Thereafter, we show the impact of our proposed encrypted traffic analysis-based adversarial schemes on the same ICS test-bed. Table 6.2 shows a summary of the attacks launched by the adversary. We also discuss the details of the attacks in § 6.4.3.1–§ 6.4.3.3. Thereafter, we present an analysis of the network side-channel leakage in an ICS and their possible causes and effects. Through our experiments and analyses, we intend to answer the following questions:

- (Q1) Does the communication protocol used by an ICS affect the network side-channel leakage? If so, how?
- (Q2) How much can the adversary affect a targeted ICS even when it does not have any information about its design and implementation details?
- (Q3) How stealthy are the proposed ETA-based attacks?

Case No.	Network perturbations introduced	Frequency of triggering network perturbations	Packet loss (in %)	Packets delayed (in %)
1 - 3	1. Delay of 10s for a duration of upto 1 minute 2. Continuous packet drop for upto 1 minute	Delay and drop triggered randomly at an interval of upto 2 mins for 1 hour	0.7	34
4 - 5	1. Delay of 10s for a duration of upto 10 mins 2. Continuous packet drop for upto 10 mins	Delay and drop triggered randomly at an interval of upto 2 mins for 1 hour	4.7	73.5
6 - 7	Delay of 10s to 1min for a duration of upto 10 minutes	Triggered at an interval of upto 2 mins for 1 hour	10	78.5
8 - 10	1. Delay of upto 1 min for a duration of upto 20 minutes 2. Continuous packet drop for 1 - 2 minutes	Delay and drop triggered randomly at an interval of 2-20 mins for 1 hour	1	29

Table 6.1: Details of random network perturbations and their impact on network traffic of the ICS set-up

6.4.3.1 Intermittent random network perturbations (No traffic analysis)

To show the impact of intermittent random network perturbations at S1 on our ICS setup, we introduced a mix of packet drops and delays in random sequence for random periods of time over an hour. The experiment was repeated 10 times for different amounts of duration and frequency of packet drops and delays. Table 6.1 shows the nature of network perturbations introduced over one hour and their average impact on the ICS traffic.

Figure 6.6 shows the impact of the network perturbations on Sub-process 1 at the end of one hour of operation. For Case-1–Case-6, Case-8 and Case-9, we noticed non-uniformity in the rate at which Tank T1 got filled. However, neither any mechanical damage nor any reduction in output of Sub-process 1 was observed by the end of one hour. Also, for all of these 8 cases, the traffic passing through S1 had already lost 1.4% packets and 35% of the packets were delayed (on average) even before any non-uniformity in Sub-process 1 was observed. Such a high impact observed on the ICS traffic is enough to alert the

Sl. No.	Scenario	Attack	Attack Duration	Impact	
				CIP over ENIP + TLS	Modbus/TCP + TLS
1	Random attack	Add jitter and drop packets randomly	Intermittently over 1 hour	In 8 out of 10 cases, we noticed slight non-uniformity in the rate at which Tank T1 got filled. However, there was no mechanical damage or reduction in output of Sub-process 1. In one case Tank T1 overflowed. In the last case, there was a reduction in the output of Sub-process 1.	
2	1	Drop packets when PLC1 is about to switch from Mode-1 to Mode-2	Varied between 45s and 90s.	Delay in filling of T1. Anomaly in volume of water pumped out of T1. The impact is proportional to attack duration.	
3	1	Introduce delay in switch when PLC1 is about to switch from Mode-1 to Mode-2	Varied between 30s and 60s. Delay varied between 100ms and 800ms.	Result is similar to packet drop attack. The delay caused is proportional to attack duration. However, impact is lesser as compared to packet drops.	
4	1	Drop packets when PLC1 is about to switch from Mode-3 to Mode-2	Varied between 10s and 90s.	Tank T1 overflows. Volume of overflowed water is proportional to the duration of the attack.	
5	1	Introduce delay when PLC1 is about to switch from Mode-3 to Mode-2	Varied between 30s and 60s. Delay varied between 100ms and 800ms.	Tank T1 overflows. Volume of overflowed water depends on both amount of delay and duration of the attack. Impact is lesser as compared to packet drops.	
6	2	Drop packets whenever the traffic pattern is about to change	Packets dropped for 90s for each pattern.	Pattern 1: Delay in filling of T1 and anomaly in volume of water pumped out Pattern 2: Drop in the rate at which T1 is filled.	Only one pattern detected. Impact: Reduction in output of Sub-process 1 by 5.4% at the end of 30 mins.
7	2	Introduce delay whenever the traffic pattern is about to change	Introduced 800ms delay for 60s for each pattern.	Impact is similar to but lesser than packet drop attack	Impact is similar to but lesser than packet drop attack
8	2	Drop packets that are the most infrequent	Dropped packets that are large and few in number.	Sub-process 1 completely got disrupted. Tank T1 overflowed eventually.	

Table 6.2: Summary of attacks executed by proposed adversary on our test-bed and their impact

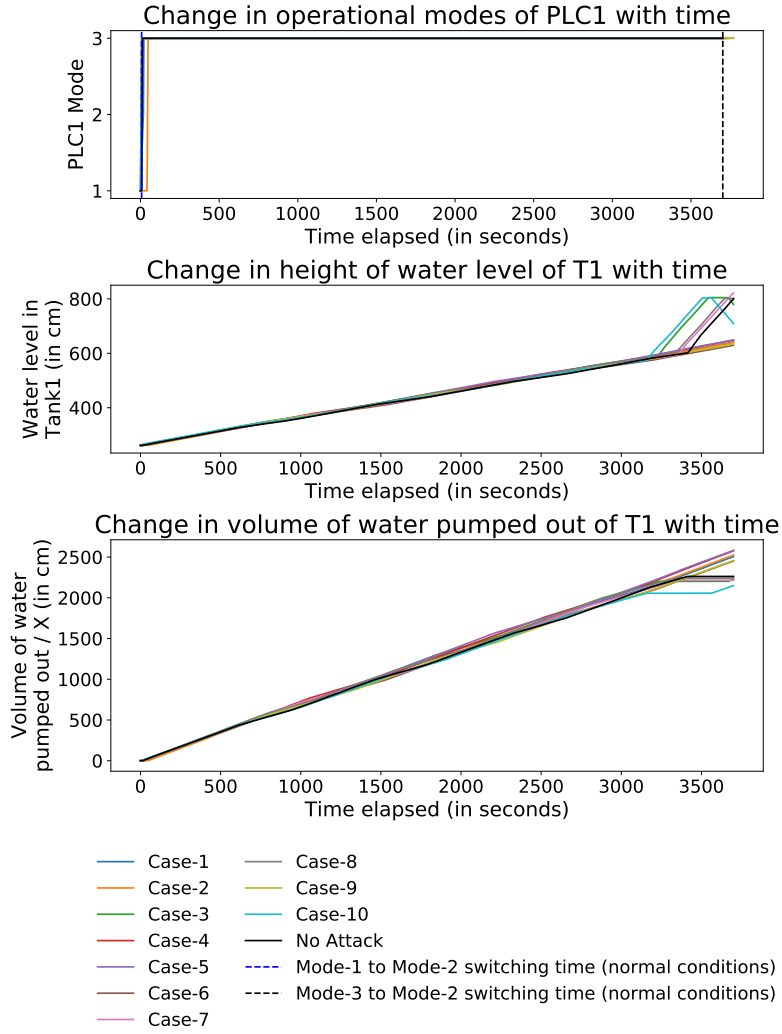


Figure 6.6: Effect of intermittent network perturbations caused by a network adversary (X: T1 cross-section)

ICS administrator. This also shows that our ICS setup is tolerant to a certain amount of network fluctuations. In only 2 out of the 10 cases, we observed damages caused to the ICS, probably due to loss of sensitive packets. In Case-7, Tank T1 overflowed, while in Case-10 there was an 8% reduction in the volume of water pumped out of T1 at the end of one hour. Note that, in Case-7 the ICS traffic lost 11% of the packets and 80% of the packets were delayed, while in Case-10, 0.8% of the packets were lost and 25% of the packets were delayed. Such a high amount of traffic disruption will invariably alert the ICS administrators and prompt them to adopt corrective measures. Therefore,

we conclude that random network perturbations have a very high probability of getting detected and a very low success rate (20% in this case), which makes it unsuitable for stealthy and effective attacks.

6.4.3.2 Scenario 1 - ICS design and implementation details known to adversary:

We now evaluate our proposed encrypted traffic analysis-based attack technique when it is used by an adversary who is aware of the design, control logic, implementation details, and network configuration of our ICS setup. We demonstrate two different attacks in this scenario.

Attack 1: Delaying the operations of Sub-process 1 to reduce the process output.

For executing this attack, the adversary followed the process described in § 6.3.3.1.

In this case, the objective of the adversary was to reduce the volume of water supplied to the second sub-process. This could be achieved by keeping the valve MV101 closed for an extended time, and thereby delaying the filling of Tank T1. This required delaying the transition of Sub-process 1 from Mode-1 to Mode-2 (refer to Figure 6.5). By analyzing the control logic and source code of PLC1, the adversary inferred that this state transition is driven by a sensitive message sent by the SCADA to PLC1. The sensitive message indicates that the remaining five sub-processes of the plant are ready for operation. The adversary also identified 12 other non-sensitive messages that are exchanged between PLC1 and the other PLCs while PLC1 is in Mode-1. The adversary also extracted the sequence in which these messages are communicated.

Next, the adversary collected encrypted traffic samples for each of the 13 messages (1 sensitive and 12 non-sensitive), using a dummy client-server setup that uses the same communication protocols as the test-bed. The encrypted traffic feature that could classify the sensitive and non-sensitive traffic samples with the highest accuracy (100%) was **(i)** *sequence of encrypted packet sizes from PLC1* when the test-bed used CIP over ENIP with TLS, and; **(ii)** *a combination of inter-packet timing interval from PLC1 and*

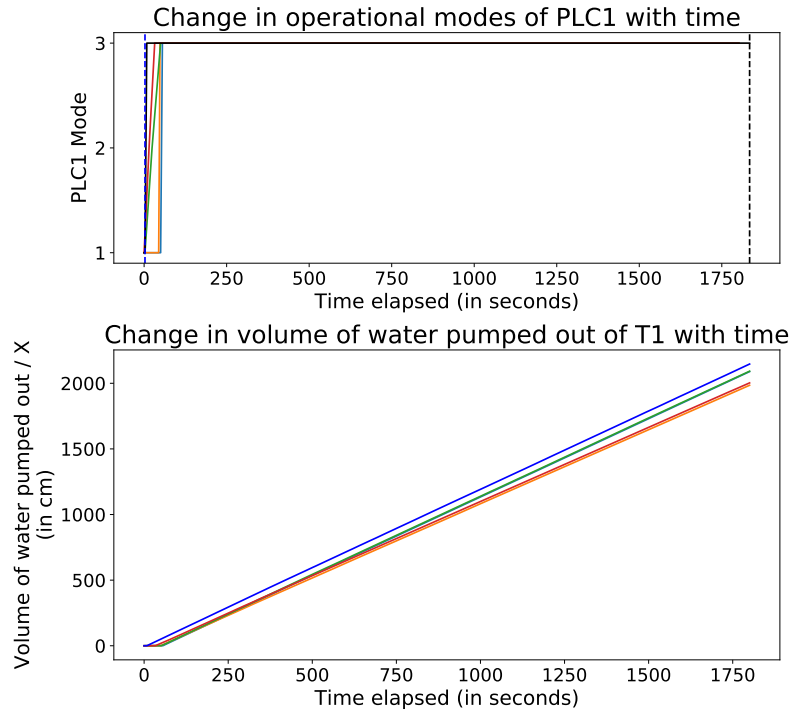


Figure 6.7: Effect of network perturbations by the adversary in Scenario 1, during transition of PLC1 from Mode-1 to Mode-2 (X: T1 cross-section)

number of outgoing packets from PLC1 per second the test-bed used MODBUS/TCP. The adversary used these features for fingerprinting the sensitive messages and the three operational modes of PLC1.

During the active attack phase, the adversary could identify the sensitive messages with 100% accuracy for both the protocols (CIP/ENIP and MODBUS/TCP). Figure 6.7 shows the impact of network perturbations on Sub-process 1 when the adversary either dropped or delayed the sensitive packets for varying amounts of time, when our emulated ICS setup was run for 30 minutes. The adversary successfully delayed the mode transition of PLC1 for both the CIP/ENIP and Modbus/TCP setup. In terms of impact (i.e., the amount of reduction in throughput), packet drops were more effective than packet delays.

The maximum impact that we could observe was a 7.5% reduction in the volume of water pumped out of T1 at the end of 30 minutes, when the adversary introduced packet drops

Attack 2: Overflowing Tank T1. The attack mechanism was similar to Attack 1. In this attack, the objective of the adversary was to disrupt the operations of MV101 and thereby cause Tank T1 to overflow. For this, the adversary needs to delay the transition of Sub-process 1 from Mode-3 to Mode-2 (refer to Figure 6.5). This transition is caused by the message sent by PLC3 to PLC1 that indicates Tank T3 is full. In this case, the adversary identified one sensitive message and 14 non-sensitive messages. The adversary fingerprinted the sensitive message and the process states using the same encrypted traffic features used in Attack 1. During the active attack phase, the adversary was able to identify the operational mode of PLC1 as well as the sensitive packets with 100% accuracy for both the protocols (CIP/ENIP and MODBUS/TCP).

Figure 6.8 shows the impact of the attack on Sub-process 1, when our emulated ICS setup was run for 40 minutes. We can observe that for both CIP/ENIP and Modbus/TCP protocols, the adversary was successful in making tank T1 overflow. It is also noteworthy that the adversary was successful in causing T1 to overflow by dropping packets for only 10 seconds. We further observed that the extent of impact of the attack varied across different types of network perturbations. In this case, the extent of the impact can be measured by the volume of water that overflows before PLC1 switches to Mode-2. We observed that the impact caused by packet delays was lesser than that caused by packet drops (as is expected).

The maximum impact that we could observe was an overflow of 6.5% more water than the threshold, caused by packet drops for 45 seconds, which resulted in 0.006% of the packets getting dropped and 1.8% of the packets being delayed. Our ETA-based attack is as effective as the random perturbation-based attack that caused T1 to overflow (Case-7 in § 6.4.3.1), but 50× stealthier.

Thus, an adversary with a high prediction accuracy can cause significant damage to an ICS in an extremely stealthy manner by dropping/delaying only a small number of targeted packets.

6.4.3.3 Scenario 2 - ICS design and implementation details NOT known to adversary:

In this scenario, the adversary first recorded multiple instances of encrypted ICS traffic corresponding to one hour of operations each, for both CIP/ENIP and Modbus versions of our test-bed. Next, it analysed the recorded flows using the techniques described in § 6.3.3.2.

For the CIP/ENIP protocol, the adversary selected the patterns corresponding to the feature *sequence of encrypted packet sizes from PLC1*. Using this feature, the adversary discovered three different patterns that occur in a repetitive periodic manner. For instance, over a period of one hour, the first pattern (number of packets = 2) always occurs 7 times, followed by the second pattern (number of packets = 16) occurring 730 times, followed by the third pattern (number of packets = 15) occurring 245 times, on an average. For each pattern, the adversary considers the packets at the last repetition in each period to be sensitive, and attempts to drop or delay them. For the Modbus/TCP version of our test-bed, it selected the pattern generated by the feature *inter-packet timing interval from PLC1*. However, for this protocol, the adversary discovered only two repetitive sub-sequences. Furthermore, for both CIP/ENIP and MODBUS/TCP setups, the adversary observed a very small fraction of packets that were much larger than the remaining packets. For instance, over a one-hour period, the adversary observed 3 packets that are 35% larger in size than all the other packets. These packets occurred intermittently, without any specific temporal pattern.

After discovering the patterns, the adversary adopted the following strategies to disrupt the ICS operations.

Attack 1: Preventing change in network patterns. The adversary had an average state transition prediction accuracy of $\approx 99\%$ for both encrypted CIP/ENIP and Modbus/TCP traffic. Figure 6.10 shows the impact on Sub-process 1 over a time period of 40 minutes when the adversary dropped packets for 90 seconds whenever it expected a transition

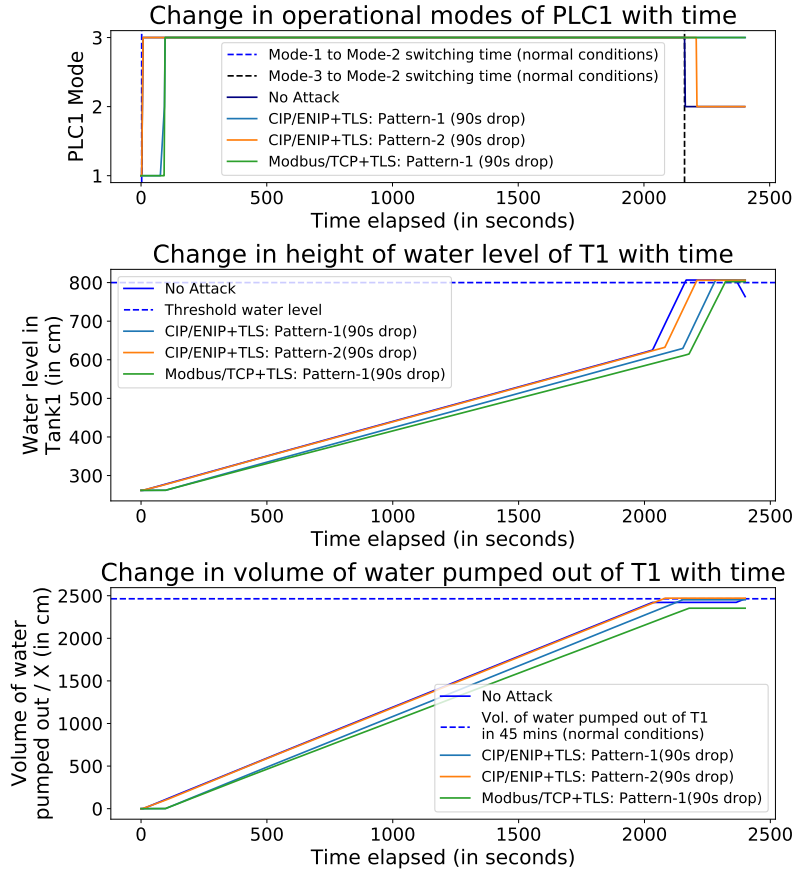


Figure 6.10: Effect of packets dropped by the adversary in Scenario 2 (X: T1 cross-section)

in traffic patterns. In case of the CIP/ENIP setup, for the first pattern transition, we observe that dropping packets delayed the transition of PLC1 from Mode-1 to Mode-2, which caused a delay in the ICS operations. Therefore, the water level in T1 reached the threshold after a delay of ≈ 4 minutes, effectively resulting in a diminished output.

For the second pattern transition, we did not observe any significant impact on the ICS operations due to dropping packets. However, there was a slight drop in the rate at which T1 was filled, resulting in a delay in transition from Mode-3 to Mode-2.

In the case of the Modbus/TCP setup, even though the adversary discovered only two patterns, we observed a significant attack impact when the adversary dropped packets for 90 seconds. This delayed the transition of PLC1 from Mode-1 to Mode-2, thereby

reducing the output of T1 by $\approx 5.4\%$ by the end of 30 minutes. This attack resulted in only 0.002% of the packets getting dropped, and only 5% of the packets getting delayed. Therefore, this attack was as stealthy as Attack 1 in Scenario 1 (refer to § 6.4.3.2), while being almost equally effective.

The success rate of the adversary in Scenario 2 (66.7%) is lower than that in Scenario 1 (100%). However, in cases where the adversary succeeded, its impact and stealthiness were on par with an adversary who is aware of the ICS design and implementation.

Attack 2: Dropping infrequently occurring packets. Dropping the infrequently occurring packets caused PLC1 to stall operations, eventually resulting in an overflow of Tank T1 in ≈ 45 minutes. However, compared to all the other attacks, this attack was the least stealthy. This is because, an entire sub-process getting stalled eventually caused all subsequent network communications to get disrupted. This would invariably trigger a Denial-of-service (DoS) attack alarm at the ICS.

On closer inspection, we observed that these packets contained service discovery information encapsulated in MDNS messages. In a real ICS, similar communications can take place using encrypted DNS traffic (Hu *et al.*, 2016). This attack would still succeed in identifying such packets, because we observed that the DNS packets generated by the SWaT test-bed (SUTD iTrust, n.d.) would also be 25% larger than the other packets when encrypted.

6.5 POSSIBLE COUNTERMEASURES

An adversary such as the one we proposed can be detected neither at the ICS end systems, nor by a conventional intrusion detection system. Using the side-channel information, the adversary can introduce carefully crafted network perturbations that are comparable to natural network fluctuations, yet it can potentially drop/delay sensitive communications and disrupt the ICS operations. This makes it difficult to detect and mitigate such attacks.

We discuss some of the potential countermeasures and the challenges associated with their practical implementation in ICS.

1. *Ensuring availability of redundant adversary-free network paths* – Besides being an expensive solution, ensuring that the redundant paths are free from compromised devices is difficult in the presence of a rogue ISP. While it is intuitive to have redundant ISP connections to make critical infrastructures fail-safe, it might not *always* guarantee resilience under adversarial conditions. This is because of the following reasons.

- a) Both the original and redundant ISPs used by an ICS might be connected to a common higher-tier ISP. A powerful adversary might position itself at a network node on the higher-tier ISP. In that case, switching to the redundant ISP might not guarantee an adversary-free path. This situation might arise in the case of ICS networks connected to lower-tier ISPs which need to transmit their traffic through a higher-tier ISP.
- b) Even if an ICS has redundant ISP connections, it would be difficult for an ICS to decide when to switch to the redundant connection. This is because, *first*, most ICSes are resilient to small-scale network perturbations. Therefore, a small number of packet drops or delays are not considered to be a network failure; and, *second*, when an ICS exhibits abnormal behavior, state-of-the-art ICS security protocols try to detect the point of the attack within the ICS network. To the best of our knowledge, there exists no mechanism for correlating small number of packet drops outside the ICS perimeter with possible reasons for failure.

2. *Communication over anonymized channels such as Tor (Tor, n.d.)* – Such solutions would prevent an adversary from identifying the traffic of the targeted ICS on the Internet, unless it is capable of compromising the Tor guard node (Platzer *et al.*, 2020). However, anonymized communications have a high latency, which makes them unsuitable for time-sensitive communications, such as those used in an ICS OT network.

3. *Obfuscation of network side-channels* – Popular techniques (Dyer *et al.*, 2012) used in the context of web-browsing privacy include padding of packets and randomization of traffic patterns by introducing spurious packets in the network. However, these solutions often incur high computational and network delays (Dyer *et al.*, 2012), and are not suitable for time-sensitive safety-critical systems.

4. *Application Layer countermeasures* – Developing a mechanism for measuring network side-channel leakage by inspecting the application design and implementation would be useful for designing new ICS applications. However, it might not be practical to implement changes at the application layer for legacy ICS systems.

5. *Correlating network failures and affected ICS operations* – Correlating repeated network outages with the sensitivity of the ICS communications affected might provide valuable insights regarding the nature of the network outage. Such correlation techniques would aid the development of stronger intrusion detection systems (IDS). However, developing such an IDS would require a lot of data points, which means the ICS has to withstand a significant number of network outages before its IDS can be strengthened. A potential workaround for this would be to use a honeypot that mimics the original ICS communications. A honeypot can be used to check the sanity of the network path prior to sending the original sensitive ICS messages. If the honeypot experiences network fluctuations, the timing of the fluctuations can be correlated with the sensitivity of the honeypot messages for detecting the presence of an adversary on the path. It would still be a challenge to ensure that the honeypot messages follow the exact same path as the ICS.

6.6 DISCUSSIONS

We now discuss the insights we obtained from our case study.

Generality of the proposed technique. A network adversary can use the proposed

technique if (1) The goal of the attack is achievable by disrupting state transitions, even if it requires disrupting multiple sub-processes simultaneously. However, the proposed technique cannot be used for achieving goals that require manipulation of data that is not communicated over the Internet, and; (2) There exists at least one network side-channel that can differentiate sensitive and non-sensitive messages accurately when encrypted.

Impact of communication protocol on network side-channel leakage. Our experiments show that the communication protocol used by the targeted ICS affects the network side-channel leakage. For instance, we observed that while *sequence of encrypted packet sizes from PLC1* resulted in the highest prediction accuracy for the CIP/ENIP protocol, it did not reveal any information in case of Modbus/TCP protocol. We found that this is due to the different representations of the data and commands in different protocols. For instance, CIP/ENIP uses named data tags of variable length to represent different pieces of information exchanged on the Internet. If the difference in length of the tag name is high, the corresponding encrypted packet sizes would also be different. Modbus/TCP, on the other hand, uses fixed sized registers to represent different pieces of information, which results in packets being the same size when encrypted.

Stealthiness of the adversary. Existing works have shown that distinguishing low volumes of network perturbations from natural network fluctuations is extremely challenging on the Internet (Lackner *et al.*, 2018), due to the highly dynamic temporal characteristics of the Internet. Our case study showed that using our technique, an adversary can cause significant damage to an ICS with a small footprint.

Defeating fault tolerance mechanisms. Existing fault tolerance mechanisms (Zhang *et al.*, 2000) have shown high efficiency in cases where ICS messages were dropped/delayed while the ICS was operating within a state. Fault tolerance mechanisms rely either on replicated back-up components that replace failed components, or on data reconstruction techniques in the event of a network failure. Replicated components cannot recover an ICS from the proposed adversarial technique unless the ICS can

discover an alternate network path that is devoid of adversarial control. Data reconstruction based techniques (Sikdar, 2017) would not work if reconstructing the missing message requires updates from other ICS processes over compromised Internet paths. However, these techniques assume that the system is free from adversarial control during data reconstruction. This assumption might not be applicable if the adversary is as powerful as a rogue ISP, which has occurred in practice (Stone-Gross *et al.*, 2009). Furthermore, most ICS fault tolerance mechanisms often fail to handle rare events such as network failures occurring precisely during state transitions.

6.7 SUMMARY

In this chapter, we have shown that an adversary can use encrypted traffic analysis-based attacks to cause physical damages to and ICS or to degrade its performance. We hope that this work motivates ICS security researchers to investigate countermeasures for the network side channel leaks. Since the protocols for ICS security are rapidly being deployed in commercial systems, we believe that the research community must address this issue before it is too late. We have proposed some basic countermeasures, and pointed out the challenges in implementing them in today's ICS systems. The time for us to influence the adoption of these protocols is now, and it is imperative that we open a dialogue with ICS security experts to ensure these issues are addressed.

CHAPTER 7

CONCLUSION

In this thesis, we investigated the vulnerability of four modern real-world web-based applications and systems to encrypted traffic analysis based surveillance. This thesis is motivated by the fact that modern internet-connected systems and applications transmit a massive volume of information that can be sensitive to different stakeholders in the system (e.g., internet users, business organizations, and industries).

To this end, in Chapters 3- 6, we explored four different scenarios where we analysed the encrypted network traffic of modern and diverse internet-connected applications and communication systems, namely, interactive video streaming platform of Netflix; websites hosted on HTTP/2 servers; mass-scale traffic exchanged between a website and its diverse set of users; and, internet-connected control communications of industrial control systems. We proposed encrypted traffic analysis techniques that were able to infer, sometimes with 100% accuracy, sensitive information such as choices made by a viewer while watching interactive Netflix videos, political preferences of users of a survey website, fine-grained web browsing behavior of a large number of visitors of a targeted website, and sensitive control communications of an internet-connected industrial control system. In each chapter, we also discussed the implications of leakage of these sensitive information to network surveillants. Furthermore, we touched upon some basic defense techniques to prevent the leakage of such sensitive information from traffic metadata. We conclude by paving the way for a few directions for future work in the area of ETA, as presented in § 7.2.

7.1 IMPLICATIONS AND POTENTIAL OF THIS THESIS

In this thesis, we showed that modern web applications are vulnerable to encrypted traffic analysis based security and privacy attacks. Modern web applications transmit a rich plethora of information over the internet and this thesis shows how encrypted traffic metadata can be exploited by surveillants to infer information that might affect different groups of internet users. For instance, in Chapter 4, we show how the identity of an image of a webpage might reveal the political inclination of a targeted internet user. On the other hand, Chapter 5 shows how merely the URLs accessed by a large number of website users might reveal the business performance of a targeted organization. Furthermore, this thesis adds an extra dimension to ETA-based attacks by showing how different types of application designs (in the context of Netflix in Chapter 3) and application layer protocols (in the context of ICS networks in Chapter 6) affect information leakage from encrypted traffic.

We believe that this thesis is a stepping stone towards the development of a holistic framework for evaluating the resilience of various web applications from the perspective of various users. An important milestone in this direction would be to extend the Open Web Application Security Project (OWASP) with ETA-based vulnerabilities. Eventually, we can proceed toward developing a framework that can evaluate different types of web applications and grade them based on their vulnerabilities to encrypted traffic analysis. In addition to the techniques proposed in this thesis, building a framework for end-to-end evaluation of applications would require automating the identification of encrypted traffic metadata that can potentially leak different types of sensitive information from a given web application design. Such an evaluation framework would also help web developers design applications that are resilient to ETA-based surveillance. Websites resilient to ETA-based surveillance would benefit both ordinary internet users as well as industries and business organizations.

7.2 LIMITATIONS AND FUTURE WORK

In this section, we state the limitations of this thesis and suggest possible extensions.

Adoption at scale. Adopting the traffic analysis techniques proposed in this thesis for evaluating large-scale real-life systems would need the following: **(i)** An automated technique for generating dummy traffic corresponding to a wide range of operational conditions for any given web application; **(ii)** automating the identification of sensitive information for a given web application; and, **(iii)** automated techniques for identifying encrypted traffic metadata that leaks a given piece of sensitive information from a given web application. This is challenging especially due to the continuous evolution and sophistication of web application designs and communication protocols.

Opaque box analysis of encrypted network traffic. The traffic analysis techniques presented in this thesis rely on the fact that the targeted applications/systems can either be accessed by the surveillants (for e.g., public-facing websites), or use communication protocols and techniques known to the surveillants (for e.g., communication and encryption protocols used by industrial control systems). Therefore, the proposed techniques cannot be used for inferring the behavior of users of applications that are opaque or inaccessible to the surveillants. Such applications/systems include internal web portals of organizations, industrial systems using proprietary communication protocols, etc. Hence, a potential future research direction would be to design application- and protocol-agnostic traffic analysis techniques. Such techniques would require reverse engineering the behavior of the targeted application users from their encrypted traffic metadata.

Assessing the vulnerabilities of other modern traffic security protocols. This thesis demonstrates ETA-based adversarial surveillance on four different types of modern internet-connected communication systems. These works were the first-of-their-kind analyses on these systems. Therefore, we started by analyzing network traffic encrypted with the most widely used encryption standard, i.e., the Transport Layer Security (TLS)

standard. In future, similar analyses could be conducted on systems that use different encryption standards, such as Tor (Tor, n.d.) and VPN (Kaspersky, n.d.). Eventually, such analyses might help web developers choose the encryption standard that is best suited for their applications.

ETA-based surveillance on modern mobile app traffic. Technological advancements in solid-state electronics and telecommunications have facilitated the development of low-cost smartphones and IoT (Internet of Things) devices (Fortune Business Insights, 2022). Consequently, the number of mobile apps have been growing by leaps and bounds. Today, smartphone users use mobile apps for a plethora of services and activities, such as banking, shopping, medical consultation, entertainment, gaming, social media, live streaming, controlling home IoT devices and monitoring wearable electronic devices, to name a few. These applications transmit a huge volume of sensitive personal information of internet users, albeit in an encrypted state.

Existing ETA techniques on encrypted mobile traffic (Aceto *et al.*, 2019; Wu *et al.*, 2021; Aceto *et al.*, 2021) have focused on discerning the application being used by a targeted mobile user, and has not yet been successful in inferring fine-grained in-app user activities. This is because, unlike web applications, the TCP layer and IP layer metadata of mobile traffic do not help the adversary easily discern the application being used by the targeted user (van Ede *et al.*, 2020). While the techniques presented in this thesis are meant for analyzing web traffic metadata, and cannot be directly used for analyzing mobile traffic metadata, it would be interesting to see if ETA techniques based on similar principles can be used in conjunction with existing mobile app based ETA techniques (Aceto *et al.*, 2019; Wu *et al.*, 2021; Aceto *et al.*, 2021) to infer fine-grained in-app activities of targeted users.

Assessing the vulnerabilities of futuristic applications to ETA-based surveillance. The number of futuristic internet-connected applications has been on the rise. This includes a large number of Augmented Reality (Azuma *et al.*, 2001) and Virtual

Reality (Anthes *et al.*, 2016) based applications. To facilitate such applications, newer communication standards such as the 5G (Qualcomm, 2022; Ericsson, 2022, 2020) and 6G (Nokia, 2021) technologies are being developed. Besides entertainment and gaming, these applications would also be used for security-critical purposes such as remote medical surgeries and warfare (Khor *et al.*, 2016; XR Labs, 2022).

These applications would not only transmit a huge volume of highly sensitive information about its users, but also have various operational constraints (such as high sensitivity to data accuracy, low latency and high availability). Furthermore, due to the differences in underlying communication protocols, the encrypted traffic metadata of these systems would also be different from conventional web traffic metadata. Therefore, it would be interesting to explore the vulnerability of these futuristic applications to ETA-based surveillance before deploying them on public domains.

Hardware acceleration for ETA attacks. The ETA-based attacks presented in this thesis rely on software-based techniques that work offline. It would be interesting to explore if similar techniques can be implemented in hardware for online prediction of activities in targeted internet-connected systems. Such attacks would be more powerful than software-based offline privacy attacks. This is because, online prediction of user activities might be used by a network adversary as a precursor for launching stronger attacks in real time (for e.g., DoS attacks and ransomware attacks).

This idea is motivated by the fact that today Cisco uses hardware implementation of ETA techniques for identifying encrypted malware traffic in real time (Cisco, n.d.). It would be interesting to see if similar techniques can be used by a malicious network device manufacturer for designing hardware switches for ETA-based surveillance.

Designing countermeasures against ETA-based surveillance. Designing effective and practical countermeasures against ETA-based surveillance is challenging because of the following reasons: (i) *Degradation of application performance* - While obfuscating the

network metadata that leaks information seems simple at the outset, the simplest solutions often either degrade the application performance, or violate operational constraints of the applications. For instance, privacy solutions such as VPN is not compatible with Netflix traffic, and Tor is compatible with neither Netflix nor ICS traffic; (ii) *Identifying the metadata to be obfuscated* - ETA-based techniques, such as the ones presented in this thesis, can determine if an application is vulnerable. However, determining the exact metadata that are responsible for information leakage is not easy. Moreover, countermeasures designed against one type of ETA technique might result in vulnerabilities to a different ETA technique. Therefore, the countermeasures need to be evaluated repeatedly against different ETA-based techniques to ensure that they are resilient to information leakage; and, (iii) *Understanding the concerns of different stakeholders in a system* - It is important to understand the privacy concerns of different users of a system for designing effective countermeasures. For instance, a solution for preventing information leaks that concern only an organization but not its users, should not require involvement of the users for deployment. Most of the state-of-the-art countermeasures (Papadogiannaki and Ioannidis, 2021) against ETA-based attacks require intervention of both the client and the web server, irrespective of the information they aim to protect. Due to these reasons, till date there is no known effective countermeasure against ETA-based surveillance that has been widely deployed in the real-world. An interesting future research direction would be to exploit the insights provided by the ETA-based techniques for designing effective countermeasures against them.

REFERENCES

1. **Aceto, G., G. Bovenzi, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé** (2021). Characterization and prediction of mobile-app traffic using Markov modeling. *IEEE Transactions on Network and Service Management*, **18**(1), 907–925.
2. **Aceto, G., D. Ciuonzo, A. Montieri, and A. Pescapé** (2019). Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. *IEEE Transactions on Network and Service Management*, **16**(2), 445–458.
3. **Al-Naami, K., A. El-Ghamry, M. S. Islam, L. Khan, B. Thuraisingham, K. W. Hamlen, M. Alrahmawy, and M. Z. Rashad** (2019). Bimorphing: A bi-directional bursting defense against website fingerprinting attacks. *IEEE Transactions on Dependable and Secure Computing*, **18**(2), 505–517.
4. **Alan, H. F.** (2019). Implementation of existing webpage fingerprinting techniques. URL: <https://github.com/hfalan/codaspy19>, Last accessed: Oct 19, 2022.
5. **Alan, H. F. and J. Kaur**, Client diversity factor in HTTPS webpage fingerprinting. *In Proceedings of the Ninth ACM Conference on Data and Application Security and Privacy*. 2019, 279–290.
6. **Andreeva, O., S. Gordeychik, G. Gritsai, O. Kochetova, E. Potseluevskaya, S. I. Sidorov, and A. A. Timorin** (2016). Industrial Control Systems and their misc Availability. URL: https://media.kasperskycontenthub.com/wp-content/uploads/sites/43/2016/07/07190427/KL_REPORT_ICS_Availability_Statistics.pdf, Last accessed: Oct 19, 2022.
7. **Anthes, C., R. J. García-Hernández, M. Wiedemann, and D. Kranzlmüller**, State of the art of virtual reality technology. *In 2016 IEEE aerospace conference*. IEEE, 2016, 1–19.
8. **Antonioli, D. and N. O. Tippenhauer**, MiniCPS: A toolkit for security research on CPS networks. *In Proceedings of the First ACM workshop on cyber-physical systems-security and/or privacy*. 2015, 91–100.
9. **Azuma, R., Y. Baillot, R. Behringer, S. Feiner, S. Julier, and B. MacIntyre** (2001). Recent advances in augmented reality. *IEEE computer graphics and applications*, **21**(6), 34–47.
10. **Bar-Yanai, R., M. Langberg, D. Peleg, and L. Roditty**, Realtime classification for encrypted traffic. *In International Symposium on Experimental Algorithms*. Springer, 2010, 373–385.

11. **Barbieri, G., M. Conti, N. O. Tippenhauer, and F. Turrin**, Assessing the Use of Insecure ICS Protocols via IXP Network Traffic Analysis. *In 2021 International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2021, 1–9.
12. **BBC** (2019). A history of cyber-surveillance. URL: <https://www.sciencefocus.com/science/a-history-of-cyber-surveillance/>, Last accessed: Oct 19, 2022.
13. **Belshe, M., R. Peon, and E. M. Thomson** (2015). Hypertext Transfer Protocol Version 2 (HTTP/2). URL: <https://tools.ietf.org/html/rfc7540>, Last accessed: Oct 19, 2022.
14. **Bird, S., I. Segall, and M. Lopatka**, Replication: why we still can't browse in peace: on the uniqueness and reidentifiability of web browsing histories. *In Sixteenth Symposium on Usable Privacy and Security (SOUPS)*. 2020, 489–503.
15. **Cai, X., R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg**, A systematic approach to developing and evaluating website fingerprinting defenses. *In Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 2014, 227–238.
16. **Cai, X., X. C. Zhang, B. Joshi, and R. Johnson**, Touching from a distance: Website fingerprinting attacks and defenses. *In Proceedings of the 2012 ACM conference on Computer and communications security*. 2012, 605–616.
17. **Callegati, F., W. Cerroni, and M. Ramilli** (2009). Man-in-the-Middle Attack to the HTTPS Protocol. *IEEE Security & Privacy*, 7(1), 78–81.
18. **Chan, R.** (2019). The Cambridge Analytica whistleblower explains how the firm used Facebook data to sway elections. URL: <https://www.businessinsider.in/tech/news/the-cambridge-analytica-whistleblower-explains-how-the-firm-used-facebook-data-to-sway-elections/articleshow/71461113.cms>, Last accessed: Oct 19, 2022.
19. **Chapman, P. and D. Evans**, Automated black-box detection of side-channel vulnerabilities in web applications. *In Proceedings of the 18th ACM conference on Computer and communications security*. 2011, 263–274.
20. **Chen, B., N. Pattanaik, A. Goulart, K. L. Butler-Purpy, and D. Kundur**, Implementing attacks for modbus/TCP protocol in a real-time cyber physical system test bed. *In 2015 IEEE International Workshop Technical Committee on Communications Quality and Reliability (CQR)*. IEEE, 2015, 1–6.
21. **Chen, S., R. Wang, X. Wang, and K. Zhang**, Side-channel leaks in web applications: A reality today, a challenge tomorrow. *In Proceedings of IEEE Symposium on Security and Privacy*. 2010, 191–206.
22. **Chen, Y.** (2020). SWaT Logic. URL: https://github.com/yuqiChen94/Swat_Simulator, Last accessed: Oct 19, 2022.

23. **Cheng, H.** and **R. Avnur** (1998). Traffic analysis of ssl encrypted web browsing. <http://www.cs.berkeley.edu/~daw/teaching/cs261-f98/projects/final-reports/ronathan-heyning.ps>.
24. **Chow, M.-Y.** and **Y. Tipsuwan**, Network-based control systems: A tutorial. *In IECON'01. 27th Annual Conference of the IEEE Industrial Electronics Society (Cat. No. 37243)*, volume 3. IEEE, 2001, 1593–1602.
25. **Cisco** (n.d.). Encrypted Traffic Analytics (ETA). URL: <https://www.cisco.com/c/en/us/solutions/enterprise-networks/enterprise-network-security/eta.html>, Last accessed: Oct 19, 2022.
26. **Cloudflare** (n.d.a). Ping (ICMP) flood DDoS attack. URL: <https://www.cloudflare.com/learning/ddos/ping-icmp-flood-ddos-attack/>, Last accessed: Oct 19, 2022.
27. **Cloudflare** (n.d.b). What is HTTPS? URL: <https://www.cloudflare.com/learning/ssl/what-is-https/>, Last accessed: Oct 19, 2022.
28. **Cui, W., J. Yu, Y. Gong, and E. Chan-Tin**, Realistic Cover Traffic to Mitigate Website Fingerprinting Attacks. *In Proceedings of IEEE International Conference on Distributed Computing Systems (ICDCS)*. 2018, 1579–1584.
29. **Cybersecurity and Infrastructure Security Agency** (2019). Using Caution with USB Drives. URL: <https://www.cisa.gov/uscert/ncas/tips/ST08-001>, Last accessed: Oct 19, 2022.
30. **Danezis, G.** (2010). Traffic Analysis of the HTTP Protocol over TLS. URL: <http://www0.cs.ucl.ac.uk/staff/G.Danezis/papers/TLSanon.pdf>, Last accessed: Oct 19, 2022.
31. **Deri, L.** (2022). nDPI - Open Source Deep Packet Inspection Software Toolkit. URL: <https://github.com/ntop/nDPI>, Last accessed: Oct 19, 2022.
32. **Dewey, C.** (2016). Thousands of people have changed their votes after taking this quiz. Will you? URL: <https://www.washingtonpost.com/news/the-intersect/wp/2016/11/07/thousands-of-people-have-changed-their-votes-after-taking-this-quiz-will-you/>, Last accessed: Oct 19, 2022.
33. **Diffie, W.** and **S. Landau** (2008). Internet Eavesdropping: A Brave New World of Wiretapping. URL: <https://www.scientificamerican.com/article/internet-eavesdropping/>, Last accessed: Oct 19, 2022.
34. **Dubroy, P.** and **R. Balakrishnan**, A study of tabbed browsing among mozilla firefox users. *In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 2010, 673–682.

35. **Dyer, K. P., S. E. Coull, T. Ristenpart, and T. Shrimpton**, Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail. *In 2012 IEEE symposium on security and privacy*. IEEE, 2012, 332–346.
36. **Edwards, J.** (2020). A Brief History of Network Monitoring. URL: <https://www.whatsupgold.com/blog/a-brief-history-of-network-monitoring>, Last accessed: Oct 19, 2022.
37. **Ericsson** (2020). How 5G and Edge Computing can enhance virtual reality. URL: <https://www.ericsson.com/en/blog/2020/4/how-5g-and-edge-computing-can-enhance-virtual-reality>, Last accessed: Oct 19, 2022.
38. **Ericsson** (2022). EE brings AR to consumers with 5G. URL: <https://www.ericsson.com/en/cases/2022/ar-planet-experience>, Last accessed: Oct 19, 2022.
39. **European Network and Information Security Agency** (2011). Protecting Industrial Control Systems. URL: <https://www.enisa.europa.eu/publications/protecting-industrial-control-systems.-recommendations-for-europe-and-member-states>, Last accessed: Oct 19, 2022.
40. **Fauri, D., B. de Wijs, J. den Hartog, E. Costante, E. Zambon, and S. Etalle**, Encryption in ICS networks: A blessing or a curse? *In 2017 IEEE International Conference on Smart Grid Communications (SmartGridComm)*. IEEE, 2017, 289–294.
41. **Federal Trade Commission, USA** (2009). FTC Shuts Down Notorious Rogue Internet Service Provider. URL: <https://www.ftc.gov/news-events/news/press-releases/2009/06/ftc-shuts-down-notorious-rogue-internet-service-provider-3fn-service-specializes-hosting-spam>, Last accessed: Oct 19, 2022.
42. **Fielding, R., J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee** (1999). Hypertext Transfer Protocol – HTTP/1.1. URL: <https://www.rfc-editor.org/rfc/rfc2616>, Last accessed: Oct 19, 2022.
43. **Fortinet** (n.d.). <https://www.fortinet.com/resources/cyberglossary/cyber-warfare>. URL: <https://www.fortinet.com/resources/cyberglossary/cyber-warfare>, Last accessed: Oct 19, 2022.
44. **Fortune Business Insights** (2022). Smartphone Market Size, Share & Covid-19 Impact Analysis. URL: <https://www.fortunebusinessinsights.com/industry-reports/smartphone-market-100308>, Last accessed: Oct 19, 2022.
45. **Galbrun, E., P. Cellier, N. Tatti, A. Termier, and B. Crémilleux**, Mining periodic patterns with a MDL criterion. *In Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2018, 535–551.

46. **Galloway, B.** and **G. P. Hancke** (2012). Introduction to industrial control networks. *IEEE Communications surveys & tutorials*, **15**(2), 860–880.
47. **Gao, W.** and **T. H. Morris** (2014). On cyber attacks and signature based intrusion detection for modbus based industrial control systems. *Journal of Digital Forensics, Security and Law*, **9**(1), 3.
48. **Garimella, K., T. Smith, R. Weiss,** and **R. West**, Political polarization in online news consumption. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 15. 2021, 152–162.
49. **Ghi tte, V.** and **C. Doerr**, Scaling website fingerprinting. In *2020 IFIP Networking Conference (Networking)*. IEEE, 2020, 199–207.
50. **Goel, S., J. Hofman,** and **M. Sirer**, Who does what on the web: A large-scale study of browsing behavior. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 6. 2012, 130–137.
51. **Golbeck, J.** and **E. Norris**, Personality, movie preferences, and recommendations. In *Proceedings of the 2013 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*. 2013, 1414–1415.
52. **Gong, J.** and **T. Wang**, Zero-delay lightweight defenses against website fingerprinting. In *29th USENIX Security Symposium (USENIX Security 20)*. 2020, 717–734.
53. **Gong, X., N. Borisov, N. Kiyavash,** and **N. Schear**, Website detection using remote traffic analysis. In *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2012, 58–78.
54. **Google** (2022). HTTPS encryption on the web. URL: <https://transparencyreport.google.com/https/overview>, Last accessed: Oct 19, 2022.
55. **Gu, X., M. Yang,** and **J. Luo**, A novel website fingerprinting attack against multi-tab browsing behavior. In *2015 IEEE 19th international conference on computer supported cooperative work in design (CSCWD)*. IEEE, 2015, 234–239.
56. **Gualandi, G.** and **E. Casalicchio**, Use of redundancy in the design of a secure software defined industrial control application. In *2019 Sixth International Conference on Software Defined Systems (SDS)*. IEEE, 2019, 102–109.
57. **Hayes, J.** and **G. Danezis**, k-fingerprinting: A robust scalable website fingerprinting technique. In *25th USENIX Security Symposium (USENIX Security 16)*. 2016, 1187–1203.
58. **Heo, J., C. S. Hong, M. S. Choi, S. H. Ju,** and **Y. H. Lim**, Identity-based mutual device authentication schemes for PLC system. In *2008 IEEE International Symposium on Power Line Communications and Its Applications*. IEEE, 2008, 47–51.
59. **Hickman, K. E.** (1995). The SSL Protocol. URL: <https://datatracker.ietf.org/doc/html/draft-hickman-netscape-ssl-00>, Last accessed: Oct 19, 2022.

60. **Holloway, M.** (2015). Slammer Worm and David-Besse Nuclear Plant. URL: <http://large.stanford.edu/courses/2015/ph241/holloway2/>, Last accessed: Oct 19, 2022.
61. **Hong, J.** (2012). The state of phishing attacks. *Communications of the ACM*, **55**(1), 74–81.
62. **Hu, Z., L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman** (2016). Specification for DNS over Transport Layer Security (TLS). URL: <https://datatracker.ietf.org/doc/html/rfc7858>, Last accessed: Oct 19, 2022.
63. **Huang, J. and R. W. White**, Parallel browsing behavior on the web. In *Proceedings of the 21st ACM conference on Hypertext and Hypermedia*. 2010, 13–18.
64. **Husák, M., M. Čermák, T. Jirsík, and P. Čeleda** (2016). HTTPS traffic analysis and client identification using passive SSL/TLS fingerprinting. *EURASIP Journal on Information Security*, **2016**(1), 1–14.
65. **Hussain, A., J. Heidemann, and C. Papadopoulos**, A framework for classifying denial of service attacks. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications*. 2003, 99–110.
66. **Illmer, A.** (2021). Singapore reveals Covid privacy data available to police. URL: <https://www.bbc.com/news/world-asia-55541001>, Last accessed: Oct 19, 2022.
67. **Information Sciences Institute** (1981). INTERNET PROTOCOL. URL: <https://www.rfc-editor.org/rfc/rfc791>, Last accessed: Oct 19, 2022.
68. **Information Sciences Institute of USC** (1981). Transmission Control Protocol. URL: <https://datatracker.ietf.org/doc/html/rfc793>, Last accessed: Oct 19, 2022.
69. **Isidewith** (2022). www.iSideWith.com. URL: <https://www.isidewith.com/>, Last accessed: Oct 19, 2022.
70. **James, F.** (2012). Web Quiz Tells You Which Presidential Candidate Best Fits Your Worldview. URL: <https://www.npr.org/sections/itsallpolitics/2012/07/19/157048377/web-quiz-tells-you-which-presidential-candidate-best-fits-your-worldview>, Last accessed: Oct 19, 2022.
71. **Juarez, M., S. Afroz, G. Acar, C. Diaz, and R. Greenstadt**, A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 2014, 263–274.
72. **Kaspersky** (2018). What is Spyware? URL: <https://www.kaspersky.co.in/resource-center/threats/spyware>, Last accessed: Oct 19, 2022.
73. **Kaspersky** (n.d.). What is VPN? How It Works, Types of VPN. URL: <https://www.kaspersky.com/resource-center/definitions/what-is-a-vpn>, Last accessed: Oct 19, 2022.

74. **Kazlouski, A., T. Marchioro, H. Manifavas, and E. P. Markatos**, I still See You! Inferring Fitness Data from Encrypted Traffic of Wearables. In *HEALTHINF*. 2021, 369–376.
75. **Kent, S. and K. Seo** (2005). Security Architecture for the Internet Protocol. URL: <https://datatracker.ietf.org/doc/html/rfc4301>, Last accessed: Oct 19, 2022.
76. **Khor, W. S., B. Baker, K. Amin, A. Chan, K. Patel, and J. Wong** (2016). Augmented and virtual reality in surgery—the digital surgical environment: applications, limitations and legal pitfalls. *Annals of translational medicine*, **4**(23).
77. **Kirrmann, H., K. Weber, O. Kleineberg, and H. Weibel**, HSR: Zero recovery time and low-cost redundancy for Industrial Ethernet (High availability seamless redundancy, IEC 62439-3). In *2009 IEEE Conference on Emerging Technologies & Factory Automation*. IEEE, 2009, 1–4.
78. **Klein, D.** (2019). 4 of the Most Devastating Data Center Breaches of the Past 5 Years. (And How They Could Have Been Prevented). URL: <https://www.akamai.com/blog/security/recent-data-center-breaches-and-prevention>, Last accessed: Oct 19, 2022.
79. **Königs, P.** (2022). Government Surveillance, Privacy, and Legitimacy. *Philosophy & Technology*, **35**(1), 1–22.
80. **Korkmaz, E., A. Dolgikh, M. Davis, and V. Skormin**, ICS security testbed with delay attack case study. In *MILCOM 2016-2016 IEEE Military Communications Conference*. IEEE, 2016, 283–288.
81. **Krotofil, M., A. A. Cárdenas, B. Manning, and J. Larsen**, CPS: Driving cyber-physical systems to unsafe operating conditions by timing DoS attacks on sensor signals. In *Proceedings of the 30th Annual Computer Security Applications Conference*. 2014, 146–155.
82. **Lackner, C., F. Wilches-Bernal, B. J. Pierre, and D. A. Schoenwald** (2018). A tool to characterize delays and packet losses in power systems with synchrophasor data. *IEEE Power and Energy Technology Systems Journal*, **5**(4), 117–128.
83. **Lassey, B.** (2020). Intent to Remove: HTTP/2 and gQUIC server push. URL: <https://groups.google.com/a/chromium.org/g/blink-dev/c/K3rYLvmQUBY/m/vOWBKZGoAQAJ>, Last accessed: Oct 19, 2022.
84. **Laštovička, M., S. Špaček, P. Velan, and P. Čeleda**, Using TLS fingerprints for OS identification in encrypted traffic. In *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, 1–6.
85. **Le, V. L., I. Welch, X. Gao, and P. Komisarczuk**, Anatomy of drive-by download attack. In *Proceedings of the Eleventh Australasian Information Security Conference-Volume 138*. 2013, 49–58.

86. **Lescisin, M.** and **Q. Mahmoud**, Tools for Active and Passive Network Side-Channel Detection for Web Applications. *In Proceedings of USENIX Workshop on Offensive Technologies (WOOT)*. 2018.
87. **Li, F., J. W. Chung**, and **M. Claypool**, Silhouette: Identifying youtube video flows from encrypted traffic. *In Proceedings of the 28th ACM SIGMM Workshop on Network and Operating Systems Support for Digital Audio and Video*. 2018, 19–24.
88. **Liaropoulos, A. N.** (2016). Reconceptualising cyber security: Safeguarding human rights in the era of cyber surveillance. *International Journal of Cyber Warfare and Terrorism (IJCWT)*, **6**(2), 32–40.
89. **Liberatore, M.** and **B. N. Levine**, Inferring the source of encrypted HTTP connections. *In Proceedings of the 13th ACM conference on Computer and communications security*. 2006, 255–263.
90. **Lin, Q., S. Adepur, S. Verwer**, and **A. Mathur**, TABOR: A graphical model-based approach for anomaly detection in industrial control systems. *In Proceedings of the 2018 on asia conference on computer and communications security*. 2018, 525–536.
91. **Lin, W., S. Reddy**, and **N. Borisov** (2019). Measuring the Impact of HTTP/2 and Server Push on Web Fingerprinting. *Workshop on Measurements, Attacks, and Defenses for the Web*.
92. **Ling, Z., G. Xiao, W. Wu, X. Gu, M. Yang**, and **X. Fu**, Towards an Efficient Defense against Deep Learning based Website Fingerprinting. *In IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, 310–319.
93. **Linux** (n.d.). tc-qdisc. URL: <https://man7.org/linux/man-pages/man8/tc.8.html>, Last accessed: Oct 19, 2022.
94. **Long, M., C.-H. Wu**, and **J. Y. Hung** (2005). Denial of service attacks on network-based control systems: impact and mitigation. *IEEE Transactions on Industrial Informatics*, **1**(2), 85–96.
95. **Luo, X., P. Zhou, E. W. Chan, W. Lee, R. K. Chang**, and **R. Perdisci**, HTTPOS: Sealing Information Leaks with Browser-side Obfuscation of Encrypted Flows. *In Proceedings of NDSS*, volume 11. 2011.
96. **M. Bishop, E.** (2020). Hypertext Transfer Protocol Version 3 (HTTP/3). URL: <https://tools.ietf.org/html/draft-ietf-quic-http-32>, Last accessed: Oct 19, 2022.
97. **Macaskill, E.** and **G. Dance** (2013). NSA Files: Decoded. URL: <https://www.theguardian.com/world/interactive/2013/nov/01/snowden-nsa-files-surveillance-revelations-decoded>, Last accessed: Oct 19, 2022.

98. **Malwarebytes** (2022). Keylogger. URL: <https://www.malwarebytes.com/keylogger>, Last accessed: Oct 19, 2022.
99. **Marshall, A.** (2022). The best Netflix VPN 2022. URL: <https://www.techradar.com/vpn/best-netflix-vpn>, Last accessed: Oct 19, 2022.
100. **Matherly, J.** (2022). Shodan. URL: <https://www.shodan.io/>, Last accessed: Oct 19, 2022.
101. **Maynard, P., K. McLaughlin, and B. Haberler**, Towards Understanding Man-In-The-Middle Attacks on IEC 60870-5-104 SCADA Networks. In *2nd International Symposium for ICS & SCADA Cyber Security Research 2014 (ICS-CSR 2014)* 2. 2014, 30–42.
102. **Mazhar Abbas, M. A. Z.** (2021). Espionage in ancient India. URL: <https://www.thenews.com.pk/tns/detail/834352-espionage-in-ancient-india>, Last accessed: Oct 19, 2022.
103. **Meeker, M.** (2019). Internet Trends 2019. URL: <https://www.bondcap.com/reports/it19>, Last accessed: Oct 19, 2022.
104. **Meenan, P.** (2018). Optimizing HTTP/2 prioritization with BBR and tcp_notsent_lowat. URL: <https://blog.cloudflare.com/http-2-prioritization-with-nginx/>, Last accessed: Oct 19, 2022.
105. **Middleton, S. E., D. C. De Roure, and N. R. Shadbolt**, Capturing knowledge of user preferences: ontologies in recommender systems. In *Proceedings of the 1st international conference on Knowledge capture*. 2001, 100–107.
106. **Miller, B., L. Huang, A. D. Joseph, and J. D. Tygar**, I know why you went to the clinic: Risks and realization of https traffic analysis. In *International Symposium on Privacy Enhancing Technologies Symposium*. Springer, 2014, 143–163.
107. **Mininet** (n.d.). Mininet. URL: <http://mininet.org/>, Last accessed: Oct 19, 2022.
108. **Mitra, G., P. K. Vairam, P. Slpsk, N. Chandrachoodan, and V. Kamakoti** (2019a). Netflix interactive video traffic dataset. Github link: <https://github.com/Gargi-Mitra/SIGCOMM2019-NetflixInteractive.git>, Last accessed: Oct 19, 2022.
109. **Mitra, G., P. K. Vairam, P. Slpsk, N. Chandrachoodan, and V. Kamakoti** (2019b). White Mirror: Leaking Sensitive Information from Interactive Netflix Movies using Encrypted Traffic Analysis. *arXiv preprint arXiv:1903.06475*.
110. **Mitra, G., P. K. Vairam, P. Slpsk, N. Chandrachoodan, and V. Kamakoti**, Depending on HTTP/2 for Privacy? Good Luck! In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2020, 278–285.
111. **Morla, R.**, Effect of Pipelining and Multiplexing in Estimating HTTP/2.0 Web Object Sizes. In *CoRR*. 2017. URL <http://arxiv.org/abs/1707.00641>.

112. **Müller, H.** and **A. Sedley**, HaTS: large-scale in-product measurement of user attitudes & experiences with happiness tracking surveys. *In Proceedings of the 26th Australian Computer-Human Interaction Conference on Designing Futures: the Future of Design.* 2014, 308–315.
113. **Nave, G., J. Rentfrow,** and **S. Bhatia** (2020). We are what we watch: Movie plots predict the personalities of those who “like” them. *PsyArXiv*.
114. **Netflix** (2022a). Interactive content on Netflix. URL: <https://help.netflix.com/en/node/62526>, Last accessed: Oct 19, 2022.
115. **Netflix** (2022b). Watching TV shows and movies through a VPN. URL: <https://help.netflix.com/en/node/114701>, Last accessed: Oct 19, 2022.
116. **Netflix Technology Blog** (2016). Protecting Netflix Viewing Privacy at Scale. URL: <https://netflixtechblog.com/protecting-netflix-viewing-privacy-at-scale-39c675d88f45>, Last accessed: Oct 19, 2022.
117. **Newman, L. H.** (2018). Don’t Trust the VPN Facebook Wants You to Use. URL: <https://www.wired.com/story/facebook-onavo-protect-vpn-privacy/>, Last accessed: Oct 19, 2022.
118. **Newman, L. H.** (2019). Hackers Can Tell What Netflix Bandersnatch Choices You Make. URL: <https://www.wired.com/story/netflix-interactive-bandersnatch-hackers-choices/>, Last accessed: Oct 19, 2022.
119. **Nokia** (2021). 6G explained. URL: <https://www.nokia.com/about-us/newsroom/articles/6g-explained/>, Last accessed: Oct 19, 2022.
120. **ODVA** (2016). Common Industrial Protocol (CIP). URL: <https://www.odva.org/technology-standards/key-technologies/common-industrial-protocol-cip/>, Last accessed: Oct 19, 2022.
121. **Okada, Y., S. Ata, N. Nakamura, Y. Nakahira,** and **I. Oka**, Comparisons of Machine Learning Algorithms for Application Identification of Encrypted Traffic. *In 2011 10th International Conference on Machine Learning and Applications and Workshops*, volume 2. IEEE, 2011, 358–361.
122. **Olejnik, L., C. Castelluccia,** and **A. Janc**, Why johnny can’t browse in peace: On the uniqueness of web browsing history patterns. *In 5th Workshop on Hot Topics in Privacy Enhancing Technologies (HotPETs 2012)*. 2012.
123. **Openvswitch.org** (n.d.). Open vSwitch Manual. URL: <https://www.openvswitch.org/support/dist-docs/ovs-ofctl.8.txt>, Last accessed: Oct 19, 2022.

124. **Pal, S., B. Sikdar, and J. Chow**, Real-time detection of packet drop attacks on synchrophasor data. *In 2014 IEEE International Conference on Smart Grid Communications (SmartGridComm)*. IEEE, 2014, 896–901.
125. **Panchenko, A., F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle**, Website Fingerprinting at Internet Scale. *In Proceedings of NDSS*. 2016.
126. **Panchenko, A., L. Niessen, A. Zinnen, and T. Engel**, Website fingerprinting in onion routing based anonymization networks. *In Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*. 2011, 103–114.
127. **Papadogiannaki, E. and S. Ioannidis** (2021). A survey on encrypted network traffic analysis applications, techniques, and countermeasures. *ACM Computing Surveys (CSUR)*, **54**(6), 1–35.
128. **Platzer, F., M. Schäfer, and M. Steinebach**, Critical traffic analysis on the tor network. *In Proceedings of the 15th International Conference on Availability, Reliability and Security*. 2020, 1–10.
129. **Pollak, M.** (2015). A Short History of Wiretapping. URL: <https://www.nytimes.com/2015/03/01/nyregion/a-short-history-of-wiretapping.html>, Last accessed: Oct 19, 2022.
130. **PrivacyPolicies.com** (2022). <https://www.privacypolicies.com/blog/isp-tracking-you/>. URL: <https://www.privacypolicies.com/blog/isp-tracking-you/>, Last accessed: Oct 19, 2022.
131. **PTI** (2020). Cyber criminals leak personal data of 2.9 cr Indians on dark web for free. URL: <https://economictimes.indiatimes.com/tech/ites/cyber-criminals-leak-personal-data-of-2-9-cr-indians-on-dark-web-for-free/articleshow/75904331.cms>, Last accessed: Oct 19, 2022.
132. **Qualcomm** (2022). Everything you need to know about 5G. URL: <https://www.qualcomm.com/5g/what-is-5g>, Last accessed: Oct 19, 2022.
133. **Rafter, D.** (2021). Are free VPNs safe? 7 things to know before using free VPNs. URL: <https://ca.norton.com/internetsecurity-privacy-are-free-vpns-safe.html#>, Last accessed: Oct 19, 2022.
134. **Rahman, M. S., P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright** (2020). Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks. *Proceedings on Privacy Enhancing Technologies*, **3**, 5–24.
135. **Raman, R. S., L. Evdokimov, E. Wurstrow, J. A. Halderman, and R. Ensafi**, Investigating large scale HTTPS interception in Kazakhstan. *In Proceedings of the ACM Internet Measurement Conference*. 2020, 125–132.

136. **Rao, R.** and **G. Kesidis**, Detecting malicious packet dropping using statistically regular traffic patterns in multihop wireless networks that are not bandwidth limited. *In GLOBECOM'03. IEEE Global Telecommunications Conference (IEEE Cat. No. 03CH37489)*, volume 5. IEEE, 2003, 2957–2961.
137. **Real Time Automation** (n.d.a). An Introduction to MODBUS RTU. URL: <https://www.rtautomation.com/technologies/modbus-rtu/>, Last accessed: Oct 19, 2022.
138. **Real Time Automation** (n.d.b). An Introduction to MODBUS TCP/IP. URL: <https://www.rtautomation.com/technologies/modbus-tcpip/>, Last accessed: Oct 19, 2022.
139. **Real Time Automation** (n.d.c). EtherNet/IP. URL: <https://www.rtautomation.com/technologies/ethernetip/>, Last accessed: Oct 19, 2022.
140. **Reed, A.** and **M. Kranch**, Identifying https-protected netflix videos in real-time. *In Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*. 2017, 361–368.
141. **Rescorla, E.** (2018). The Transport Layer Security (TLS) Protocol Version 1.3. URL: <https://datatracker.ietf.org/doc/html/rfc8446>, Last accessed: Oct 19, 2022.
142. **Rimmer, V., D. Preuveneers, M. Juarez, T. Van Goethem, and W. Joosen**, Automated website fingerprinting through deep learning. *In NDSS*. 2018.
143. **Rinaldi, J. S.** (2019). EtherNet/IP and Transport Layer Security. URL: <https://www.rtautomation.com/rtas-blog/ethernet-ip-and-transport-layer-security/>, Last accessed: Oct 19, 2022.
144. **Robinson, M., K. Jones, and H. Janicke** (2015). Cyber warfare: Issues and challenges. *Computers & security*, **49**, 70–94.
145. **Sanchez, G.** (2019). Man-In-The-Middle Attack Against Modbus TCP Illustrated with Wireshark. *SANS Institute*.
146. **Schneider Electric USA, Inc.** (2018). MODBUS/TCP Security. URL: https://modbus.org/docs/MB-TCP-Security-v21_2018-07-24.pdf, Last accessed: Oct 19, 2022.
147. **Schuster, R., V. Shmatikov, and E. Tromer**, Beauty and the burst: Remote identification of encrypted video streams. *In 26th USENIX Security Symposium (USENIX Security 17)*. 2017, 1357–1374.
148. **Semrush** (n.d.). See your competitors' traffic with Traffic Analytics. URL: <https://www.semrush.com/lp/traffic-analytics-7/en/?kw=semrush>, Last accessed: Oct 19, 2022.

149. **SentinelOne** (2019). The History of Cyber Security — Everything You Ever Wanted to Know. URL: <https://www.sentinelone.com/blog/history-of-cyber-security/>, Last accessed: Oct 19, 2022.
150. **Seo, Y.-W. and B.-T. Zhang**, Learning user’s preferences by analyzing Web-browsing behaviors. *In Proceedings of the fourth international conference on Autonomous agents*. 2000, 381–387.
151. **Setayeshfar, O., K. Subramani, X. Yuan, R. Dey, D. Hong, I. K. Kim, and K. H. Lee** (2022). Privacy invasion via smart-home hub in personal area networks. *Pervasive and Mobile Computing*, **85**, 101675.
152. **Shahbaz, A.** (2018). The Rise of Digital Authoritarianism. URL: https://freedomhouse.org/sites/default/files/FOTN_2018_Final.pdf, Last accessed: Oct 19, 2022.
153. **Shen, M., Y. Liu, S. Chen, L. Zhu, and Y. Zhang**, Webpage fingerprinting using only packet length information. *In ICC 2019-2019 IEEE International Conference on Communications (ICC)*. IEEE, 2019, 1–6.
154. **Shen, M., Y. Liu, L. Zhu, X. Du, and J. Hu** (2020). Fine-grained webpage fingerprinting using only packet length information of encrypted traffic. *IEEE Transactions on Information Forensics and Security*, **16**, 2046–2059.
155. **Shodan** (2022). Explore: vulnerable routers. URL: <https://www.shodan.io/explore/search?query=vulnerable+routers>, Last accessed: Oct 19, 2022.
156. **Shorman, J. and S. Vockrodt** (2021). Ex-Employee Remotely Hacks Kansas Water Treatment Plant. URL: <https://www.governing.com/security/ex-employee-remotely-hacks-kansas-water-treatment-plant.html>, Last accessed: Oct 19, 2022.
157. **Sikdar, B.**, Spatio-temporal correlations in cyber-physical systems: a defense against data availability attacks. *In Proceedings of the 3rd ACM Workshop on Cyber-Physical System Security*. 2017, 103–110.
158. **Singel, R.** (2008). Leaked Report: ISP Secretly Added Spy Code To Web Sessions, Crashing Browsers. URL: <https://www.wired.com/2008/06/isp-spying-made/>, Last accessed: Oct 19, 2022.
159. **Singer-Vine, J.** (2010). Open This Story in a New Tab. URL: <https://slate.com/human-interest/2010/12/a-new-data-set-from-firefox-reveals-our-browsing-habits.html>, Last accessed: Oct 19, 2022.
160. **Sirinam, P., M. Imani, M. Juarez, and M. Wright**, Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. *In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2018, 1928–1943.

161. **Sirinam, P., N. Mathews, M. S. Rahman, and M. Wright**, Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning. *In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2019, 1131–1148.
162. **Sottek, T. and J. Kopfstein** (2013). Everything you need to know about PRISM. URL: <https://www.theverge.com/2013/7/17/4517480/nsa-spying-prism-surveillance-cheat-sheet>, Last accessed: Oct 19, 2022.
163. **Spillane, J.** (2022). From Edison to Internet: A History of Video Surveillance. URL: <https://www.business2community.com/tech-gadgets/from-edison-to-internet-a-history-of-video-surveillance-0578308>, Last accessed: Oct 19, 2022.
164. **Stanger, J.** (2021). Why DDoS attacks are a major threat to industrial control systems. URL: <https://www.controleng.com/articles/why-ddos-attacks-are-a-major-threat-to-industrial-control-systems/>, Last accessed: Oct 19, 2022.
165. **Stone-Gross, B., C. Kruegel, K. Almeroth, A. Moser, and E. Kirda**, Fire: Finding rogue networks. *In 2009 Annual Computer Security Applications Conference*. IEEE, 2009, 231–240.
166. **Sun, Q., D. R. Simon, Y.-M. Wang, W. Russell, V. N. Padmanabhan, and L. Qiu**, Statistical identification of encrypted web browsing traffic. *In Proceedings of IEEE Symposium on Security and Privacy*. 2002, 19.
167. **SUTD iTrust** (n.d.). Secure Water Treatment (SWaT) Test-bed, SUTD, Singapore. URL: https://itrust.sutd.edu.sg/itrust-labs-home/itrust-labs_swat/, Last accessed: Oct 19, 2022.
168. **Symantec** (2011). W32.Stuxnet Dossier. URL: https://www.wired.com/images_blogs/threatlevel/2011/02/Symantec-Stuxnet-Update-Feb-2011.pdf, Last accessed: Oct 19, 2022.
169. **Syverson, P., R. Dingleline, and N. Mathewson**, Tor: The second generation onion router. *In Usenix Security*, volume 10. 2004.
170. **Tan, M. K. S., S. Goode, and A. Richardson** (2021). Understanding negotiated anti-malware interruption effects on user decision quality in endpoint security. *Behaviour & Information Technology*, **40**(9), 903–932.
171. **Teiss** (2021). Personal data of 533m Facebook users leaked for free on the Dark Web. URL: <https://www.teiss.co.uk/533m-facebook-users-data-leaked-dark-web/>, Last accessed: Oct 19, 2022.

172. **Thomas, R. J.** and **T. Chothia**, Learning from Vulnerabilities-Categorising, Understanding and Detecting Weaknesses in Industrial Control Systems. *In Computer Security*. Springer, 2020, 100–116.
173. **Tor** (2022). Users: Top-10 countries by bridge users. URL: <https://metrics.torproject.org/userstats-bridge-table.html>, Last accessed: Oct 19, 2022.
174. **Tor** (n.d.). The Onion Router. URL: <https://www.torproject.org/>, Last accessed: Oct 19, 2022.
175. **Trevisan, M., S. Traverso, E. Bassi, and M. Mellia** (2019). 4 Years of EU Cookie Law: Results and Lessons Learned. *Proceedings of Privacy Enhancing Technologies*, **2019**(2), 126–145.
176. **United States Army** (1991). United States Army in the World War, 1917–1919. Reports of the Commander-In-Chief, Staff Sections and Services.
177. **Urbina, D., J. Giraldo, N. O. Tippenhauer, and A. Cardenas**, Attacking Fieldbus Communications in ICS: Applications to the SWaT Testbed. *In Proceedings of the Singapore Cyber-Security Conference (SG-CRC) 2016*. IOS Press, 2016, 75–89.
178. **van Ede, T., R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Choffnes, M. van Steen, and A. Peter**, Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. *In Network and Distributed System Security Symposium (NDSS)*, volume 27. 2020.
179. **Verde, N. V., G. Ateniese, E. Gabrielli, L. V. Mancini, and A. Spognardi**, No NAT'd user left behind: Fingerprinting users behind NAT from NetFlow records alone. *In 2014 IEEE 34th International Conference on Distributed Computing Systems*. IEEE, 2014, 218–227.
180. **Wang, K., J. Zhang, G. Bai, R. Ko, and J. S. Dong**, It's Not Just the Site, It's the Contents: Intra-domain Fingerprinting Social Media Websites Through CDN Bursts. *In Proceedings of the Web Conference 2021*. 2021, 2142–2153.
181. **Wang, T., X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg**, Effective attacks and provable defenses for website fingerprinting. *In USENIX Security*. 2014, 143–157.
182. **Wireshark** (n.d.). tshark: Terminal-based Wireshark. URL: https://www.wireshark.org/docs/wsug_html_chunked/AppToolstshark.html, Last accessed: Oct 19, 2022.
183. **Witsch, D., M. Ricken, B. Kormann, and B. Vogel-Heuser**, PLC-statecharts: An approach to integrate umlstatecharts in open-loop control engineering. *In 2010 8th IEEE International Conference on Industrial Informatics*. IEEE, 2010, 915–920.

184. **Wright, C. V., S. E. Coull, and F. Monrose**, Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis. *In Proceedings of NDSS Symposium*, volume 9. 2017.
185. **Wu, H., G. Cheng, and X. Hu**, Inferring ADU Combinations from Encrypted QUIC Stream. *In Proceedings of International Conference on Future Internet Technologies*. 2019, 1–6.
186. **Wu, H., L. Wang, G. Cheng, and X. Hu**, Mobile Application Encryption Traffic Classification Based On TLS Flow Sequence Network. *In 2021 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2021, 1–6.
187. **XR Labs** (2022). Virtual Reality and Augmented Reality Training in Defense. URL: <https://xrlabs.co/virtual-reality-and-augmented-reality-training-in-defense/>, Last accessed: Oct 19, 2022.
188. **Xu, Y., T. Wang, Q. Li, Q. Gong, Y. Chen, and Y. Jiang**, A multi-tab website fingerprinting attack. *In Proceedings of the 34th Annual Computer Security Applications Conference*. 2018, 327–341.
189. **Yan, J. and J. Kaur** (2018). Feature Selection for Website Fingerprinting. *Proceedings of Privacy Enhancing Technologies*, **2018**(4), 200–219.
190. **Yuen, D. M.**, *Deciphering Sun Tzu: How to Read 'The Art of War'*. Oxford University Press, 2014, 110–111.
191. **Zetter, K.** (2015). A Cyberattack Has Caused Confirmed Physical Damage for the Second Time Ever. URL: <https://www.wired.com/2015/01/german-steel-mill-hack-destruction/>, Last accessed: Oct 19, 2022.
192. **Zhang, K., Z. Li, R. Wang, X. Wang, and S. Chen**, Sidebuster: automated detection and quantification of side-channel leaks in web application development. *In Proceedings of the 17th ACM conference on Computer and communications security*. 2010, 595–606.
193. **Zhang, X., S. F. Wu, Z. Fu, and T.-L. Wu**, Malicious packet dropping: how it might impact the TCP performance and how we can detect it. *In Proceedings 2000 International Conference on Network Protocols*. IEEE, 2000, 263–272.
194. **Zhuo, Z., Y. Zhang, Z.-l. Zhang, X. Zhang, and J. Zhang** (2017). Website fingerprinting attack on anonymity networks based on profile hidden markov model. *IEEE Transactions on Information Forensics and Security*, **13**(5), 1081–1095.
195. **Ziebermayr, T. and S. Probst**, Web service authorization framework. *In Proceedings. IEEE International Conference on Web Services, 2004..* IEEE, 2004, 614–621.

CURRICULUM VITAE

NAME Gargi Mitra

CS15D303

DATE OF BIRTH August 30, 1991

EDUCATION QUALIFICATIONS

2013 Bachelor of Technology

Institution	St. Thomas' College of Engineering & Technology, Kolkata, India
Specialization	Computer Science & Engineering

Doctor of Philosophy

Institution	Indian Institute of Technology Madras
Specialization	Computer Science & Engineering
Registration Date	July 15, 2015

DOCTORAL COMMITTEE

Chairperson

Dr. P. Sreenivasa Kumar
Professor, Department of Computer Science & Engineering
Indian Institute of Technology Madras

Guide(s)

Dr. V. Kamakoti
Department of Computer Science & Engineering
Indian Institute of Technology Madras

Dr. Nitin Chandrachoodan
Department of Electrical Engineering
Indian Institute of Technology Madras

Member(s)

Dr. Chester Rebeiro
Department of Computer Science & Engineering
Indian Institute of Technology Madras

Dr. R. Manivasakan
Department of Electrical Engineering
Indian Institute of Technology Madras

Dr. Karthik Raman
Department of Biotechnology
Indian Institute of Technology Madras